KONGUNADU COLLEGE OF ENGINEERING AND TECHNOLOGY

(AUTONOMOUS)

NAMAKKAL- TRICHY MAIN ROAD, THOTTIAM, TRICHY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

20CS503PE – DISTRIBUTED SYSTEM

COURSE HANDOUTS

UNIT 1 COMMUNICATION IN DISTRIBUTED ENVIRONMENT

Introduction - Examples of Distributed Systems - Focusing on resource Sharing- Challenges- API for Internet Protocol - External Data Representation and Marshaling- Multicast communication- Remote Procedure Call-Group Communication- Publish-subscribe systems.

1.1 Introduction:

A distributed system is a collection of independent computers, interconnected via a network, capable of collaborating on a task. Distributed computing is computing performed in a distributed system.

The definition of distributed systems deals with two aspects that:

- **Deals with hardware:** The machines linked in a distributed system are autonomous.
- **Deals with software:** A distributed system gives an impression to the users that theyare dealing with a single system.

Features of Distributed Systems:

- Communication is hidden from users
- Applications interact in uniform and consistent way
- High degree of scalability
- A distributed system is functionally equivalent to the systems of which it is composed.
- Resource sharing is possible in distributed systems.
- Distributed systems act as fault tolerant systems
- Enhanced performance
- No common clock
- Geographical isolation
- Autonomous
- Heterogeneous

Issues in distributed systems

- Concurrency
- Distributed system function in a heterogeneous environment.So adaptability is amajor issue.
- Latency
- Memory considerations: The distributed systems work on both local and sharedmemory.
- Synchronization issues
- Applications must need to adapt gracefully without affecting other parts of the systemsin case of failures.
- Since they are widespread, security is a major issue.
- Limits imposed on scalability
- They are less transparent.
- Knowledge about the dynamic network topology is a must.

QOS parameters

The distributed systems must offer the following QOS:

- Performance
- Reliability
- Availability
- Security

1.2 Examples of Distributed Systems:

Distributed systems encompass many of the most significant technological developments of recent years and hence an understanding of the underlying technology is absolutely central to a knowledge of modern computing.

We now look at more specific examples of distributed systems.

1.2.1 Web search

Web search has emerged as a major growth industry in the last decade. The task of a web search engine is to index the entire contents of the World Wide Web, encompassing a wide range of information styles including web pages, multimedia sources and (scanned) books.

Finance and commerce:

The growth of eCommerce as exemplified by companies such as Amazon and eBay, and underlying payments technologies such as PayPal.

The information society:

The growth of the World Wide Web as a repository of information and knowledge; the development of web search engines such as Google and Yahoo to search this vast repository, the emergence of digital libraries and the large-scale digitization of legacy information sources such as books

Creative industries and entertainment:

The emergence of online gaming as a novel and highly interactive form of entertainment; the availability of music and film in the home through networked media centers and more widely in the Internet via downloadable or streaming content.

Healthcare The growth of health informatics as a discipline with its emphasis on online electronic patient records and related issues of privacy

Education The emergence of e-learning through for example web-based tools such as virtual learning environments, associated support for distance learning; support for collaborative or community-based learning.

Transport and logistics The use of location technologies such as GPS in route finding systems and more general traffic management systems; the modern car itself as an example of a complex distributed system (also applies to other forms of transport such as aircraft), the development of web-based map

services such as MapQuest, Google Maps and Google Earth.

Science The emergence of the Grid as a fundamental technology for eScience, including the use of complex networks of computers to support the storage, analysis and processing of (often very large quantities of) scientific data, the associated use of the Grid as an enabling technology for worldwide collaboration between groups of scientists.

Environmental management The use of (networked) sensor technology to both monitor and manage

the natural environment, for example to provide early warning of natural disasters such as earthquakes, floods or tsunamis and to coordinate emergency response; the collation and analysis of global environmental parameters to better understand complex natural phenomena such as climate change.

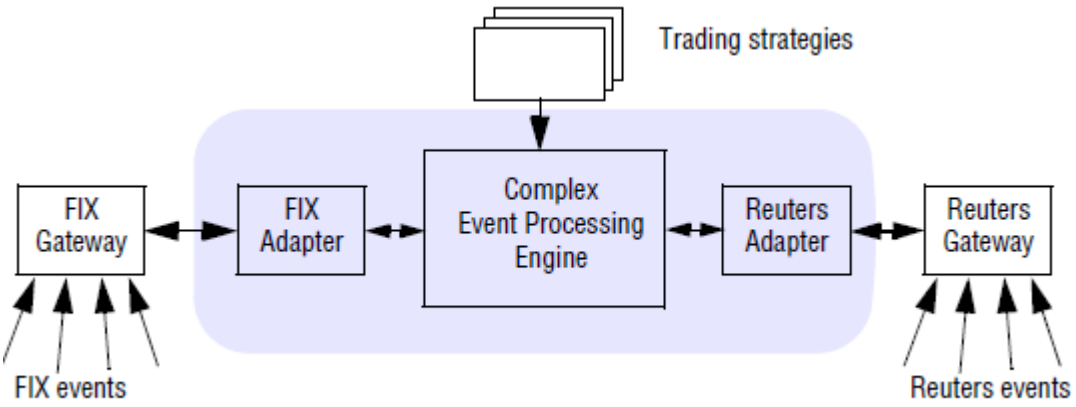
1.2.2 Massively multiplayer online games (MMOGs)

Massively multiplayer online games offer an immersive experience whereby very large numbers of users interact through the Internet with a persistent virtual world, for example EVE, Online consists of a universe with over 5,000 star systems. The number of players is also rising, with systems able to support over 50,000 simultaneous online players

The engineering of MMOGs represents a major challenge for distributed systems technologies, particularly because of the need for fast response times to preserve the user experience of the game.

1.2.3 Financial trading

The financial industry has long been at the cutting edge of distributed systems technology with its need, in particular, for real-time access to a wide range of information sources (for example, current share prices and trends, economic and political developments).



Firstly, the sources are typically in a variety of formats, such as Reuters market data events and FIX events (events following the specific format of the Financial Information eXchange protocol), and indeed from different event technologies, thus illustrating the problem of heterogeneity as encountered in most distributed systems. The figure shows the use of adapters which translate heterogeneous formats into a common internal format. Secondly, the trading system must deal with a variety of event streams, all arriving at rapid rates, and often requiring real-time processing to detect patterns that indicate trading opportunities. This used to be a manual process but competitive pressures have led to increasing automation in terms of what is known as Complex Event Processing (CEP), which offers a way of composing event occurrences together into logical, temporal or spatial patterns.

1.3 Focus on resource sharing

Users are so accustomed to the benefits of resource sharing that they may easily overlook their significance. We routinely share hardware resources such as printers, data resources such as files, and resources with more specific functionality such as search engines.

Looked at from the point of view of hardware provision, we share equipment such as printers and disks to reduce costs. But of far greater significance to users is the sharing of the higher-level resources that play a part in their applications and in their everyday work and social activities. For example, users are concerned with sharing data in the form of a shared database or a set of web pages – not the disks and processors on which they are implemented. Similarly, users think in terms of shared resources such as a search engine or a currency converter, without regard for the server or servers that provide these.

- In practice, patterns of resource sharing vary widely in their scope and in how closely users work together.
- At one extreme, a search engine on the Web provides a facility to users throughout the world, users who need never come into contact with one another directly.
- At the other extreme, in computer-supported cooperative working (CSCW), a group of users who cooperate directly share resources such as documents in a small, closed group.
- We use the term service for a distinct part of a computer system that manages a collection of related resources and presents their functionality to users and applications. For example, we access shared files through a file service.
- The term server is probably familiar to most readers. It refers to a running program (a process) on a networked computer that accepts requests from programs running on other computers to perform a service and responds appropriately.
- The requesting processes are referred to as clients, and the overall approach is known as client-server computing.
- A complete interaction between a client and a server, from the point when the client sends its request to when it receives the server's response, is called a remote invocation.
- Distributed system written in an object-oriented language, resources may be encapsulated as objects and accessed by client objects, in which case we speak of a client object invoking a method upon a server object.

1.4 Challenges:

The scope of distributed systems and to suggest the issues that arise in their design. In many of them, significant challenges were encountered and overcome.

1. Heterogeneity

The Internet enables users to access services and run applications over a heterogeneous collection of computers and networks. Heterogeneity (that is, variety and difference) applies to all of the following:

- Networks,
- Computer hardware,
- Operating systems,
- Programming languages,
- Implementations by different developers.

Data types such as integers may be represented in different ways on different sorts of hardware – for example; there are two alternatives for the byte ordering of integers. These differences in representation must be dealt with if messages are to be exchanged between programs running on different hardware.

Different programming languages use different representations for characters and data structures such as arrays and records. These differences must be addressed if programs written in different languages are to be able to communicate with one another.

Middleware:

The term middleware applies to a software layer that provides a programming abstraction as well as masking the heterogeneity of the underlying networks, hardware, operating systems and programming languages.

In addition to solving the problems of heterogeneity, middleware provides a uniform computational model for use by the programmers of servers and distributed applications.

Heterogeneity and mobile code:

The term mobile code is used to refer to program code that can be transferred from one computer to another and run at the destination – Java applets are an example. Code suitable for running on one computer is not necessarily suitable for running on another because executable programs are normally specific both to the instruction set and to the host operating system.

2. Openness

The openness of a computer system is the characteristic that determines whether the system can be extended and reimplemented in various ways. The openness of distributed systems is determined primarily by the degree to which new resource-sharing services can be added and be made available for use by a variety of client programs.

3. Security

Many of the information resources that are made available and maintained in distributed systems have a high intrinsic value to their users.

Security for information resources has three components:

- Confidentiality (protection against disclosure to unauthorized individuals)
- Integrity (protection against alteration or corruption)
- Availability (protection against interference with the means to access the resources).

4. Scalability

Distributed systems operate effectively and efficiently at many different scales, ranging from a small intranet to the Internet. A system is described as scalable if it will remain effective when there is a significant increase in the number of resources and the number of users. The number of computers and servers in the Internet has increased dramatically.

5. Failure handling

Computer systems sometimes fail. When faults occur in hardware or software, programs may produce incorrect results or may stop before they have completed the intended computation.

The following techniques for dealing with failures:

Detecting failures: Some failures can be detected. For example, checksums can be used to detect corrupted data in a message or a file.

Masking failures: Some failures that have been detected can be hidden or made less severe.

Two examples of hiding failures:

1. Messages can be retransmitted when they fail to arrive.
2. File data can be written to a pair of disks so that if one is corrupted, the other may still be correct.

Tolerating failures: Most of the services in the Internet do exhibit failures – it would not be practical for them to attempt to detect and hide all of the failures that might occur in such a large network with so many components.

6. Concurrency

Both services and applications provide resources that can be shared by clients in a distributed system. There is therefore a possibility that several clients will attempt to access a shared resource at the same time. For example, a data structure that records bids for an auction may be accessed very frequently when it gets close to the deadline time.

7. Transparency

Transparency is defined as the concealment from the user and the application programmer of the separation of components in a distributed system, so that the system is perceived as a whole rather than as a collection of independent components. The implications of transparency are a major influence on the design of the system software.

8. Quality of service

Once users are provided with the functionality that they require of a service, such as the file service in a distributed system, we can go on to ask about the quality of the service provided. The main nonfunctional properties of systems that affect the quality of the service experienced by clients and users are reliability, security and performance.

QoS applies to operating systems as well as networks. Each critical resource must be reserved by the applications that require QoS, and there must be resource managers that provide guarantees.

1.5 The API for the Internet protocols

The characteristics of interprocess communication:

Message passing between a pair of processes can be supported by two message communication operations, send and receive, defined in terms of destinations and messages.

To communicate, one process sends a message (a sequence of bytes) to a destination and another process at the destination receives the message.

This activity involves the communication of data from the sending process to the receiving process and may involve the synchronization of the two processes

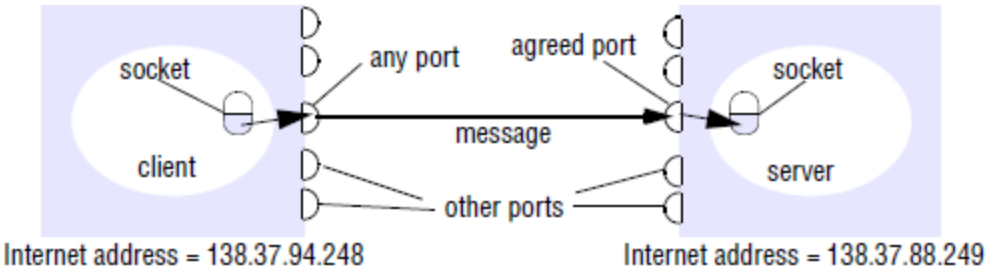
Synchronous and asynchronous communication

• A queue is associated with each message destination. Sending processes cause messages to be added to remote queues and receiving processes remove messages from local queues. Communication between thesending and receiving processes may be either synchronous or asynchronous. In the synchronous form of communication, the sending and receiving processes synchronize at every message. In this case, both send and receive are blocking operations. Whenever a send is issued the sending process (or thread) is blocked until the corresponding receive is issued. Whenever a receive is issued by a process (or thread), it blocks until a message arrives.

Sockets

Both forms of communication (UDP and TCP) use the socket abstraction, which provides an endpoint for communication between processes. Sockets originate from BSD UNIX but are also present in most other versions of UNIX, including Linux as well as Windows and the Macintosh OS. Interprocess communication consists of transmitting a message between a socket in one process and a socket in another process, as illustrated in **Figure** For a process to receive messages, its socket must be bound to a local port and one of the Internet addresses of the computer on which it runs. Messages sent to a particular Internet address and port number can be received only by a process whose socket is associated with that Internet address and port number. Processes may use the same socket for sending and receiving messages. Each computer has a large number (216) of possible port numbers for use by local processes for receiving messages.

Sockets and ports



UDP datagram communication
A datagram sent by UDP is transmitted from a sending process to a receiving process without acknowledgement or retries. If a failure occurs, the message may not arrive. A datagram is transmitted between processes when one process sends it and another receives it.

To send or receive messages a process must first create a socket bound to an Internet address of the local host and a local port. A server will bind its socket to a server port – one that it makes known to clients so that they can send messages to it. A client binds its socket to any free local port.

The following are some issues relating to datagram communication:

Message size: The receiving process needs to specify an array of bytes of a particular size in which to receive a message. If the message is too big for the array, it is truncated on arrival.

Blocking: Sockets normally provide non-blocking sends and blocking receives for datagram communication

Timeouts: The receive that blocks forever is suitable for use by a server that is waiting to receive requests from its clients. But in some programs, it is not appropriate that a process that has invoked a receive **Timeouts:**operation should wait indefinitely in situations where the sending process may have crashed or the expected message may have been lost.

Receive from any: The receive method does not specify an origin for messages. Instead, an invocation of receive gets a message addressed to its socket from any origin.

UDP datagrams suffer from the following failures:
Omission failures: Messages may be dropped occasionally, either because of a checksum error or because no buffer space is available at the source or destination.
Ordering: Messages can sometimes be delivered out of sender order.

Java API for UDP datagrams
• The Java API provides datagram communication by means of two classes: DatagramPacket and DatagramSocket.

DatagramPacket: This class provides a constructor that makes an instance out of an array of bytes comprising a message, the length of the message and the Internet address and local port number of the destination socket, as follows:

Datagram packet

array of bytes containing message	length of message	Internet address	port number

An instance of DatagramPacket may be transmitted between processes when one process sends it and another receives it.

TCP stream communication

The API to the TCP protocol, which originates from BSD 4.x UNIX, provides the abstraction of a stream of bytes to which data may be written and from which data may be read.

The following characteristics of the network are hidden by the stream abstraction:

Message sizes: The application can choose how much data it writes to a stream or reads from it.

Lost messages: The TCP protocol uses an acknowledgement scheme.
If the sender does not receive an acknowledgement within a timeout, it retransmits the message.

Flow control: The TCP protocol attempts to match the speeds of the processes that read from and write to a stream.

Message duplication and ordering: Message identifiers are associated with each IP packet, which enables the recipient to detect and reject duplicates, or to reorder messages that do not arrive in sender order.

Message destinations: A pair of communicating processes establish a connection before they can communicate over a stream. Once a connection is established, the processes simply read from and write to the stream without needing to use Internet addresses and ports.

The following are some outstanding issues related to stream communication:

1.6 External Data Representation and Marshaling.

- Marshalling is the process of taking a collection of data items and assembling them into a form suitable for transmission in a message.
- Unmarshalling is the process of disassembling them on arrival to produce an equivalent collection of data items at the destination.

1. CORBA's Common Data Representation (CDR)

- CORBA CDR is the external data representation defined with CORBA 2.0 .
- CDR can represent all of the data types that can be used as arguments and return values in remote invocations in CORBA.
- These consist of 15 primitive types, which include short (16-bit), long (32-bit), unsigned short, unsigned long, float (32-bit), double (64-bit), char, boolean (TRUE, FALSE), octet (8-bit), and any (which can represent any basic or constructed type)
- **Primitive types:** CDR defines a representation for both big-endian and little-endian orderings. Values are transmitted in the sender's ordering, which is specified in each message.
- **Constructed types:** The primitive values that comprise each constructed type are added to a sequence of bytes in a particular order

CORBA CDR for constructed types

Type	*Representation*
sequence	length (unsigned long) followed by elements in order
string	length (unsigned long) followed by characters in order (can also have wide characters)
array	array elements in order (no length specified because it is fixed)
struct	in the order of declaration of the components
enumerated	unsigned long (the values are specified by the order declared)
union	type tag followed by the selected member

CORBA CDR message

index in sequence of bytes	◄4 bytes ►	*notes on representation*
0–3	5	*length of string*
4–7	"Smit"	*'Smith'*
8–11	"h___"	
12–15	6	*length of string*
16–19	"Lond"	*'London'*
20–23	"on__"	
24–27	1984	*unsigned long*

The flattened form represents a *Person* struct with value: {'Smith', 'London', 1984}

Marshalling in CORBA

- Marshalling operations can be generated automatically from the specification of the types of data items to be transmitted in a message.
- The types of the data structures and the types of the basic data items are described in CORBA IDL, which provides a notation for describing the types of the arguments and results of RMI methods.

2. Java object serialization

In Java RMI, both objects and primitive data values may be passed as arguments and results of method invocations. An object is an instance of a Java class. For example, the Java class equivalent to the Person struct defined in CORBA IDL might be:

```
public class Person implements Serializable
{
```

```
private String name;
    private String place;
    private int year;
    public Person(String aName, String aPlace, int aYear)
    {
    name = aName;
    place = aPlace; year = aYear;
    }
    // followed by methods for accessing the instance variables
}
```

The above class states that it implements the Serializable interface, which has no methods. Stating that a class implements the Serializable interface (which is provided in the java.io package) has the effect of allowing its instances to be serialized.

Indication of Java serialized form

Serialized values				*Explanation*
Person	8-byte version number		h0	*class name, version number*
3	int year	java.lang.String name	java.lang.String place	*number, type and name of instance variables*
1984	5 Smith	6 London	h1	*values of instance variables*

The true serialized form contains additional type markers; h0 and h1 are handles.

Java object serialization uses reflection to find out the class name of the object to be serialized and the names, types and values of its instance variables. That is all that is needed for the serialized form.

3 Extensible Markup Language (XML)

- XML is a markup language that was defined by the World Wide Web Consortium (W3C) for general use on the Web. In general, the term markup language refers to a textual encoding that represents both a text and details as to its structure or its appearance.
- Both XML and HTML were derived from SGML (Standardized Generalized Markup Language), a very complex markup language. HTML was designed for defining the appearance of web pages. XML was designed for writing structured documents for the Web.
- XML data items are tagged with ‘markup’ strings. The tags are used to describe the logical structure of the data and to associate attribute-value pairs with logical structures.

- XML is used to enable clients to communicate with web services and for defining the interfaces and other properties of web services.

  XML definition of the Person structure

```
<person id="123456789">
<name>Smith</name>
<place>London</place>
<year>1984</year>
<!-- a comment -->
</person >
```

4. Remote object references

- This section applies only to languages such as Java and CORBA that support the distributed object model.
- When a client invokes a method in a remote object, an invocation message is sent to the server process that hosts the remote object.

Representation of a remote object reference

32 bits 32 bits 32 bits 32 bits

Internet address	port number	time	object number	interface of remote object

To allow remote objects to be relocated into a different process on a different computer, the remote object reference should not be used as the address of the remote object.

1.7 Multicast communication

- The pairwise exchange of messages is not the best model for communication from one process to a group of other processes, which may be necessary, for example, when a service is implemented as a number of different processes in different computers, perhaps to provide fault tolerance or to enhance availability.
- A multicast operation is more appropriate – this is an operation that sends a single message from one process to each of the members of a group of processes, usually in such a way that the membership of the group is transparent to the sender.

1.8 Remote Procedure Call

1 Design issues for RPC

Programming with interfaces

- Most modern programming languages provide a means of organizing a program as a set of modules that can communicate with one another.
- Communication between modules can be by means of procedure calls between modules or by direct access to the variables in another module.
- In order to control the possible interactions between modules, an explicit interface is defined for each module.
- The interface of a module specifies the procedures and the variables that can be

accessed from other modules.

- Modules are implemented so as to hide all the information about them except that which is available through its interface.

RPC call semantics:

•Request-reply protocols showed that doOperation can be implemented in different ways to provide different delivery guarantees.

The main choices are:

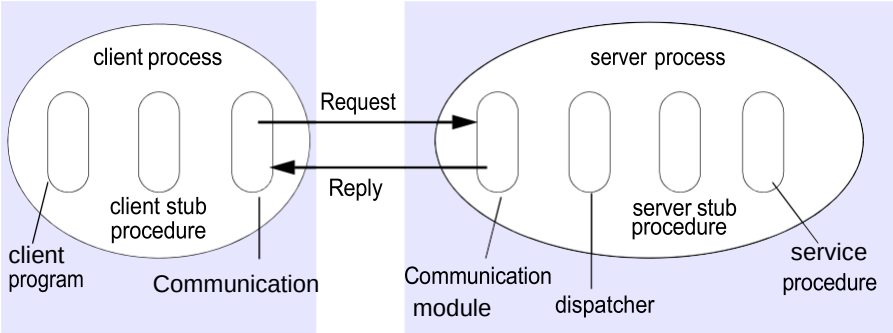
- **Retry request message:** Controls whether to retransmit the request message until either a reply is received or the server is assumed to have failed.
- **Duplicate filtering:** Controls when retransmissions are used and whether to filter out duplicate requests at the server.
- **Retransmission of results:** Controls whether to keep a history of result messages to enable lost results to be retransmitted without re-executing the operations at the server.

Transparency

- The originators of RPC, to make remote procedure calls as much like local procedure calls as possible, with no distinction in syntax between a local and a remote procedure call.
- All the necessary calls to marshalling and message-passing procedures were hidden from the programmer making the call.
- Although request messages are retransmitted after a timeout, this is transparent to the caller to make the semantics of remote procedure calls like that of local procedure calls.

Implementation of RPC

- The software components required to implement RPC are shown in the Figure. The client that accesses a service includes one stub procedure for each procedure in the service interface.
- The stub procedure behaves like a local procedure to the client, but instead of executing the call, it marshals the procedure identifier and the arguments into a request message, which it sends via its communication module to the server.
- When the reply message arrives, it unmarshals the results.
- The server process contains a dispatcher together with one server stub procedure and one service procedure for each procedure in the service interface.
- The dispatcher selects one of the server stub procedures according to the procedure identifier in the request message.
- The server stub procedure then unmarshals the arguments in the request message, calls the corresponding service procedure and marshals the return values for the reply message.
- RPC is generally implemented over a request-reply protocol.



Role of client and server stub procedures in RPC

1.8 Group communication

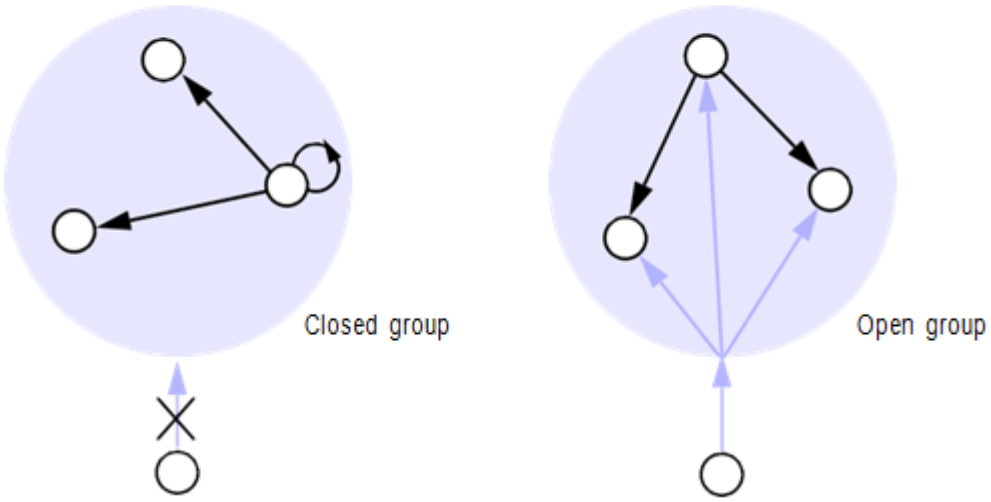
- Group communication provides our first example of an indirect communication paradigm.
- Group communication offers a service whereby a message is sent to a group and then this message is delivered to all members of the group.
- In this action, the sender is not aware of the identities of the receivers.
- Group communication represents an abstraction over multicast communication and may be implemented over IP multicast or an equivalent overlay network, adding significant extra value in terms of managing group membership, detecting failures and providing reliability and ordering guarantees. With the added guarantees, group communication is to IP multicast what TCP is to the point-to-point service in IP.
- Group communication is an important building block for distributed systems, and particularly reliable distributed systems, with key areas of application including:
  - the reliable dissemination of information to potentially large numbers of clients, including in the financial industry, where institutions require accurate and up-to-date access to a wide variety of information sources;
  - support for collaborative applications, where again events must be disseminated to multiple users to preserve a common user view – for example, in multiuser games
  - support for a range of fault-tolerance strategies, including the consistent update of replicated data or the implementation of highly available servers.
  - support for system monitoring and management, including for example load balancing strategies.

Process groups and object groups :

- Most work on group services focuses on the concept of process groups, that is, groups where the communicating entities are processes. Such services are relatively low-level in that:
- Messages are delivered to processes and no further support for dispatching is provided.
- Messages are typically unstructured byte arrays with no support for marshalling of complex data types
- **Closed and open groups:** A group is said to be closed if only members of the group may multicast to it. A process in a closed group delivers to itself any message that it multicasts to the group. A group is open if processes outside the group may send to it. (The categories ‘open’ and ‘closed’ also apply with analogous meanings to mailing lists.) Closed groups of processes are useful, for example, for cooperating servers to send messages to one another that only they should receive. Open groups are useful, for

example, for delivering events to groups of interested processes.

Open and closed groups



- **Overlapping and non-overlapping groups**: In overlapping groups, entities (process-es or objects) may be members of multiple groups, and non-overlapping groups im- ply that membership does not overlap (that is, any process belongs to at most one group). Note that in real-life systems, it is realistic to expect that group membership will overlap.
- **Synchronous and asynchronous systems:** There is a requirement to consider group communication in both environments.

Implementation issues

Reliability and ordering in multicast

- In group communication, all members of a group must receive copies of the messages sent to the group, generally with delivery guarantees.
- Reliability in one-to-one communication was defined in terms of two properties: integrity (the message received is the same as the one sent, and no messages are delivered twice) and validity (any outgoing message is eventually delivered).
- group communication services offer ordered multicast, with the option of one or more of the following properties
    - FIFO ordering: First-in-first-out (FIFO) ordering (also referred to as source ordering) is concerned with preserving the order from the perspective of a sender process, in that if a process sends one message before another, it will be delivered in this order at all processes in the group.
    - Causal ordering: Causal ordering takes into account causal relationships between messages, in that if a message happens before another message in the distributed system this so-called causal relationship will be preserved in the delivery of the associated messages at all processes (see Chapter 14 for a detailed discussion of the meaning of ‘happens before’).
    - Total ordering: In total ordering, if a message is delivered before another message at one process, then the same order will be preserved at all processes.

1.10 Publish-subscribe systems

- A publish-subscribe system is a system where publishers publish structured events to an event service and subscribers express interest in particular events through subscriptions which can be arbitrary patterns over the structured events.
- A subscriber could express an interest in all events related to this textbook, such as the availability of a new edition or updates to the related web site.
- The task of the publish- subscribe system is to match subscriptions against published events and ensure the correct delivery of event notifications.

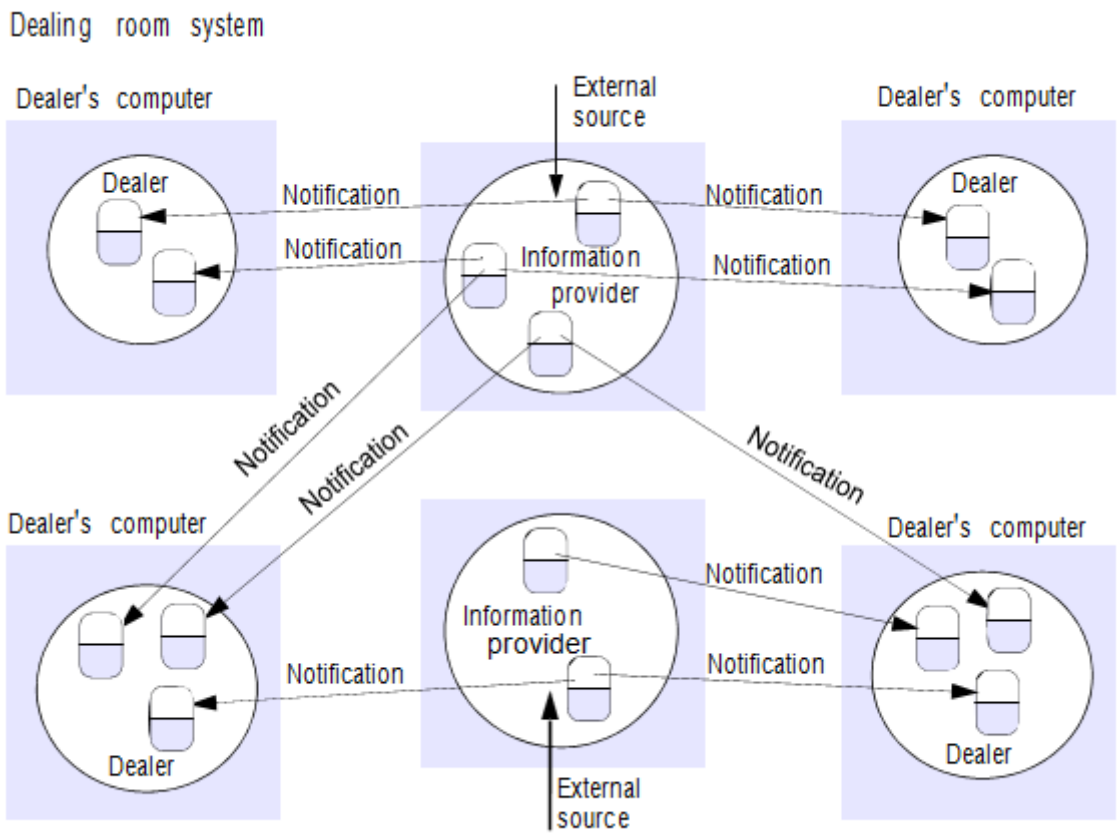
Applications of publish-subscribe systems

Publish-subscribe systems are used in a wide variety of application domains, particularly those related to the large-scale dissemination of events.

- Financial information systems.
- Other areas with live feeds of real-time data.
- Support for cooperative working, where a number of participants need to be informed of events of shared interest;
- Support for ubiquitous computing, including the management of events emanating from the ubiquitous infrastructure (for example, location events);
- A broad set of monitoring applications, including network monitoring in the Internet.

Dealing room system: Consider a simple dealing room system whose task is to allow dealers using computers to see the latest information about the market prices of the stocks they deal in.

- Dealers are typically interested only in their own specialist stocks. A dealing room system could be implemented by processes with two different tasks:
- An information provider process continuously receives new trading information from a single external source. Each of the updates is regarded as an event. The information provider publishes such events to the publish-subscribe system for delivery to all of the dealers who have expressed an interest in the corresponding stock. There will be a separate information provider process for each external source.
- A dealer process creates a subscription representing each named stock that the user asks to have displayed. Each subscription expresses an interest in events related to a given stock at the relevant information provider. It then receives all the information sent to it in notifications and displays it to the user. The communication of notifications is illustrated in the Figure .



Characteristics of publish-subscribe systems:

- Heterogeneity:

  components in a distributed system that were not designed to interoperate can be made to work together.

- Asynchronicity:

  Notifications are sent asynchronously by event-generating publishers to all the subscribers

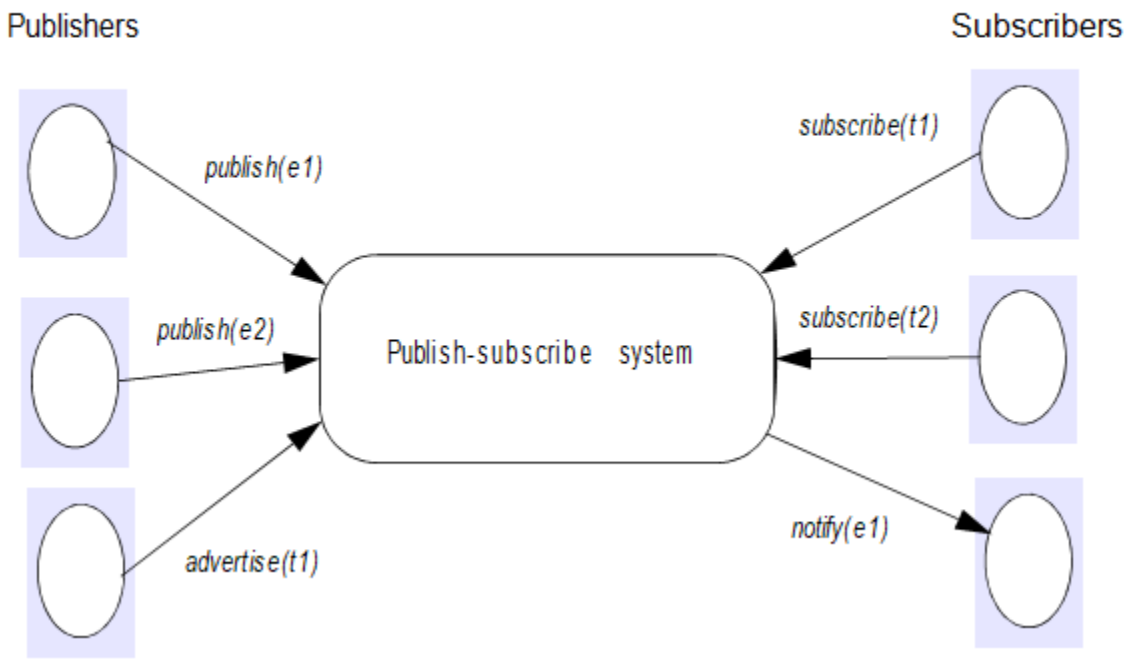
The programming model

- The programming model in publish-subscribe systems is based on a small set of operations, captured in Figure Publishers disseminate an event e through a publish(e) operation and subscribers express an interest in a set of events through subscriptions.

- In particular, they achieve this through a subscribe(f) operation where f refers to a filter – that is, a pattern defined over the set of all possible events.

- Subscribers can later revoke this interest through a corresponding unsubscribe(f) operation. When events arrive at a subscriber, the events are delivered using a notify(e) operation.

The publish-subscribe paradigm

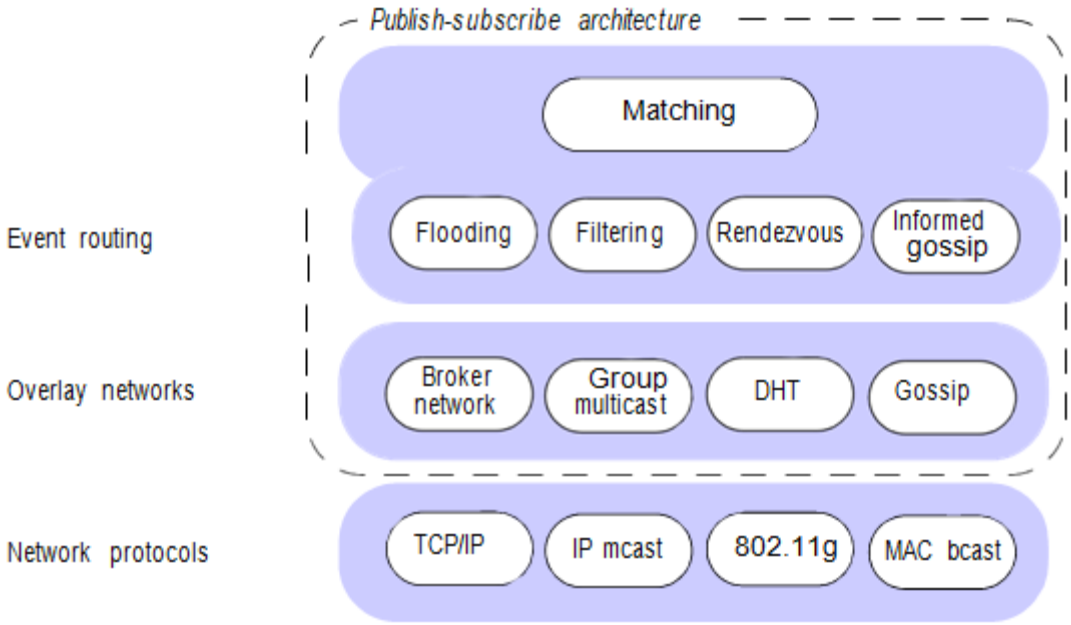


- In other words, subscribers declare their interests in terms of subscriptions and publishers optionally declare the styles of events they will generate through advertisements. Advertisements can be revoked through a call of unadvertise(f).

Overall systems architecture

- As mentioned above, the implementation of centralized schemes is relatively straightforward, with the central service maintaining a repository of subscriptions and matching event notifications with this set of subscriptions.
- Similarly, the implementations of channel-based or topic-based schemes are relatively straightforward. For example, a distributed implementation can be achieved by mapping channels or topics onto associated groups and then using the underlying multicast communication facilities to deliver events to interested parties (using reliable and ordered variants, as appropriate).
- The distributed implementation of content-based (or by extrapolation, type-based) approaches is more complex and deserving of further consideration. The range of architectural choices for such approaches is captured in Figure
- In the bottom layer, publish-subscribe systems make use of a range of interprocess communication services, such as TCP/IP, IP multicast (where available) or more specialized services, as offered for example by wireless networks. The heart of the architecture is provided by the event routing layer supported by a network overlay infrastructure.

The architecture of publish-subscribe systems



- Within this overall architecture, there is a wide variety of implementation approaches. We step through a select set of implementations to illustrate the general principles behind content-based routing:
- Flooding: The simplest approach is based on flooding, that is, sending an event notification to all nodes in the network and then carrying out the appropriate matching at the subsciber end.
- Filtering: One principle that underpins many approaches is to apply filtering in the network of brokers. This is referred to as filtering-based routing. Brokers forward notifications through the network only where there is a path to a valid subscriber.
- Advertisements: The pure filtering-based approach described above can generate a lot of traffic due to propagation of subscriptions, with subscriptions essentially using a flooding approach back towards all possible publishers.
- Rendezvous: Another approach to control the propagation of subscriptions (and to achieve a natural load balancing) is the rendezvous approach.

UNIT II PROCESS AND SYNCHRONIZATION

Processes – Threads - Communication and Invocation - Clocks, Events and Process States - Synchronization Physical Clocks - Logical Time and Logical Clocks - Global States – Distributed Mutual Exclusion - Elections- Distributed Transactions.

2.1 Processes

- Processes play a crucial role in distributed systems.
- The concept of a process originates from the field of operating systems where it is generally defined as a program in execution
- a process consists of an execution environment together with one or more threads. A *thread* is the operating system abstraction of an activity
- As many older operating systems allow only one thread per process, we shall sometimes use the term *multi-threaded process* for emphasis.

Address spaces

- An address space, introduced in the previous section, is a unit of management of a process's virtual memory.
- It is large (typically up to 2^{32} bytes, and sometimes up to 2^{64}bytes) and consists of one or more *regions*, separated by inaccessible areas of virtual memory.
- A region is an area of contiguous virtual memory that is accessible by the threads of the owning process. Regions do not overlap. Note that we distinguish between the regions and their contents.
  - Each region is specified by the following properties:
    - its extent (lowest virtual address and size);
    - read/write/execute permissions for the process's threads;
    - whether it can be grown upwards or downwards.

- The need to share memory between processes, or between processes and the kernel, is another factor leading to extra regions in the address space.
- A *shared memory region* (or *shared region* for short) is one that is backed by the same physical memory as one or more regions belonging to other address spaces.
- The uses of shared regions include the following:
  - ***Libraries*:** Library code can be very large and would waste considerable memory if itwas loaded separately into every process that used it.
  - ***Kernel*:** Often the kernel code and data are mapped into every address space at the same location. When a process makes a system call or an exception occurs, there is no need to switch to a new set of address mappings.
  - ***Data sharing and communication*:** Two processes, or a process and the kernel, mightneed to share data in order to cooperate on some task.

Creation of a new process

- The creation of a new process has traditionally been an indivisible operation provided by the operating system.
- For example, the UNIX *fork* system call creates a process with an execution environment copied from the caller (except for the return value from *fork*).

- The UNIX *exec* system call transforms the calling process into one executing the codeof a named program.

Creation of a new execution environment

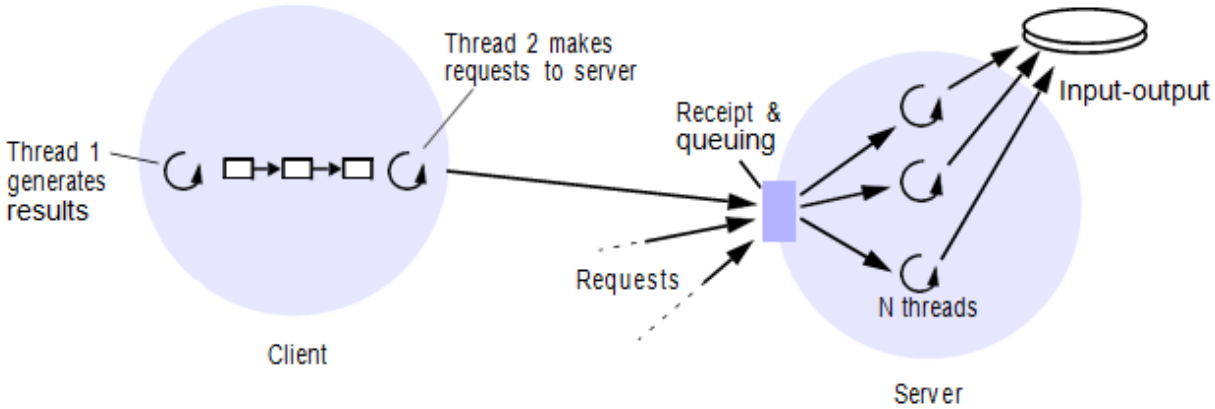
- Once the host computer has been selected, a new process requires an execution environment consisting of an address space with initialized contents (and perhaps other resources, such as default open files).
- There are two approaches to defining and initializing the address space of a newlycreated process.
- The first approach is used where the address space is of a statically defined format. For example, it could contain just a program text region, heap region and stack region. In this case, the address space regions are created from a list specifying their extent. Address space regions are initialized from an executable file or filled withzeros as appropriate.
- Alternatively, the address space can be defined with respect to an existing execution environment. In the case of UNIX *fork* semantics, for example, the newly created child process physically shares the parent's text region and has heap and stack regions that are copies of the parent's in extent

2.2 Threads

- Threads communicate and synchronize with each other using fast shared memory mechanisms. Threads are quite useful for distributed system applications. Many systems interact using a client/server model of communication where a server process listens for client requests, executes them and returns the results.

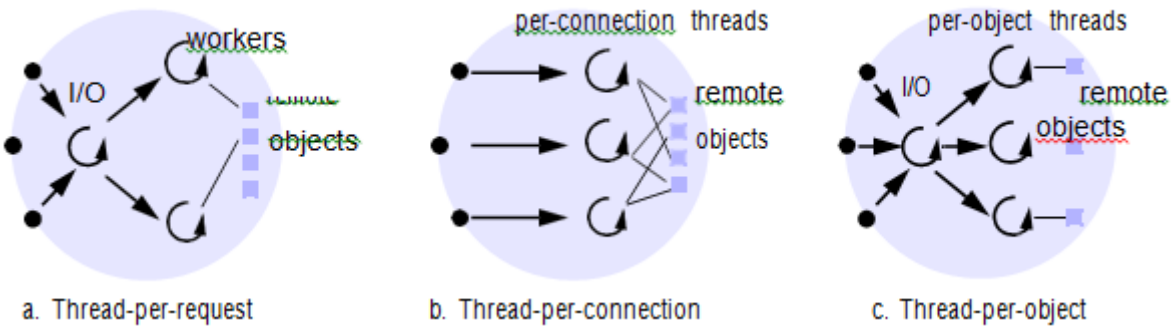
- The server has a pool of one or more threads, each of which repeatedly removes a request from a queue of received requests and processes it.
- We shall not concern ourselves forthe moment with how the requests are received and queued up for the threads. Also, forthe sake of simplicity, we assume that each thread applies the same procedure to processthe requests.
- Let us assume that each request takes, on average, 2 milliseconds of processing plus 8 milliseconds of I/O (input/output) delay when the server reads from a disk (there is no caching). Let us further assume for the moment that the server executes at a single-processor computer.
- Consider the maximum server throughput, measured in client requests handled per second, for different numbers of threads. If a single thread has to perform all processing, then the turnaround time for handling any request is on average 2 + 8 = 10 milliseconds, so this server can handle 100 client requests per second.

Client and server with threads



Architectures for multi-threaded servers

- We have described how multi-threading enables servers to maximize their throughput, measured as the number of requests processed per second.
- Figure shows one of the possible threading architectures, the worker pool architecture. In its simplest form, the server creates a fixed pool of 'worker' threads to process the requests when it starts up. The module marked 'receipt and queuing' in Figure is typically implemented by an 'I/O' thread, which receives requests from a collection of sockets or ports and places them on a shared request queue for retrieval bythe workers.



- In the thread-per-request architecture the I/O thread spawns a newworker thread for each request, and that worker destroys itself when it has processed therequest against its designated remote object.
- This architecture has the advantage that thethreads do not contend for a shared queue, and throughput is potentially maximized because the I/O thread can create as many workers as there are outstanding requests. Itsdisadvantage is the overhead of the thread creation and destruction operations.
- The thread-per-connection architecture associates a thread with each connection. The server creates a new worker thread when a client makes a connection and destroys the thread when the client closes the connection. In between, the client may make many requests over the connection, targeted at one or more remote
- The thread-per-object architecture associates a thread with each remote object. An I/O thread receives requests and queues them for the workers, but this time there is a per-object queue.

2.3 Communication and invocation

Communication primitives

Some kernels designed for distributed systems have provided communication primitives tailored to the types of invocation.

For example, provides doOperation, getRequest and sendReply as primitives.

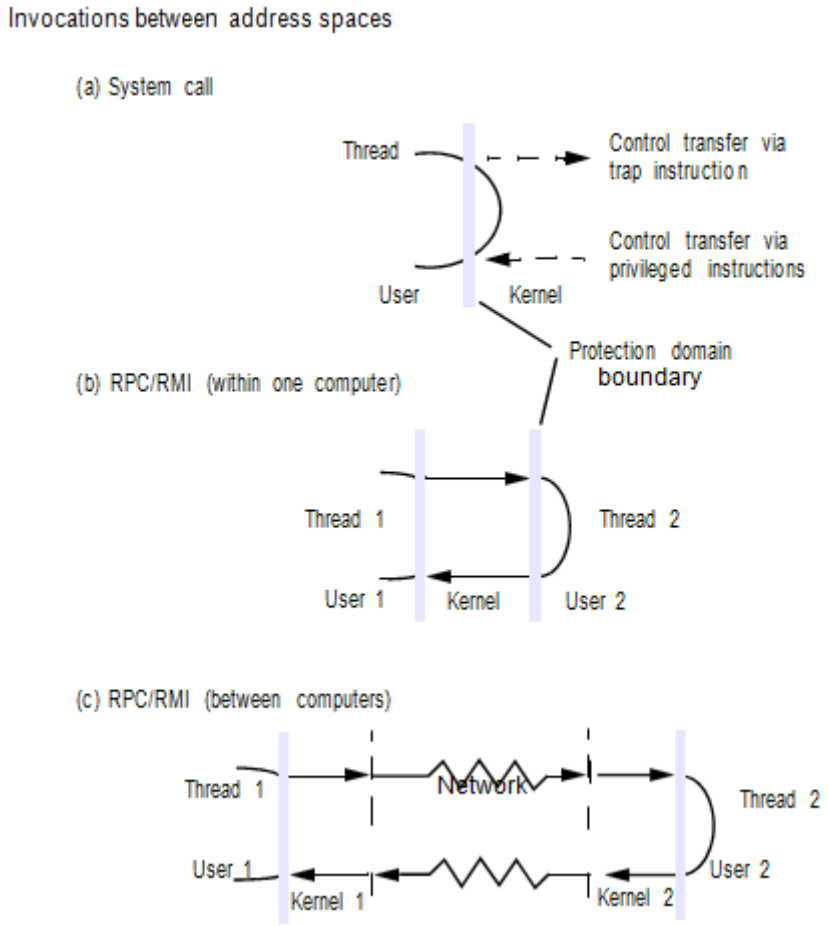
Placing relatively high-level communication functionality in the kernel has the advantage of efficiency. If, for example, middleware provides RMI over UNIX's connected (TCP) sockets, then a client must make two communication system calls (socket write and read) for each remote invocation.

RPC and RMI implementations have been the subject of study because of the

widespread acceptance of these mechanisms for general-purpose client-server processing.

Invocation costs • Calling a conventional procedure or invoking a conventional method, making a system call, sending a message, remote procedure calling and remote method invocation are all examples of invocation mechanisms.

The Figure shows the particular cases of a system call, a remote invocation between processes hosted at the same computer, and a remote invocation between processes at different nodes in the distributed system.



Invocation over the network • A null RPC (and similarly, a null RMI) is defined as an RPC without parameters that executes a null procedure and returns no values. Its execution involves an exchange of messages carrying some system data but no user data.

Null invocation (RPC, RMI) costs are important because they measure a fixed overhead, the latency.

The delay is roughly proportional to the size until the size reaches a threshold at about network packet size. Beyond that threshold, at least one extra packet has to be sent, to carry the extra data.

RPC throughput (or bandwidth) is also of concern when data has to be transferred in bulk. This is the rate of data transfer between computers in a single RPC.

The following are the main components accounting for remote invocation delay, besides network transmission times:

Marshalling: Marshalling and unmarshalling, which involve copying and converting data, create a significant overhead as the amount of data grows.

Transfers between the network interface and main memory are usually handled by direct memory access (DMA). The processor handles the other copies.

: This involves initializing protocol headers and trailers, including checksums. The **Packet initialization** cost is therefore proportional, in part, to the amount of data sent.

Thread scheduling and context switching: These may occur as follows:

Several system calls (that is, context switches) are made during an RPC, as stubs invoke the kernel's communication operations.

One or more server threads is scheduled.If the operating system employs a separate network manager process, then eachSend involves a context switch to one of its threads.

Waiting for acknowledgements: The choice of RPC protocol may influence delay, particularly when large amounts of data are sent.

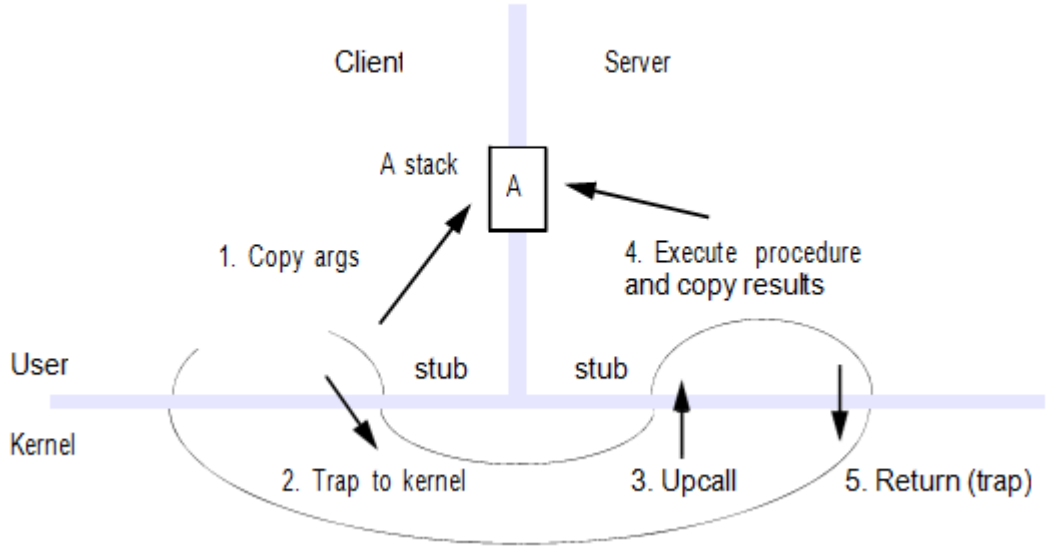
Memory sharing • Shared regions (introduced in Section 7.4) may be used for rapid communication between a user process and the kernel, or between user processes. Data is communicated by writing to and reading from the shared region.

Choice of protocol • The delay that a client experiences during request-reply interac- tions over TCP is not necessarily worse than for UDP and in fact is sometimes better, particularly for large messages.

Invocation within a computer • Bershad et al. [1990] report a study that showed that, in the installation examined, most cross-address-space invocation took place within a computer and not, as might be expected in a client-server installation, between computers.

Bershad et al. developed a more efficient invocation mechanism for the case of two processes on the same machine called lightweight RPC (LRPC). The LRPC design is based on optimizations concerning data copying and thread scheduling.

A lightweight remote procedure call



Instead of RPC parameters being copied between the kernel and user address spaces involved, the client and server are able to pass arguments and return values directly via an A stack.

Discussion of LRPC • There is little doubt that LRPC is more efficient than RPC for the local case, as long as enough invocations take place to offset the memory management costs.

Asynchronous operation

We have discussed how the operating system can help the middleware layer to provide efficient remote invocation mechanisms.

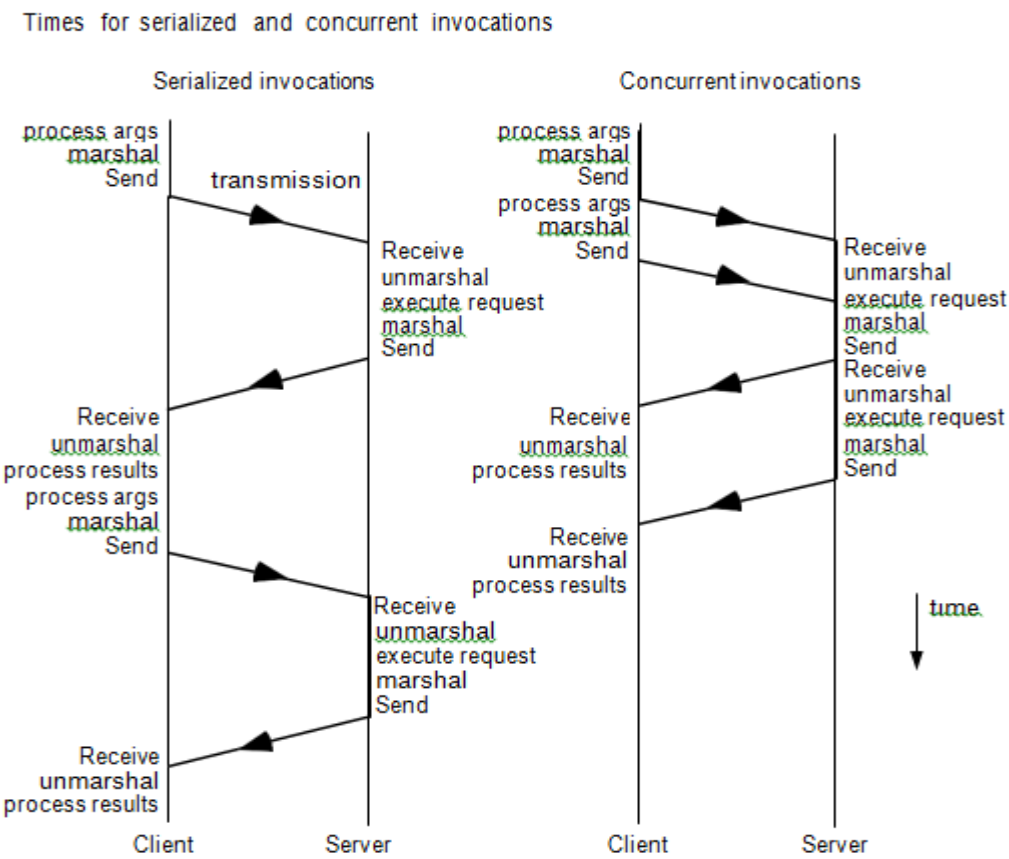
Making invocations concurrently • In the first model, the middleware provides only blocking invocations, but the application spawns multiple threads to perform blocking invocations concurrently.

A good example of such an application is a web browser. A web page typically contains several images and may contain many.

The browser does not need to obtain the images in a particular sequence, so it makes several concurrent requests at a time. That way, the time taken to complete all the image requests is typically lower than the delay that would result from making the requests serially.

The Figure shows the potential benefits of interleaving invocations (such as HTTP requests) between a client and a single server on a single-processor machine.

In the serialized case, the client marshals the arguments, calls the Send operation and then waits until the reply from the server arrives – whereupon it Receives, unmarshals and then processes the results. After this it can make the second invocation.



2.4 Clocks, events and process states

We take a distributed system to consist of a collection of N processes

pi i = 1, 2, ..N. Each process executes on a single processor, and the processors donot share memory.

Each process pi in S has a state si that, in general, it transforms as it executes. The process's state includes the values of all the variables within it. Its state may also include the values of any objects in its local operating system environment that it affects, such as files.

We assume that processes cannot communicate with one another in any way except by sending messages through the network. So, for example, if the processes operate robot arms connected to their respective nodes in the system, then they are not allowed to communicate by shaking one another's robot hands!

As each process pi executes it takes a series of actions, each of which is either a message send or receive operation, or an operation that transforms pi 's state – one that changes one or more of the values in si.

In practice, we may choose to use a high-level description of the actions, according to the application. For example, if the processes in P are engaged in an eCommerce application, then the actions may be ones such as 'client dispatched order message' or 'merchant server recorded transaction to log'.

We define an event to be the occurrence of a single action that a process carries out as it executes – a communication action or a state-transforming action. The sequence of events within a single process pi can be placed in a single, total ordering, which we

denote by the relation→i between the events. That is, e →ie' if and only if the event e occurs before e` at pi .

This ordering is well defined, whether or not the process is multi- threaded, since we have assumed that the process executes on a single processor.

Now we can define the history of process pi to be the series of events that take place within it, ordered as we have described by the relation →i:

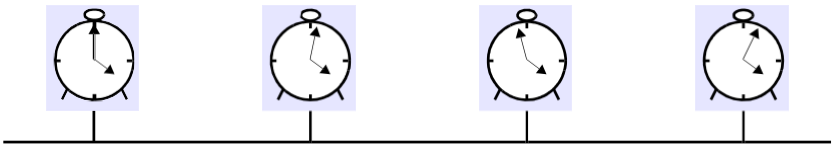
history(Pi) = hi = <ei0, ei1, ei2,…>

2.4.1 Clocks

- We have seen how to order the events at a process, but not how to timestamp them – i.e., to assign to them a date and time of day. Computers each contain their own physical clocks.
- These clocks are electronic devices that count oscillations occurring in a crystal at a definite frequency, and typically divide this count and store the result in a counter register.
- Clock devices can be programmed to generate interrupts at regular intervals in order that, for example, timeslicing can be implemented; however, we shall not concern ourselves with this aspect of clock operation.
- The operating system reads the node's hardware clock value, $H_i(t)$, scales it andadds an offset so as to produce a software clock $C_i(t) = \alpha H_i(t) + \beta$ that approximatelymeasures real, physical time t for process p_i .
- In other words, when the real time in an absolute frame of reference is t, $C_i(t)$ is the reading on the software clock. For example, $C_i(t)$ could be the 64-bit value of the number of nanoseconds that have elapsed at time t since a convenient reference time. In general, the clock is not completely accurate, so$C_i(t)$ will differ from t. Nonetheless, if C_i behaves sufficiently well (we shall examine the notion of clock correctness shortly), we can use its value to timestamp any event atp_i .
- Note that successive events will correspond to different timestamps only if the *clock resolution* – the period between updates of the clock value – is smaller than the time interval between successive events. The rate at which events occur depends on such factors as the length of the processor instruction cycle.

2.4.2 Clock skew and clock drift

- Computer clocks, like any others, tend not to be in perfect agreement (Figure 14.1). The instantaneous difference between the readings of any two clocks is called their *skew*. Also, the crystal-based clocks used in computers are, like any other clocks, subject to *clock drift*, which means that they count time at different rates,and so diverge.



- The underlying oscillators are subject to physical variations, with the consequence that their frequencies of oscillation differ. Moreover, even the same clock's frequency varies with temperature.
- Designs exist that attempt to compensate for this variation, but they cannot eliminate it.
- The difference in the oscillation period between two clocks might be extremely small, but the difference accumulated over many oscillations leads to an observable difference in the counters registered by two clocks, no matter how accurately they were initialized to the same value.
- A clock's *drift rate* is the change in the offset (difference in reading) between the clock and a nominal perfect reference clock per unit of time measured by the reference clock.
- For ordinary clocks based on a quartz crystal this is about 10^{-6} seconds/second, giving a difference of 1 second every 1,000,000 seconds, or 11.6 days. The drift rate of 'high-precision' quartz clocks is about 10^{-7} or 10^{-8}.

2.4.3 Coordinated Universal Time

- Computer clocks can be synchronized to external sources of highly accurate time. The most accurate physical clocks use atomic oscillators, whose drift rate is about one part in 1013.
- The output of these atomic clocks is used as the standard for elapsed real time, known as International Atomic Time. Since 1967, the standard second has been defined as 9,192,631,770 periods of transition between the two hyperfine levels of the ground state of Caesium-133 (Cs133).
- Coordinated Universal Time – abbreviated as UTC (from the French equivalent) – is an international standard for timekeeping.

2.5 Synchronization Physical Clocks

Centralized systems do not need clock synchronization, as they work under a commonclock. But the distributed systems do not follow common clock: each system functions based
on its own internal clock and its own notion of time.The time in distributed systems is measured in the following contexts:

- The time of the day at which an event happened on a specific machine in thenetwork.
- The time interval between two events that happened on different machines inthe network.
- The relative ordering of events that happened on different machines in thenetwork.

Clock synchronization is the process of ensuring that physically distributed processors have a common notion of time.

Clocks are synchronized to an accurate real-time standard like UTC (Universal Coordinated Time). Clocks that must not only be synchronized with each other but also haveto adhere to physical time are termed **physical clocks**. This degree of synchronization additionally enables to coordinate and schedule actions between multiple computers connected to a common network.

Basic terminologies:

If C_a and C_b are two different clocks, then:

- **Time:** The time of a clock in a machine p is given by the function $C_p(t)$,where $C_p(t)= t$ for a perfect clock.
- **Frequency:** Frequency is the rate at which a clock progresses. The frequency at time t of clock $C_a$is$C_a'(t)$.
- **Offset:**Clock offset is the difference between the time reported by a clockand thereal time. The offset of the clock C_a is given by $C_a(t)- t$. Theoffset of clock C a relative to C_b at time $t \geq 0$ is given by $C_a(t)- C_b(t)$
- **Skew:** The skew of a clock is the difference in the frequencies of the clockand the perfect clock. The skew of a clock C_a relative to clock C_b at timet is $C_a'(t)- C_b'(t)$.
- **Drift (rate):** The drift of clock C_a the second derivative of the clockvalue with respect to time. The drift is calculated as:

$$C_a''(t) - C_b''(t)$$

Clocking Inaccuracies

Physical clocks are synchronized to an accurate real-time standard like UTC (Universal Coordinated Time). Due to the clock inaccuracy discussed above, a timer (clock) issaid to be working within its specification if:

$$1-\rho \leq \frac{dC}{dt} \leq 1+\rho$$

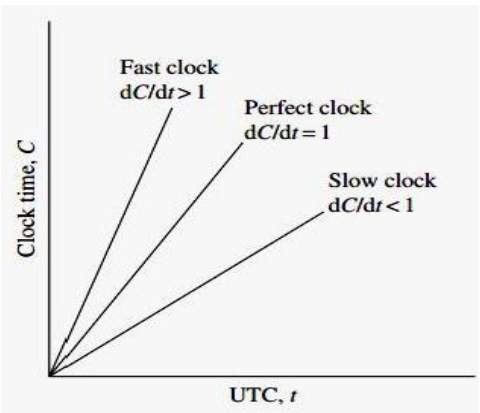
ρ- maximum skew rate.

Offset delay estimation

A time service for the Internet - synchronizes clients to UTC Reliability from redundant paths, scalable, authenticates time sources Architecture. The design of NTP involves a hierarchical tree of time servers with primary server at the root synchronizes with the UTC. The next level contains secondary servers, which act as a backup to the primary server. At the lowest level isthe synchronization subnet which has the clients.

Clock offset and delay estimation

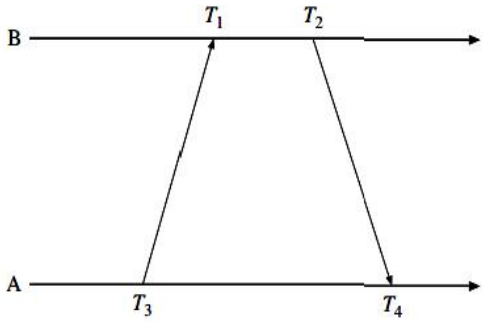
A source node cannot accurately estimate the local time on the target node due to varying message or network delays between the nodes. This protocol employs a very commonpractice of performing several trials and chooses the trial with the minimum delay.



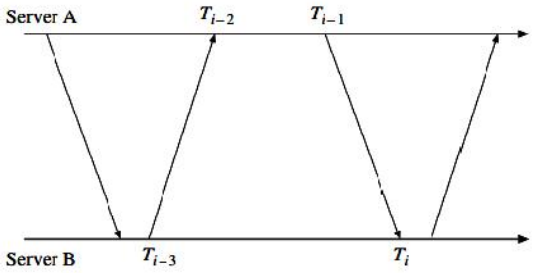
Behavior of clocks

Let T1, T2, T3, T4 be the values of the four most recent timestamps. The clocks A and B are stable and running at the same speed. Let a = T1 − T3 and b = T2 − T4. If the network delay difference from A to B and from B to A, called **differential delay**, is small, the clock offset θ and roundtrip delay δ of B relative to A at time T4 are approximately given by the following:

$\theta=(a+b)/2,$
$\delta=a-b$



Offset and delay estimationbetween processes from same server



Offset and delay estimationbetween processes from different servers

Each NTP message includes the latest three timestamps T1, T2, and T3, while T4 is determined upon arrival.

2.6 Logical Time and Logical Clocks

Logical clocks are based on capturing chronological and causal relationships of processes and ordering events based on these relationships.

Precise physical clocking is not possible in distributed systems. The asynchronous distributed systems spans logical clock for coordinating the events.Three types of logical clock are maintained in distributed systems:

- Scalar clock
- Vector clock
- Matrix clock

Differences between physical and logical clock

Physical Clock	**Logical Clock**

A physical clock is a physical procedure combined with a strategy for measuring that procedure to record the progression of time.	A logical clock is a component for catching sequential and causal connections in a dispersed framework.
The physical clocks are based on cyclic processes such as a celestial rotation.	A logical clock allows global ordering onevents from different processes.

A Framework for a system of logical clocks

A system of logical clocks consists of a time domain T and a logical clock C. Elements of T form a partially ordered set over a relation <. This relation is usually called the happened before or causal precedence.

The logical clock C is a function that maps an event e in a distributed system to an element in the time domain T denoted as C(e).

C : H ▪ T such that

for any two events e_i and e_j,. $e_i \rightarrow e_j \Rightarrow C(e_i) < C(e_j)$.

This monotonicity property is called the **clock consistency condition.**When T and C satisfy the following condition,

$$e_{i,.} \rightarrow e_j \Leftrightarrow C(e_i) < C(e_j)$$

Then

Scalar Time

Scalar time is designed by Lamport to synchronize all the events in distributed systems. A Lamport logical clock is an incrementing counter maintained in each process. Thislogical clock has meaning only in relation to messages moving between processes. When a process receives a message, it resynchronizes its logical clock with that sender maintaining causal relationship.

The Lamport's algorithm is governed using the following rules:

- The algorithm of Lamport Timestamps can be captured in a few rules:
- All the process counters start with value 0.
- A process increments its counter for each event (internal event, message sending, message receiving) in that process.
- When a process sends a message, it includes its (incremented) counter value with the message.
- On receiving a message, the counter of the recipient is updated to the greater of its current counter and the timestamp in the received message, and then incremented by one.

If C_i is the local clock for process P_i then,

- if a and b are two successive events in P_i, then $C_i(b) = C_i(a) + d1$, where $d1 > 0$
- if a is the sending of message m by P_i, then m is assigned timestamp $t_m = C_i(a)$
- if b is the receipt of m by P_j, then $C_j(b) = \max\{C_j(b), t_m + d2\}$, where $d2 > 0$

Rules of Lamport's clock

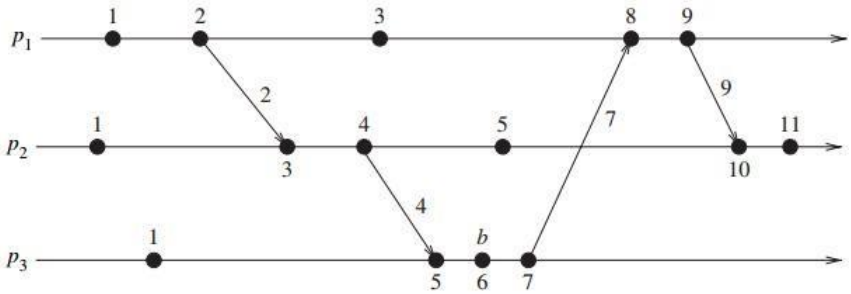
Rule 1: $C_i(b) = C_i(a) + d1$, where $d1 > 0$

Rule 2: The following actions are implemented when p_i receives a message m withtimestamp C_m:

a) *$C_i = max(C_i, C_m)$*

b) *Execute Rule 1*

c) *deliver the message*



Evolution of scalar time

Basic properties of scalar time:

1. **Consistency property:** Scalar clock always satisfies monotonicity. A monotonic clock only increments its timestamp and never jump.Hence it is consistent.

$$C(e_i) < C(e_j)$$

2. **Total Reordering:**Scalar clocks order the events in distributed systems. But all the events do not follow a common identical timestamp. Hence a tie breaking mechanism is essential to order the events. The tie breaking is done through:

   - Linearly order process identifiers.
   - Process with low identifier value will be given higher priority.

The term (t, i) indicates timestamp of an event, where t is its time of occurrence and iis the identity of the process where it occurred.

3. **Event Counting**

   If event e has a timestamp h, then h−1 represents the minimum logical duration, counted in units of events, required before producing the event e. This is called **height** of the event e. h-1 events have been produced sequentially before the evente regardless of the processes that produced these events.

4. **No strong consistency**

   The scalar clocks are not strongly consistent is that the logical local clock and logical global clock of a process are squashed into one, resulting in the loss causaldependency information among events at different processes.

Vector Time

The ordering from Lamport's clocks is not enough to guarantee that if two events precede one another in the ordering relation they are also causally related. Vector Clocks use avector counter instead of an integer counter. The vector clock of a system with N processes is a vector of N counters, one counter per process. Vector counters have to follow the

followingupdate rules:

Initially, all counters are zero.

- Each time a process experiences an event, it increments its own counter in thevector by one.
- Each time a process sends a message, it includes a copy of its own (incremented) vector in the message.
- Each time a process receives a message, it increments its own counter in the vector by one and updates each element in its vector by taking the maximum of the value in its own vector counter and the value in the vector in the received message.

Rules of Vector Time

Rule 1: Before executing an event, process pi updates its local logical timeas follows:

$$\upsilon t_i[i] := \upsilon t_i[i] + d \quad (d > 0)$$

Rule 2: Each message m is piggybacked with the vector clock vt of the senderprocess at sending time. On the receipt of such a message (m,vt), process

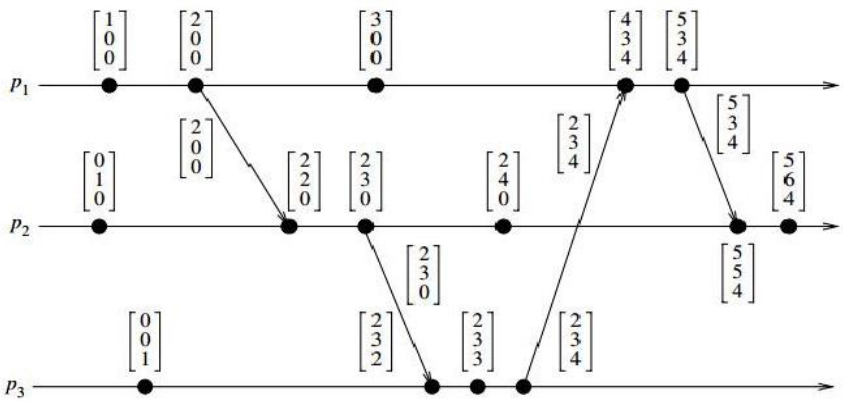
p_i executes the following sequence of actions:

1. ***update its global logical time***

$$1 \le k \le n : \upsilon t_i[k] := \max(\upsilon t_i[k], \upsilon t[k])$$

2. ***execute R1***
3. ***deliver the message m***

Evolution of vector scale

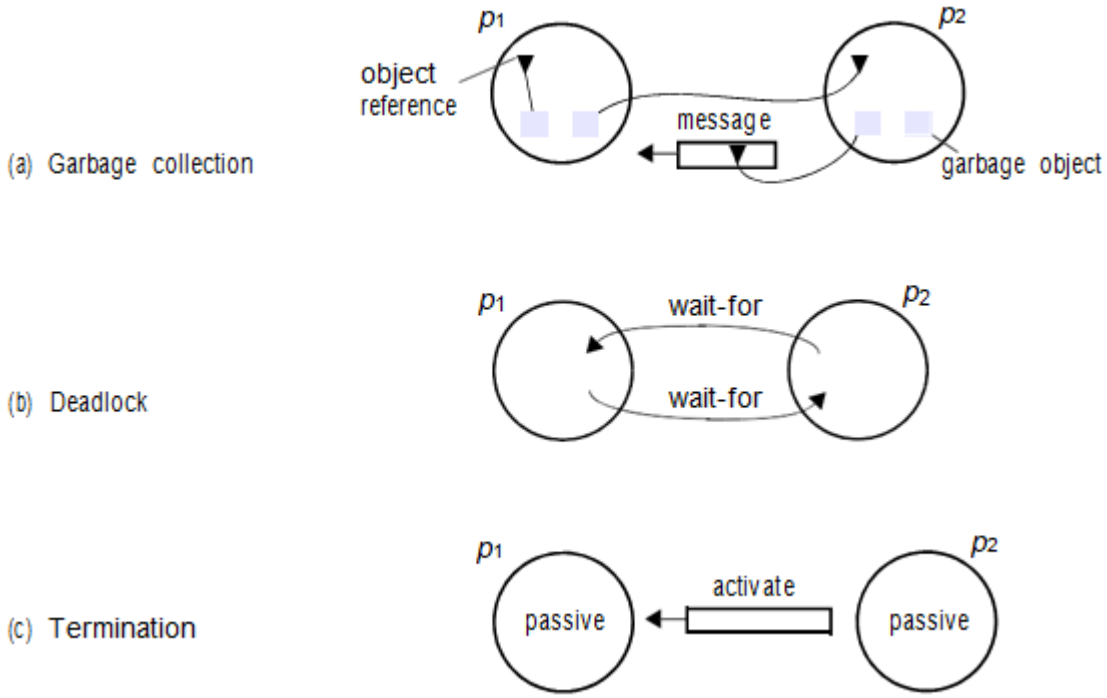


2.6 Global States

Distributed Snapshot represents a state in which the distributed system might havebeen in. A snapshot of the system is a single configuration of the system.

We begin by giving the examplesof distributed garbage collection, deadlock detection, termination

detection and debugging:



Distributed garbage collection: An object is considered to be garbage if there are nolonger any references to it anywhere in the distributed system. The memory taken upby that object can be reclaimed once it is known to be garbage.

Distributed deadlock detection: A distributed deadlock occurs when each of a collection of processes waits for another process to send it a message, and where there is a cycle in the graph of this 'waits-for' relationship.

Distributed termination detection: The problem here is how to detect that a distributed algorithm has terminated. Detecting termination is a problem that soundsdeceptively easy to solve: it seems at first only necessary to test whether each process has halted.

Distributed debugging: Distributed systems are complex to debug [Bonnaire *et al*. 1995], and care needs to be taken in establishing what occurred during the execution.For example, suppose Smith has written an application in which each process $p_i$contains a variable x_i (i = 1, 2…, N).

Global states and consistent cuts

It is possible in principle to observe the succession of states of an individual process, butthe question of how to ascertain a global state of the system – the state of the collectionof processes – is much harder to address.

The essential problem is the absence of global time. If all processes had perfectly synchronized clocks, then we could agree on a time at which each process would recordits state – the result would be an actual global state of the system.

Let us return to our general system of N processes p_i (i = 1, 2, …, N), whose execution we wish to study. We said above that a series of events occurs at each process,and that we may characterize the execution of each process by its history:

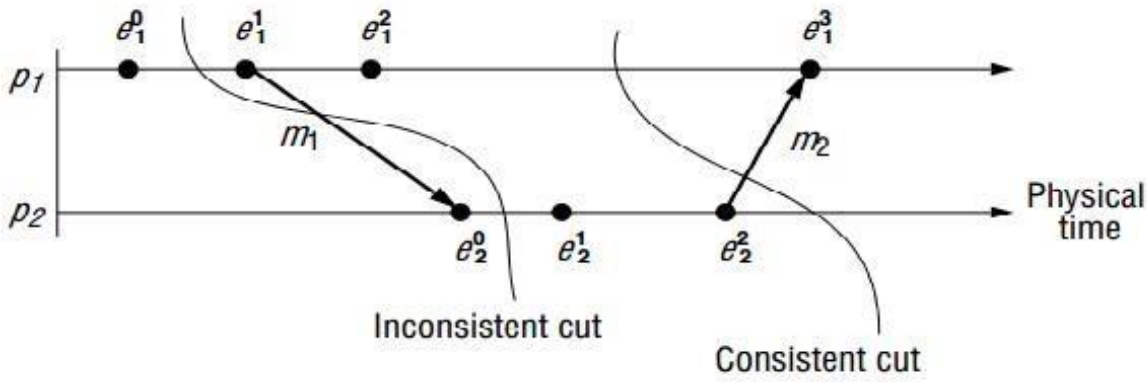
$$history(pi) = h = \langle ei^0, ei^1, ei^2, \ldots \rangle$$

Similarly, we may consider any finite prefix of the process's history:

$$h^k = <ei^0, ei^1, ei^2, \ldots>$$

Each event either is an internal action of the process (for example, the updating of one of its variables), or is the sending or receipt of a message over the communication channels that connect the processes.

Each event either is an internal action of the process. We denote by s_i^k the state of process p_i immediately before the kth event occurs. The state s_i in the global state S corresponding to the cut C is that of p_i immediately after the last event processed by p_i in thecut – ei^{ci} . The set of events ei^{ci} is called the frontier of the cut.



Consistent states: The states should not violate causality. Such states are called consistent global states and are meaningful global states.

Inconsistent global states: They are not meaningful in the sense that a distributed system can never be in an inconsistent state.

A cut is a set of cut events, one per node, each of which captures the state of the nodeon which it occurs.

The 'snapshot' algorithm of Chandy and Lamport

Chandy and Lampor. describe a 'snapshot' algorithm for determining global states of distributed systems, which we now present. The goal of the algorithm is to record a set of process and channel states (a 'snapshot') for a set of processes p_i (i = 1, 2, .. N) such that, even though the combination of recorded states may never have occurred at the same time, the recorded global state is consistent.

We shall see that the state that the snapshot algorithm records has convenient properties for evaluating stable global predicates.

The algorithm assumes that:

- Neither channels nor processes fail – communication is reliable so that every message sent is eventually received intact, exactly once.
- Channels are unidirectional and provide FIFO-ordered message delivery.
- The graph of processes and channels is strongly connected (there is a path between any two processes).
- Any process may initiate a global snapshot at any time.
- The processes may continue their execution and send and receive normal messages while the snapshot takes place.

Chandy and Lamport's 'snapshot' algorithm

Marker receiving rule for process pi
On receipt of a marker message at pi over channel c:
if (pi has not yet recorded its state) it
records its process state now;
records the state of c as the empty set;
turns on recording of messages arriving over other incoming channels;
else

pi records the state of c as the set of messages it has received over c since it saved its state.
end if
Marker sending rule for process pi
After pi has recorded its state, for each outgoing channel c: pi sends one marker message over c
(before it sends any other message over c).

The algorithm is defined through two rules, the marker receiving rule and the marker sending rule. The marker sending rule obligates processes to send a marker after they have recorded their state, but before they send any other messages.

2.8 Distributed Mutual Exclusion

Mutual exclusion is a concurrency control property which is introduced to prevent race conditions.

It is the requirement that a process cannot access a shared resource while another concurrent process is currently present or executing the same resource.

Mutual exclusion in a distributed system states that only one process is allowed toexecute the critical section (CS) at any given time.

Message passing is the sole means for implementing distributed mutual exclusion.

- The decision as to which process is allowed access to the CS next is arrived at by message passing, in which each process learns about the state of all other processes insome consistent way.
- There are three basic approaches for implementing distributed mutual exclusion:

1. Token-based approach:

- A unique token is shared among all the sites.
- If a site possesses the unique token, it is allowed to enter its critical section
- This approach uses sequence number to order requests for the critical section.
- Each requests for critical section contains a sequence number. This sequence number is used to distinguish old and current requests.
- This approach insures Mutual exclusion as the token is unique.
- Eg: Suzuki-Kasami's Broadcast Algorithm

2. Non-token-based approach:

- A site communicates with other sites in order to determine which sites should execute critical section next. This requires exchange of two or more successiveround of messages among sites.
- This approach use timestamps instead of sequence number to order requests forthe critical section.
- Whenever a site make request for critical section, it gets a timestamp. Timestamp is also used to resolve any conflict between critical section requests.
- All algorithm which follows non-token based approach maintains a logical clock. Logical clocks get updated according to Lamport's scheme.
- Eg: Lamport's algorithm, Ricart–Agrawala algorithm

3. Quorum-based approach:

- Instead of requesting permission to execute the critical section from all other sites, Each site requests only a subset of sites which is called a quorum.
- Any two subsets of sites or Quorum contains a common site.
- This common site is responsible to ensure mutual exclusion.
- Eg: Maekawa's Algorithm

Requirements of mutual exclusion algorithms

Safety property:

The safety property states that at any instant, only one process can execute the criticalsection. This is an essential property of a mutual exclusion algorithm.

Liveness property:

This property states the absence of deadlock and starvation. Two or more sites shouldnot endlessly wait for messages that will never arrive. In addition, a site must not waitindefinitely to execute the CS while other sites are repeatedly executing the CS. Thatis, every requesting site should get an opportunity to execute the CS in finite time.

Fairness:

Fairness in the context of mutual exclusion means that each process gets a fair chance to execute the CS. In mutual exclusion algorithms, the fairness property generally means that the CS execution requests are executed in order of their arrival in the system.

Performance metrics

- **Message complexity:** This is the number of messages that are required per CS execution by a site.
- **Synchronization delay:** After a site leaves the CS, it is the time required and before the next site enters the CS.
- **Response time:** This is the time interval a request waits for its CS execution to be over after its request messages have been sent out. Thus, response time does not include the time a request waits at a site before its request messages have been sentout.
- **System throughput:** This is the rate at which the system executes requests for

the CS. If SD is the synchronization delay and E is the average critical section execution time.

$$\text{System throughput} = \frac{1}{(SD + E)}$$

Distributed Mutual Exclusion Algorithm

1.LAMPORT'S ALGORITHM

- Lamport's Distributed Mutual Exclusion Algorithm is a permission based algorithm proposed by Lamport as an illustration of his synchronization schemefor distributed systems.
- In permission based timestamp is used to order critical section requests and to resolve any conflict between requests.
- In Lamport's Algorithm critical section requests are executed in the increasing order of timestamps i.e a request with smaller timestamp will be given permissionto execute critical section first than a request with larger timestamp.
- Three type of messages (REQUEST, REPLY and RELEASE) are used and communication channels are assumed to follow FIFO order.
- A site send a REQUEST message to all other site to get their permission to entercritical section.
- A site send a REPLY message to requesting site to give its permission to enter thecritical section.
- A site send a RELEASE message to all other site upon exiting the critical section.
- Every site Si, keeps a queue to store critical section requests ordered by theirtimestamps.
- request_queuei denotes the queue of site Si.
- A timestamp is given to each critical section request using Lamport's logical clock.
- Timestamp is used to determine priority of critical section requests. Smaller timestamp gets high priority over larger timestamp. The execution of critical section request is always in the order of their timestamp.

Requesting the critical section

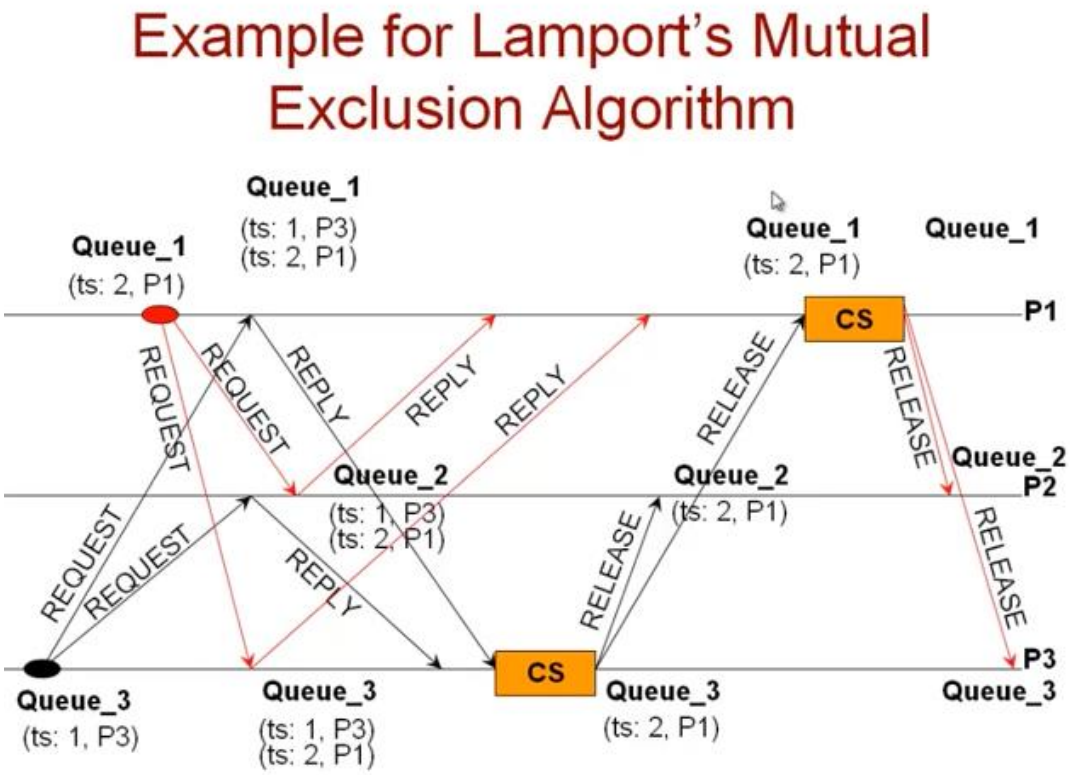
- When a site Si wants to enter the CS, it broadcasts a REQUEST(tsi, i) message to all other sites and places the request on request_queuei. ((tsi, i) denotes the timestamp of the request)
- When a site Sj receives the REQUEST(ts_i, i) message from site S_i, it places side S_i`s request on request_$queue_j$ and return a timestamped REPLY message to S_i.

Executing the critical section

Site S_i enters the CS when the following two conditions hold:

L1: Si has received a message with timestamp larger then (ts_i, i) from all other sites.

L2: S_i`s request is at the top of request_$queue_i$;



Releasing the critical section

- Site S_i, upon exiting the CS, removes its request from the top of its request queueand broadcast a timestamped RELEASE message to all other sites.
- When a site S_j receives a RELEASE messge from site S_i, it removes S_i`s request from its request queue.

To enter Critical section:

- When a site S_i wants to enter the critical section, it sends a request message Request(ts_i, i) to all other sites and places the request on request_queue$_i$. Here, T_{si}denotes the timestamp of Site $S_{i.}$

- When a site S_j receives the request message REQUEST(ts_i, i) from site Si, it returns a timestamped REPLY message to site Si and places the request of site Sion request_queue$_j$

To execute the critical section:

- A site S_i can enter the critical section if it has received the message with timestamplarger than (ts_i, i) from all other sites and its own request is at the top ofrequest_queue$_{i.}$

To release the critical section:

When a site S_i exits the critical section, it removes its own request from the top of its request queue and sends a timestamped RELEASE message to all other sites. When a site $S_j$receives the timestamped RELEASE message from site S_i, it removes the request of S_{ia} fromits request queue.

Message Complexity:

Lamport's Algorithm requires invocation of 3(N – 1) messages per critical section execution. These 3(N – 1) messages involves

- (N – 1) request messages
- (N – 1) reply messages
- (N – 1) release messages

2.RICART–AGRAWALA ALGORITHM

- Ricart–Agrawala algorithm is an algorithm to for mutual exclusion in a distributedsystem proposed by Glenn Ricart and Ashok Agrawala.
- This algorithm is an extension and optimization of Lamport's Distributed Mutual Exclusion Algorithm.
- It follows permission based approach to ensure mutual exclusion.
- Two type of messages (REQUEST and REPLY) are used and communicationchannels are assumed to follow FIFO order.
- A site send a REQUEST message to all other site to get their permission to entercritical section.
- A site send a REPLY message to other site to give its permission to enter the criticalsection.
- A timestamp is given to each critical section request using Lamport's logical clock.
- Timestamp is used to determine priority of critical section requests.
- Smaller timestamp gets high priority over larger timestamp.
- The execution of critical section request is always in the order of their timestamp.

Requesting the critical section

(a) When a site Si wants to enter the CS, it broadcasts a timestamped REQUEST message to all other sites.

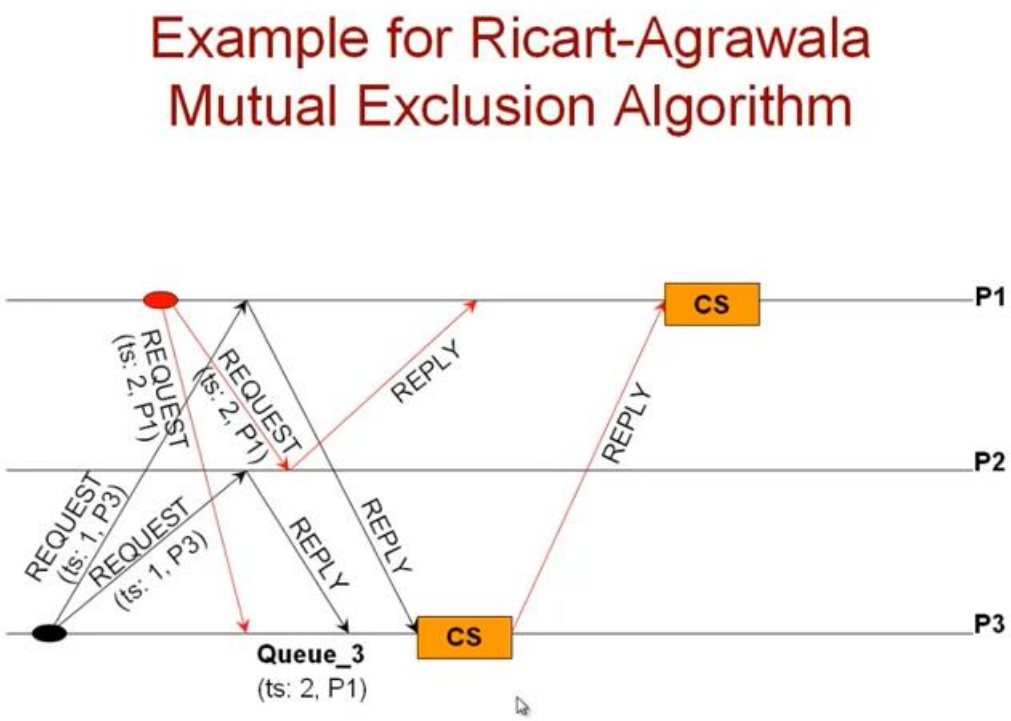
(b) When site Sj receives a REQUEST message from site Si, it sends a REPLY message to site Si if site Sj is neither requesting nor executing the CS, or if the site Sj is requesting and S_i`s request's timestamp is smaller than site $S_j$`s own request'stimestamp. Otherwise, the reply is deferred and Sj set $RD_j[i] := 1$.

Executing the critical section

(c) Site Si enters the CS after it has received a reply message from every site it sent aREQUEST message to.

Releasing the critical section

(d) When site Si exits the CS, it sends all the deferred REPLY messages: $\forall$ j if $RD_i[j] = 1$, then sends a REPLY message to S_j and set $RD_i[j] := 0$



To enter Critical section:

- When a site S_i wants to enter the critical section, it send a timestamped REQUESTmessage to all other sites.
- When a site S_j receives a REQUEST message from site S_i, It sends a REPLY message to site S_i if and only if Site S_j is neither requesting nor currently executingthe critical section.
- In case Site S_j is requesting, the timestamp of Site S_i's request is smaller than its own request.
- Otherwise the request is deferred by site S_j.

To execute the critical section:

Site S_i enters the critical section if it has received the REPLY message from all other sites.

To release the critical section:

Upon exiting site S_i sends REPLY message to all the deferred requests.

Message Complexity:

Ricart–Agrawala algorithm requires invocation of 2(N – 1) messages per critical section execution. These 2(N – 1) messages involve:

- (N – 1) request messages
- (N – 1) reply messages

3. MAEKAWA's ALGORITHM

- Maekawa's Algorithm is quorum based approach to ensure mutual exclusion in

distributed systems.

- In permission based algorithms like Lamport's Algorithm, Ricart-Agrawala Algorithm etc. a site request permission from every other site but in quorum basedapproach, a site does not request permission from every other site but from a subset of sites which is called quorum.
- Three type of messages (REQUEST, REPLY and RELEASE) are used.
- A site send a REQUEST message to all other site in its request set or quorum to get their permission to enter critical section.
- A site send a REPLY message to requesting site to give its permission to enter thecritical section.
- A site send a RELEASE message to all other site in its request set or quorum uponexiting the critical section.

Requesting the critical section

(a) A site S_i requests access to the CS by sending REQUEST(i) messages to all sites inits request set R_i.

(b) When a site Sj receives the REQUEST(i) message, it sends a REPLY(j) message to Si provided it hasn't sent a REPLY message to a site since its receipt of the last RELEASE message. Otherwise, it queues up the REQUEST(i) for later consideration.

Executing the critical section:

(c) Site S_i execute the CS only after it has received a REPLY message from every sitein R_1.

Releasing the critical section:

(d) After the execution of the CS is over, site S_i sends a RELEASE(i) message toevery site in R_i.

(e) When a site S_j receives a RELEASE(i) message from site Si, it sends a REPLY message to the next site waiting in the queue and deletes that entry from the queue.It the queue is empty, then the site updates its state to reflect that it has not sent outany REPLY message since the receipt of the last RELEASE message.

The following are the conditions for Maekawa's algorithm:

M1 $\left(\forall i\ \forall j : i \neq j, 1 \leq i,\ j \leq N :: R_i \cap R_j \neq \phi\right)$

M2 $\left(\forall i : 1 \leq i \leq N :: S_i \in R_i\right)$

M3 $\left(\forall i : 1 \leq i \leq N :: |R_i| = K\right)$

M4 Any site S_j is contained in K number of R_is, $1 \leq i,\ j \leq N$

Maekawa used the theory of projective planes and showed that N = K(K – 1)+ 1. This relation gives |Ri|= √N.

To enter Critical section:

- When a site S_i wants to enter the critical section, it sends a request messageREQUEST(i) to all other sites in the request set R_i.

- When a site S_j receives the request message REQUEST(i) from site S_i, it returns aREPLY message to site S_i if it has not sent a REPLY message to the site from the time it received the last RELEASE message. Otherwise, it queues up the request.

To execute the critical section:

- A site S_i can enter the critical section if it has received the REPLY message fromall the site in request set R_i

To release the critical section:

- When a site S_i exits the critical section, it sends RELEASE(i) message to all othersites in request set R_i
- When a site S_j receives the RELEASE(i) message from site S_i, it send REPLY message to the next site waiting in the queue and deletes that entry from the queue
- In case queue is empty, site S_j update its status to show that it has not sent any REPLY message since the receipt of the last RELEASE message.

Message Complexity:

Maekawa's Algorithm requires invocation of 3√N messages per critical section execution as the size of a request set is √N. These 3√N messages involves.

- √N request messages
- √N reply messages
- √N release messages

2.8 Elections

- An algorithm for choosing a unique process to play a particular role is called an *electionalgorithm*. For example, in a variant of our central-server algorithm for mutual exclusion, the 'server' is chosen from among the processes p_i, (i = 1, 2, …, N) that need to use the critical section.
- An election algorithm is needed for this choice. It is essential that all the processes agree on the choice. Afterwards, if the process that playsthe role of server wishes to retire then another election is required to choose areplacement.
- We say that a process calls the election if it takes an action that initiates a particular run of the election algorithm.

Our requirements are that, during any particular run of the algorithm:

E1: (safety): A participant process p_i has $elected_i = \perp$ or $elected_i = P$, where P is chosen as the non-crashed process at the end ofthe run with the largest identifier.

E2: (liveness): All processes p_i participate and eventually either set $elected_i \neq \perp$ – or crash.

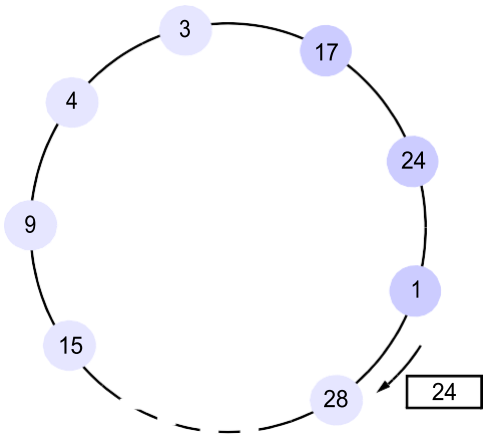
- Note that there may be processes p_j that are not yet participants, which record in $elected_j$ the identifier of the previous elected process.

A ring-based election algorithm

- The algorithm of Chang and Roberts [1979] is suitable for a collection of processes arranged in a logical ring. Each process p_i has a communication channel to the next

process in the ring, $p_{(i+1) mod N}$, and all messages are sent clockwise around the ring. We assume that no failures occur, and that the system is asynchronous. The goal of this algorithm is to elect a single process called the *coordinator*, which is the process with the largest identifier.

- Initially, every process is marked as a *non-participant* in an election. Any process can begin an election. It proceeds by marking itself as a *participant*, placing its identifier in an *election* message and sending it to its clockwise neighbour.
- When a process receives an *election* message, it compares the identifier in the message with its own. If the arrived identifier is greater, then it forwards the message to its neighbour. If the arrived identifier is smaller and the receiver is not a *participant*, then it substitutes its own identifier in the message and forwards it; but it does not forward the message if it is already a *participant*. On forwarding an *election* message in any case, the process marks itself as a *participant*.



A ring-based election in progress

- If, however, the received identifier is that of the receiver itself, then this process's identifier must be the greatest, and it becomes the coordinator. The coordinator marks itself as a *non-participant* once more and sends an *elected* message to its neighbour, announcing its election and enclosing its identity.
- When a process p_i receives an *elected* message, it marks itself as a *non- participant*, sets its variable $elected_i$ to the identifier in the message and, unless it is the new coordinator, forwards the message to its neighbour.
- It is easy to see that condition E1 is met. All identifiers are compared, since a process must receive its own identifier back before sending an *elected* message. For anytwo processes, the one with the larger identifier will not pass on the other's identifier. It is therefore impossible that both should receive their own identifier back.

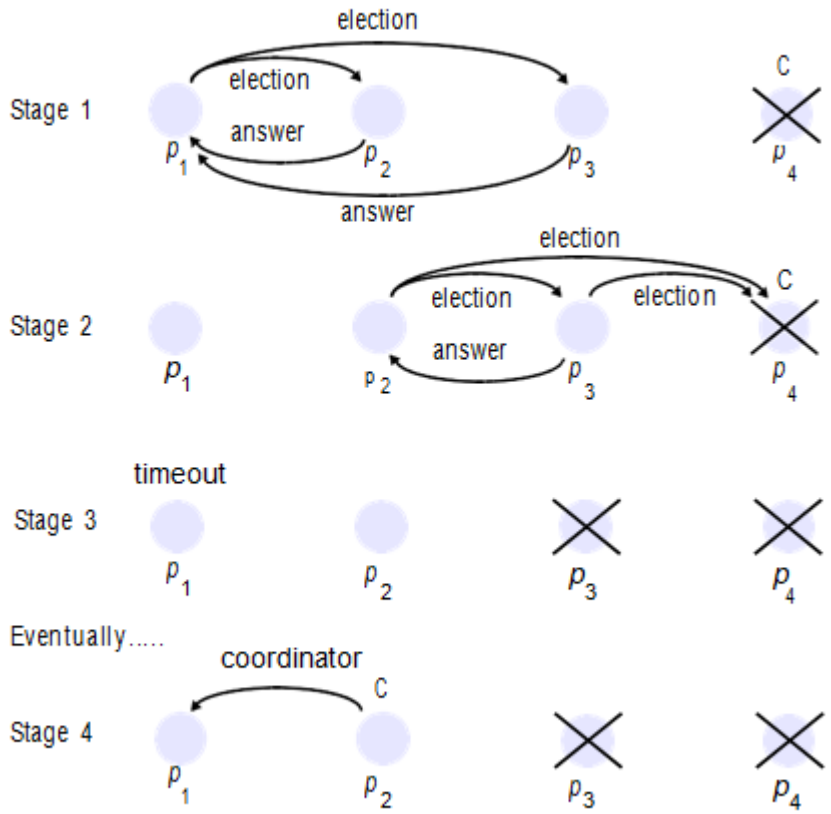
The bully algorithm

- The bully algorithm [Garcia-Molina 1982] allows processes to crash during an election, although it assumes that message delivery between processes is reliable.
- Unlike the ring-based algorithm, this algorithm assumes that the system is synchronous: it uses timeouts to detect a process failure. Another difference is that the ring-based algorithm assumed that processes have minimal *a priori* knowledge of one another: each knows only how to communicate with its neighbour, and none knows the identifiers of the

other processes.

- The bully algorithm, on the other hand, assumes that each process knows which processes have higher identifiers, and that it can communicate with all such processes.
- There are three types of message in this algorithm:
  - an *election* message is sent toannounce an election;
  - an *answer* message is sent in response to an election message
  - a *coordinator* message is sent to announce the identity of the elected process – the new coordinator'.
- A process begins an election when it notices, through timeouts, that the coordinator has failed. Several processes may discover this concurrently.

The bully algorithm



The election of coordinator p_2, after the failure of p_4 and then p_3

- The process that knows it has the highest identifier can elect itself as thecoordinator simply by sending a *coordinator* message to all processes with lower identifiers.
- If a process pi receives a coordinator message, it sets its variable electedi to the identifier of the coordinator contained within it and treats that process as the coordinator.
- If a process receives an election message, it sends back an answer message and begins another election – unless it has begun one already.
- The operation of the algorithm is shown in Figure, There are four processes, p1 – p4 . Process p1 detects the failure of the coordinator p4 and announces an election (stage 1 in the figure).

- On receiving an election message from p1 , processes p2 and p3 send answer messages to p1 and begin their own elections; p3 sends an answer message to p2 , but p3 receives no answer message from the failed process p4 (stage 2). It therefore decides that it is the coordinator. But before it can send out thecoordinator message, it too fails (stage 3).
- When p1 's timeout period T` expires (which we assume occurs before p2 's timeout expires), it deduces the absence of a coordinator message and begins another election. Eventually, p2 is elected coordinator (stage 4).
- This algorithm clearly meets the liveness condition E2, by the assumption of reliable message delivery. And if no process is replaced, then the algorithm meets condition E1.
- It is impossible for two processes to decide that they are the coordinator, since the process with the lower identifier will discover that the other exists and defer to it.
- Taking the example just given, suppose that either p_3 had not failed but was just running unusually slowly (that is, that the assumption that the system is synchronous isincorrect), or that p_3 had failed but was then replaced. Just as p_2 sends its *coordinator*message, p_3 (or its replacement) does the same.
- p_2 receives p_3 's *coordinator* messageafter it has sent its own and so sets $elected_2 = p_3$. Due to variable message transmission delays, p_1 receives p_2 's *coordinator* message after p_3 's and so eventually sets $elected_1 = p_2$. Condition E1 has been broken.

2.9 Distributed Transactions

Introduction

- When a distributed transaction comes to an end, the atomicity property of transactions requires that either all of the servers involved commit the transaction or allof them abort the transaction.
- The manner in which the coordinator achieves this depends on the protocol chosen. A protocol knownas the 'two-phase commit protocol' is the most commonly used.
- This protocol allows the servers to communicate with one another to reach a joint decision as to whether to commit or abort.
- In some cases, the transactions may be serialized at the individual servers, but a cycle of dependencies between the different servers may occur and a distributed deadlock arises.
- Transaction recovery is concerned with ensuring that all the objects involved in transactions are recoverable.

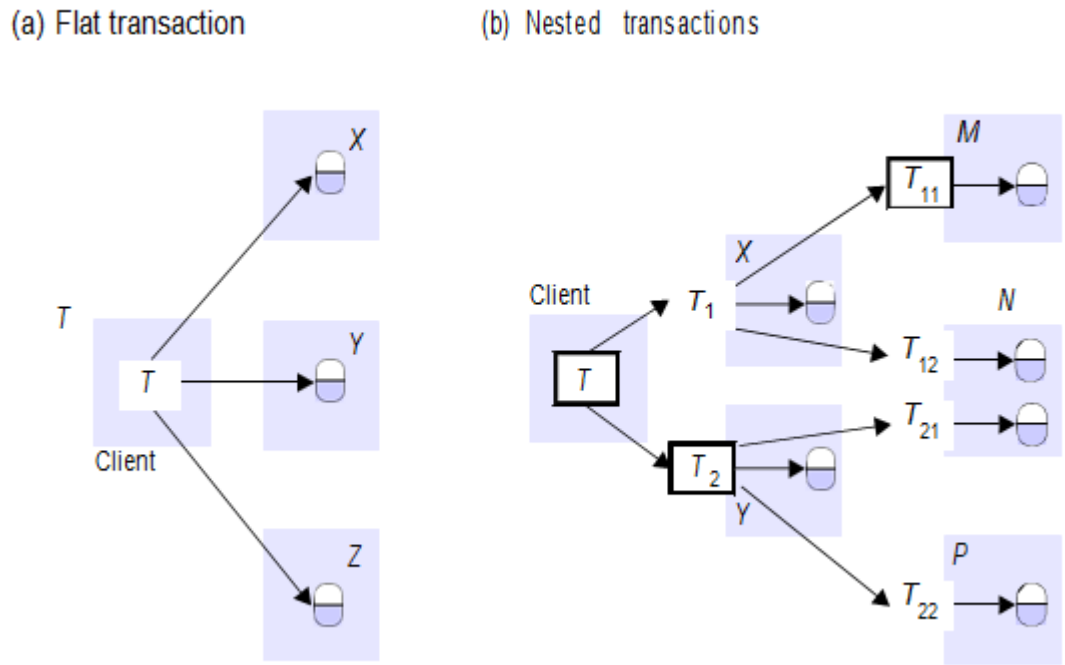
Flat and nested distributed transactions:

- A client transaction becomes distributed if it invokes operations in several different servers. There are two different ways that distributed transactions can be structured: as flat transactions and as nested transactions.
- In a flat transaction, a client makes requests to more than one server. For example, in Figure (a), transaction *T* is a flat transaction that invokes operations on objects inservers *X*, *Y* and *Z*. A flat client transaction completes each of its requests before goingon to the next one.
- In a nested transaction, the top-level transaction can open subtransactions, and each subtransaction can open further subtransactions down to any depth of nesting.
- Figure (b) shows a client transaction *T* that opens two subtransactions, T_1 and T_2, which access objects at servers *X* and *Y*.
- The subtransactions T_1 and T_2 open further subtransactions $T11$, $T12$, $T21$, and $T22$,

which access objects at servers *M*, *N* and *P*.

- In the nested case, subtransactions at the same level can run concurrently, so T_1 and T_2 are concurrent, and as they invoke objects in different servers, they can run in parallel. The four subtransactions *T*11, *T*12, *T*21 and *T*22 also run concurrently.

Distributed transactions



The coordinator of a distributed transaction

- Servers that execute requests as part of a distributed transaction need to be able to communicate with one another to coordinate their actions when the transaction commits.
- A client starts a transaction by sending an openTransaction request to a coordinator. The coordinator that is contacted carries out the openTransaction and returns the resulting transaction identifier (TID) to the client. Transaction identifiers for distributed transactions must be unique within a distributed system.
- The coordinator that opened the transaction becomes the coordinator for the distributed transaction and at the end is responsible for committing or aborting it.
- Each of the servers that manages an object accessed by a transaction is a participant in the transaction and provides an object we call the *participant*.
- Each participant is responsible for keeping track of all of the recoverable objects at that server that are involved, in the transaction. The participants are responsible for cooperating with the coordinator in carrying out the commit protocol.
- During the progress of the transaction, the coordinator records a list of references to the participants, and each participant records a reference to the coordinator.

UNIT III PEER TO PEER SERVICES

Introduction - Napster and its legacy- Peer-to-peer middleware- Routing Overlay - case studies Pastry, Tapestry- Digital signatures- Distributed debugging

3.1 INTRODUCTION

Peer to peer (P2P) systems refers to the applications that take advantage of resources like storage, time cycles, content, manpower available at the end systems of the internet. In other words, the peer to peer computing architecture contains nodes that are equal participants in data sharing. All the tasks are equally divided between all the nodes. The nodes interact with each other as required as share resources. This deals with application layer organization of network overlay for flexibility of sharing resources.

The ongoing entry and exit of various nodes, as well as dynamic insertion and deletion of objects in P2P network is called churn. The impact of churn should be as transparent as possible. There are two types of P2P systems: structured and unstructured.

Differences between structured and unstructured P2P

Unstructured P2P	**Structured P2P**
The construction of overlay network is highly flexible.	The construction of overlay network has low level of flexibility.
The resources are indexed locally.	The resources are distributed remotely and indexed using hash tables.
The messages can be broadcast or random walk.	The messages are unicast.
The network puts best effort content location.	The network guarantees the content location.
High overhead.	Comparatively low overhead.
This supports high failure rates.	Supports moderate failure rates.
This is suitable for small scale and highly dynamic applications.	This is suitable for large scale and relatively stable applications.

Overlay network is constructed over another network. For example, connecting internet over telephone lines is an overlay network. The topology of the overlay network is independent from the underlying network.

Characteristics of Peer to Peer Computing

- Peer to peer networks are usually formed by groups computers. These

computers all store their data using individual security but also share data with all the other nodes.

- The nodes in peer to peer networks both use resources and provide resources. So, if the nodes increase, then the resource sharing capacity of the peer to peer network increases.
- The nodes in peer to peer networks act as both clients and servers. Hence, it is difficult to provide adequate security for the nodes. This can lead to denial of service attacks.
- Most modern operating systems such as Windows and Mac OS contain software to implement peer to peer networks.
- Efficient usage of resources.
- Self -organizing nature: because of scalable storage, CPU power and other resources.
- Distributed control: fast and efficient searching for data.
- Symmetric: highly scalable
- Anonymity: efficient management of churns
- Naming mechanism: selection of geographically close server.
- Security, authentication, trust: Redundancy in storage

Advantages of Peer to Peer Computing

- Each computer in the peer to peer network manages itself. The network is quite easy to set up and maintain.
- The server handles all the requests of the clients. This provision is not required in peer to peer computing and the cost of the server is saved.
- It is easy to scale the peer to peer network and add more nodes. This only increases the data sharing capacity of the system.
- None of the nodes in the peer to peer network are dependent on the others for their functioning. Hence the network is fault tolerant.
- Easy deployment and organization.

Disadvantages of Peer to Peer Computing

- It is difficult to back-up the data as it is stored in different computer systems and there is no central server.
- It is difficult to provide overall security in the peer to peer network as each system is independent and contains its own data.

3.2 NAPSTER AND ITS LEGACY

- The Internet was originally built as a peer-to-peer system in the late 1960s to share computing resources within the US.
- Later the transfer of resources between systems took place by means of client

server communication.

- Later Napster was developed for peer –to-peer file sharing especially MP3 files.
- They are not fully peer-to-peer since it used central servers to maintain lists of connected systems and the files they provided, while actual transactions were conducted directly between machines.

Working of Napster

- ✓ Each user must have Napster software in order to engage in file transfers.
- ✓ The user runs the Napster program. Once executed, this program checks for an Internet connection.
- ✓ If an Internet connection is detected, another connection between the user's computer and one of Napster's Central Servers will be established. This connection is made possible by the Napster file-sharing software.
- ✓ The Napster Central Server keeps a directory of all client computers connected toit and stores information on them as described above.
- ✓ If a user wants a certain file, they place a request to the Napster Centralised Server that it's connected to.
- ✓ The Napster Server looks up its directory to see if it has any matches for the user's request.
- ✓ The Server then sends the user a list of all that matches (if any) it as found including the corresponding, IP address, user name, file size, ping number, bit rate etc.
- ✓ The user chooses the file it wishes to download from the list of matches and triesto establish a direct connection with the computer upon which the desired file resides.
- ✓ It tries to make this connection by sending a message to the client computer indicating their own IP address and the file name they want to download from the client.
- ✓ If a connection is made, the client computer where the desired file resides is now considered the host.
- ✓ The host now transfers the file to the user.
- ✓ The host computer breaks the connection with the user computer when downloading is complete.

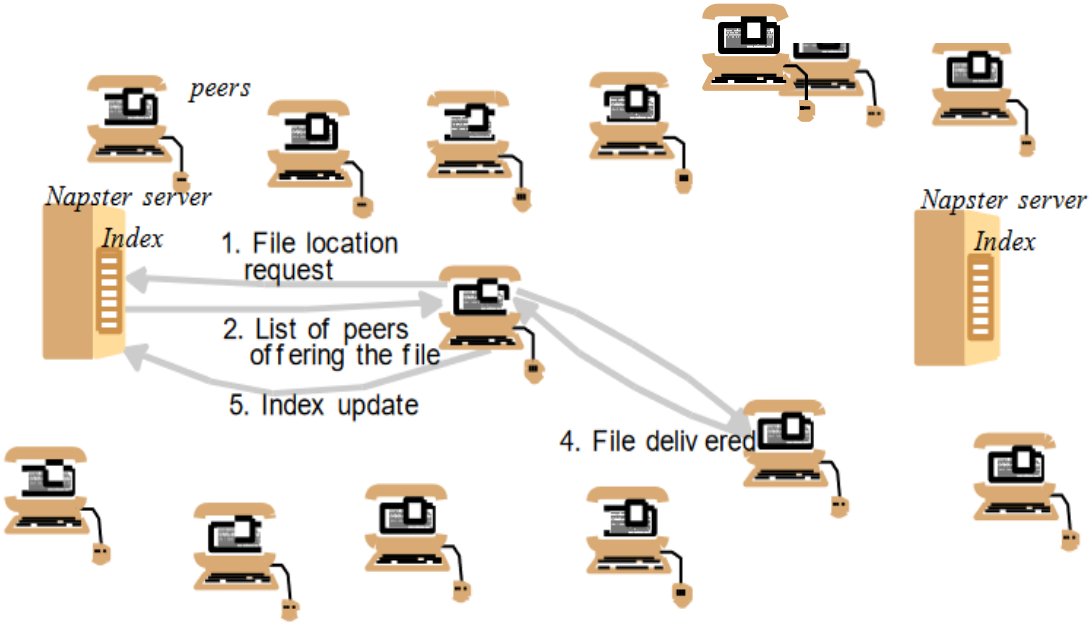


Fig 3.2: Napster

3.3 PEER TO PEER MIDDLEWARE

- ✓ Middleware is the software that manages and supports the different componentsof a distributed system.
- ✓ In essence, it sits in the middle of the system.
- ✓ Middleware is usually off Middleware is usually off-the-shelf rather than specially written software.
- ✓ A key problem in Peer-to-Peer applications is to provide a way for clients to access data resources efficiently.
- ✓ Peer clients need to locate and communicate with any available resource, even though resources may be widely distributed and configuration may be dynamic, constantly adding and removing resources and connections.
- ✓ The following are the functional requirements of the middleware:
    - ➢ Simplify construction of services across many hosts in wide network
    - ➢ Add and remove resources at will
    - ➢ Add and remove new hosts at will
    - ➢ Interface to application programmers should be simple and independentof types of distributed resources
- ✓ The following are the non functional requirements of the middleware:
    - ▪ Global Scalability
    - ▪ Load Balancing
    - ▪ Optimization for local interactions between neighboring peers
    - ▪ Accommodation to highly dynamic host availability

- Security of data in an environment simplify construction of services across many hosts in wide network
- Anonymity, deniability and resistance to censorship

✓ Sharing and balancing across large numbers of computers pose major design challenges to the development of middleware.

✓ In order to locate an object, the knowledge of object location must be distributed throughout network through replication.

3.4 ROUTING OVERLAY

A routing overlay is a distributed algorithm for a middleware layer responsible for routing requests from any client to a host that holds the object to which the requestis addressed.

- Any node can access any object by routing each request through a sequence of nodes, exploiting knowledge at each of theme to locate the destination object.
- Global User IDs (GUID) also known as **opaque identifiers** are used as names, but they do not contain the location information.
- A client wishing to invoke an operation on an object submits a request including the object‟s GUID to the routing overlay, which routes the request to a node at which a replica of the object resides.
- They are actually sub-systems within the peer-to-peer middleware that are meant locating nodes and objects.
- They implement a routing mechanism in the application layer.
- They ensure that any node can access any object by routing each request thru a sequence of nodes

Features of GUID:

✓ They are pure names or opaque identifiers that do not reveal anything about the locations of the objects.

✓ They are the building blocks for routing overlays.

✓ They are computed from all or part of the state of the object using a function that deliver a value that is very likely to be unique. Uniqueness is then checked against all other GUIDs.

✓ They are not human understandable.

Process of Routing Overlay

∗ Client submits a request including the object GUID, routing overlay routes the request to a node at which a replica of the object resides.

∗ A node introduces a new object by computing its GUID and announces it to the routing overlay.

∗ Note that clients can remove an object and also nodes may join and leave the service

Types of Routing Overlays

1. DHT – Distributed Hash Tables. GUIDs are stored based on hash values.
2. DOLR – Distributed Object Location and Routing. DOLR is a layer over the DHT that maps GUIDs and address of nodes. GUIDs host address is notified using the Publish() operation.

CASE STUDIES

3.5 PASTRY

Pastry is a generic, scalable and efficient substrate for peer-to-peer applications. Pastry nodes form a decentralized, self-organizing and fault-tolerant overlay network within the Internet.

- Pastry provides efficient request routing, deterministic object location, and load balancing in an application-independent manner.
- Furthermore, Pastry provides mechanisms that support and facilitate application-specific object replication, caching, and fault recovery.
- Each node in the Pastry network has a unique, uniform random identifier (nodeId) in a circular 128-bit identifier space.
- When presented with a message and a numeric 128-bit key, a Pastry node efficiently routes the message to the node with a nodeId that is numerically closest to the key, among all currently live Pastry nodes.
- The expected number of forwarding steps in the Pastry overlay network is O(log N), while the size of the routing table maintained in each Pastry node is only O(log N) in size (where N is the number of live Pastry nodes in the overlay network).
- At each Pastry node along the route that a message takes, the application is notified and may perform application-specific computations related to the message.
- Each Pastry node keeps track of its L immediateneighbors in the nodeId space called the leaf set, and notifies applications of new node arrivals, node failures and node recoveries within the leaf set.
- Pastry takes into account locality (proximity) in the underlying Internet.
- It seeks to minimize the distance messages travel, according to a scalar proximity metric like the ping delay.
- Pastry is completely decentralized, scalable, and self-organizing; it automatically adapts to the arrival, departure and failure of nodes.

Pastry's Routing Algorithm

The routing algorithm involves the use of a routing table at each node to route messages efficiently. This is divided into two stages:

- **First Stage:**
  - In this stage a routing scheme is used, that routes messages correctly but inefficiently without a routing table.

- Each active node stores a leaf set – a vector L (of size 2l) containing the GUIDs and IP addresses of the nodes whose GUIDs are numerically closest on either side of its own (l above and l below).

- Leaf sets are maintained by Pastry as nodes join and leave. Even after a node failure, they will be corrected within a short time.

- In the Pastry system that the leaf sets reflect a recent state of the system and that they converge on the current state in the face of failures up to some maximum rate of failure.

- The GUID space is treated in circular fashion.

- GUID 0‟s lower neighbour is 2128–1.

- Every leaf set includes the GUIDs and IP addresses of the current node‟simmediateneighbours.
- A Pastry system with correct leaf sets of size at least 2 can route messages to any GUID trivially as follows: any node A that receives a messageM with destination address D routes the message by comparing D with its own GUIDA and with each of the GUIDs in its leaf set and forwarding M to the node amongstthem that is numerically closest to D.

➢ **Second Stage:**

- This has a full routing algorithm, which routes a request to any node in O(log N) messages.

- Each Pastry node maintains a tree-structured routing table with GUIDs andIP addresses for a set of nodes spread throughout the entire range of 2^{128} possible GUID values, with increased density of coverage for GUIDs Numerically close to its own.

- The structure of routing table is: GUIDs are viewed as hexadecimal values and the table classifies GUIDs based on their hexadecimal prefixes.

- The table has as many rows as there are hexadecimal di gits in a GUID, so forthe prototype Pastry system that wear describing, there are 128/4 = 32 rows.

- Any row n contains 15 entries – one for each possible value of the nthhexadecimal digit, excluding the value in the local node ‟s GUID.
- Each entry in the table points to one of the potentially many nodes whose GUIDs have the relevant prefix.

- The routing process at any node A uses the information in its routing table R and leaf set L to handle each request from an application and each incoming message from another node according to the algorithm.

Pastry's Routing Algorithm

To handle a message M addressed to a node D (where R[p,i] is the element at column i, row p of the routing table):

1. If (L-l < D <Ll) { // the destination is within the leaf set or is the current node.

2. Forward M to the element Li of the leaf set with GUID closest to D or thecurrent node A.

3. } else { // use the routing table to despatch M to a node with a closer GUID

4. Find p, the length of the longest common prefix ofD and A,.and i, the (p+1)thhexadecimal digit of D.

5. If (R[p,i] • null) forward M to R[p,i] // route M to a node with a longer common prefix.

6. else { // there is no entry in the routing table.

7. Forward M to any node in L or R with a common prefix of length p but aGUID that is numerically closer.

}

}

Locality

* The Pastry routing structure is redundant.
* Each row in a routin g table contains 16 entries.
* The entries in the ithrow give the addresses of 16 nodes with GUIDs with i–1 initial hexadecimal digits thatmatch the current node ‟s GUID and an ith di git that takes eachof the possible hexadecimal mal values.

Fault tolerance

- The Pastry routing algorithm assumes that allentries in routing tables and leaf sets refer to live, correctly functioning nodes.
- All nodessend „heartbeat‟ messages i.e., messages sent at fixed time intervals to indicate that thesender is alive to neighbouring nodes in their leaf sets, but information about failednodes detected in this manner may not be disseminated sufficiently rapidly to eliminaterouting errors.

Dependability

- Dependability measures include the use of acknowledgements at each hop in the routing algorithm.
- If the sending host does not receive an acknowledgement after aspecified timeout, it selects an alternative route and retransmits the message.
- The nodethat failed to send an acknowledgement is then noted as a suspected failure.
- To detect failed nodes each Pastry node periodically sends aheartbeat message to its immediate neighbour to the left (i.e., with a lower GUID) in the leaf set.
- Each node also records the time of the last heartbeat message received from its immediate neighbour on the right (with a higher GUID).
- If the interval since the lastheartbeat exceeds a timeout threshold, the detecting node starts a repair procedure thatinvolves contacting the remaining nodes in the

leaf set with a notification about thefailed node and a request for suggested replacements.

3.6 TAPESTRY

- Tapestry is a decentralized distributed system.
- It is an overlay network that implements simple key-based routing.
- Each node serves as both an object store and a router that applications can contact to obtain objects.
- In a Tapestry network, objects are "published" at nodes, and once an object has been successfully published, it is possible for any other node in the network to find the location at which that objects is published.
- Identifiers are either NodeIds, which refer to computers that perform routing operations, or GUIDs, which refer to the objects.
- For any resource with GUID G there is a unique root node with GUID RG that is numerically closest to G.
- Hosts H holding replicas of G periodically invoke publish(G) to ensure that newly arrived hosts become aware of the existence of G.

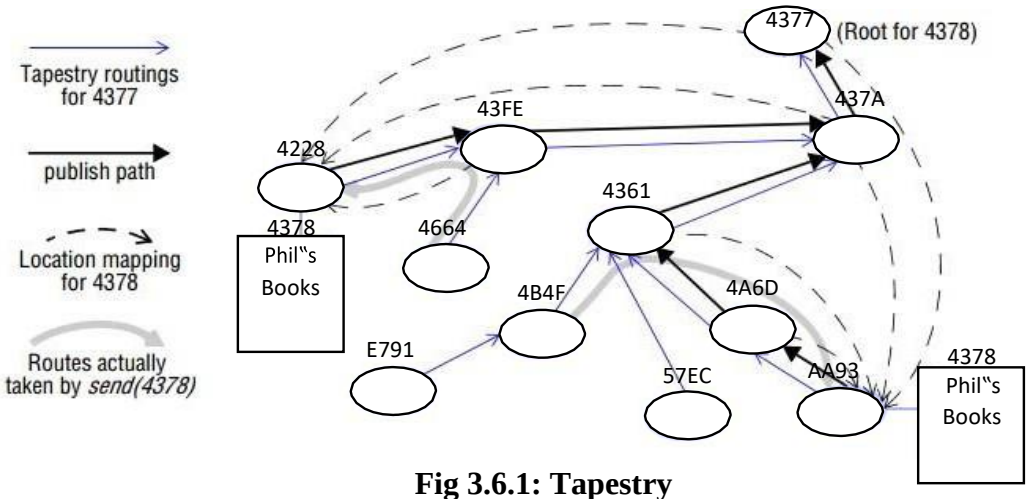


Fig 3.6.1: Tapestry

Unstructured peer-to-peer Networks

- ✓ In structured approaches, there is an overall global policy governing the topology of the network, the placement of objects in the network and the routing or search functions used to locate objects in the network.
- ✓ There is a specific data structure underpinning the associated overlay and a set of algorithms operating over that data structure.
- ✓ These algorithms are efficient and time bounded.
- ✓ In unstructured approaches, there is no overall control over the topology or the placement of objects within the network.

- ✓ The overlay iscreated in an ad hoc manner, with each node that joins the network following simple, local rules to establish connectivity.
- ✓ In particular, a joining node will establishcontact with a set of neighbours knowing that the neighbours will also be connected tofurtherneighbours and so on, forming a network that is fundamentally decentralized andself-organizing g and hence resilient to node failure.
- ✓ To locate a given object, it is thennecessary to carry out a search of the resultant network topology.
- ✓ This approachcannot offer guarantees of being able to find the object and performance will beunpredictable.
- ✓ There is a real risk of generating excessive message traffic tolocate objects.

Differences between structured and unstructured networks

Structured Network	Unstructured Network
This guarantees to locate objects and can offer time and complexity bounds on this operation. So it has relatively low message overhead.	This is self-organizing and resilient to node failure.
This needs complexoverlay structures, which can bedifficult and costly to achieve, especially in highly dynamic environments.	Probabilistic and hence cannot offer absolute guarantees on locating objects; prone to excessive messaging overhead which can affect scalability

3.7 Digital Signature

The Digital Signature is a technique which is used to validate the authenticity and integrity of the message. We know that there are four aspects of security: privacy, authentication, integrity, and non-repudiation. We have already discussed the first aspect of security and other three aspects can be achieved by using a digital signature.

The basic idea behind the Digital Signature is to sign a document. When we send a document electronically, we can also sign it. We can sign a document in two ways: to sign a whole document and to sign a digest.

Signing the Whole Document

- In Digital Signature, a public key encryption technique is used to sign a document. However, the roles of a public key and private key are different here. The sender uses a private key to encrypt the message while the receiver uses the public key of the sender to decrypt the message.
- In Digital Signature, the private key is used for encryption while the public key is used for decryption.
- Digital Signature cannot be achieved by using secret key encryption.

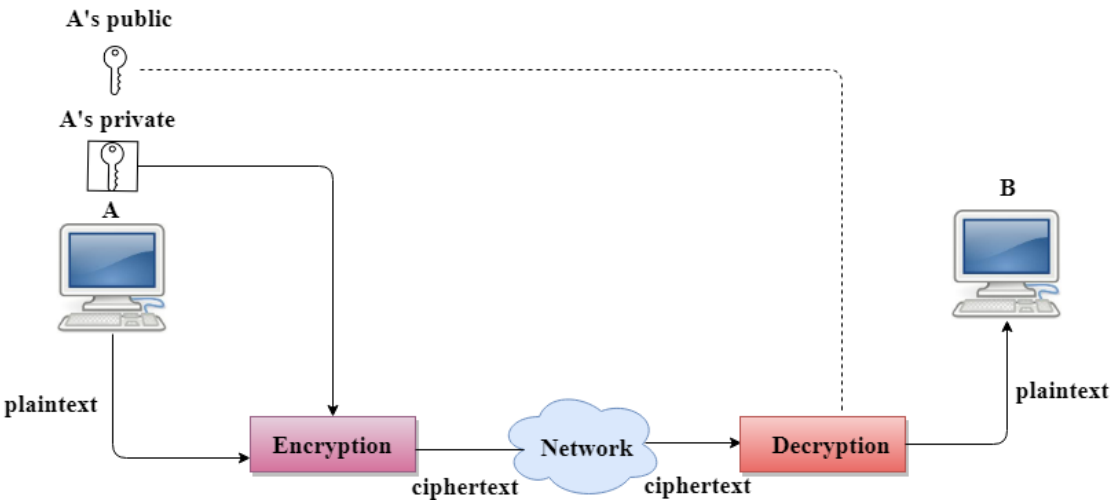


Fig 3.7.1: Encryption and Decryption

Digital Signature is used to achieve the following three aspects:

- **Integrity:** The Digital Signature preserves the integrity of a message because, if any malicious attack intercepts a message and partially or totally changes it, then the decrypted message would be impossible.
- **Authentication:** We can use the following reasoning to show how the message is authenticated. If an intruder (user X) sends a message pretending that it is coming from someone else (user A), user X uses her own private key to encrypt the message. The message is decrypted by using the public key of user A. Therefore this makes the message unreadable. Encryption with X's private key and decryption with A's public key results in garbage value.
- **Non-Repudiation:** Digital Signature also provides non-repudiation. If the sender denies sending the message, then her private key corresponding to her public key is tested on the plaintext. If the decrypted message is the same as the original message, then we know that the sender has sent the message.

Signing the Digest

- Public key encryption is efficient if the message is short. If the message is long, a public key encryption is inefficient to use. The solution to this problem is to let the sender sign a digest of the document instead of the whole document.
- The sender creates a miniature version (digest) of the document and then signs it, the receiver checks the signature of the miniature version.
- The hash function is used to create a digest of the message. The hash function creates a fixed-size digest from the variable-length message.
- The two most common hash functions used: MD5 (Message Digest 5) and SHA-1 (Secure Hash Algorithm 1). The first one produces 120-bit digest while the second one produces a 160-bit digest.
- A hash function must have two properties to ensure the success:
    - First, the digest must be one way, i.e., the digest can only be created from the message but not vice versa.
    - Second, hashing is a one-to-one function, i.e., two messages should not create the same digest.

Following are the steps taken to ensure security:

- The miniature version (digest) of the message is created by using a hash function.
- The digest is encrypted by using the sender's private key.
- After the digest is encrypted, then the encrypted digest is attached to the original message and sent to the receiver.
- The receiver receives the original message and encrypted digest and separates the two. The receiver implements the hash function on the original message to create the second digest, and it also decrypts the received digest by using the public key of the sender. If both the digests are same, then all the aspects of security are preserved.

At the Sender site

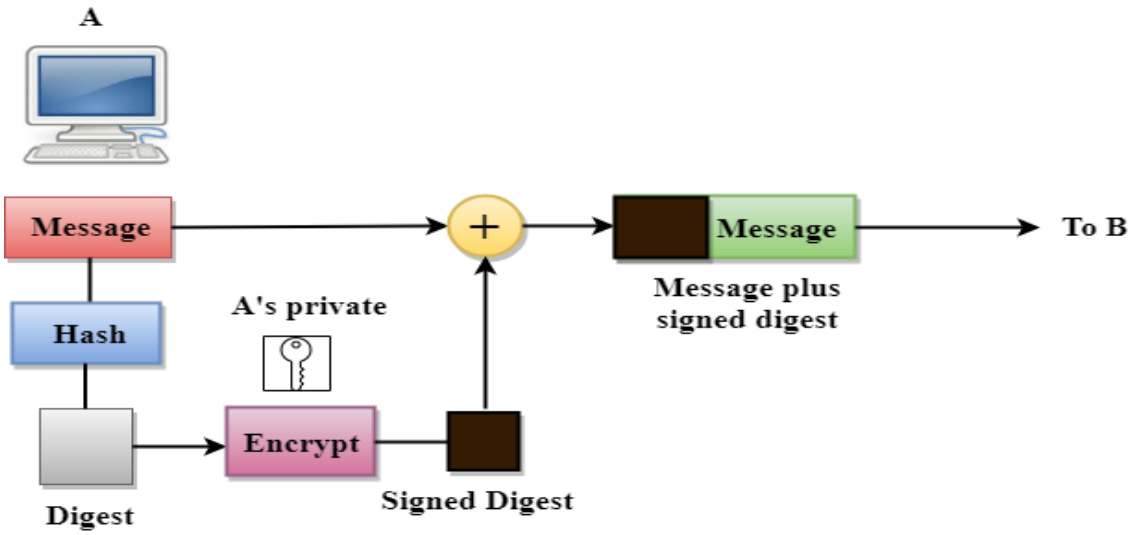


Fig 3.7.2: Data Encryption (Sender)

At the Receiver site

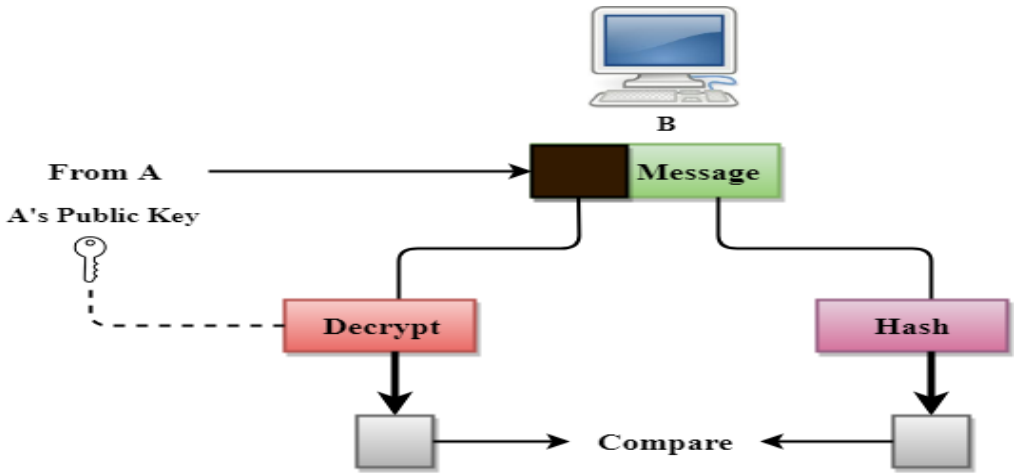


Fig 3.7.3: Data Encryption (Receiver)

3.8 DISTRIBUTED FILE SYSTEMS AND DEBUGGING

Distributed file systems support the sharing of information in the form of files and hardware resources.

- ✓ The purpose of a distributed file system (DFS) is to allow users of physically distributed computers to share data and storage resources by using a common file system.
- ✓ A typical configuration for a DFS is a collection of workstations and mainframes connected by a local area network (LAN).
- ✓ A DFS is implemented as part of the operating system of each of the connected computers.
- ✓ The following are the modules of a file systems:
  - ∗ Directory module: Relates file names to the disk.
  - ∗ File module: Relates file ID to the files
  - ∗ Access Control module: Manages access rights
  - ∗ File access module: Controls read or write operation on the files
  - ∗ Block module: Allocates disk blocks and helps in accessing them
  - ∗ Device module: Manages disk I/O and buffering.

File Attribute Record Structure

- ∗ File length specifies the size of the file.
- ∗ Creation Timestamp is the time at which the file was created.
- ∗ Read Timestamp is the time at which the last read operation was made.
- ∗ Write Timestamp is the time at which the last write operation was made.
- ∗ Reference Count is the number or links to the file.
- ∗ Owner is the creator of the file who has full access rights.
- ∗ File type describes the content of the file.
- ∗ Access Control List contains the list of users and the access rights given to them.

Unix File Systems

The time sharing Unix file system is a model for the distributed file systems. Thefollowing some of the unix commands to access files:

- ➢ filedes= open(name, mode): Opens an already existing file.
- ➢ filedes=creat(name, mode): Creates a new file
- ➢ status= close(filedes): Closes a file.
- ➢ count= read(filedes, buffer, n): Transfers n bytes from the fields to buffer and returns the number of bytes transferred.
- ➢ count= write(filedes, buffer, n): Transfers n bytes from the fields to buffer and returns the number of bytes transferred. This advances the write file pointer.
- ➢ pos= lseek(filedes, offset, whence): Moves the read-write pointer to offset

depending on whence.

- status=unlink(name): renames the specified file.
- status=unlink(name1, name2): Adds a new name (name2) for a file (name1).
- status= stat(name, buffer): Gets the file attributes for file into the buffer.

Characteristics of a File system

- ✓ File systems are responsible for the organization, storage, retrieval, naming, sharing and protection of files.
- ✓ Data and attributes are the primary characteristics of a file system.
- ✓ The data consist of a sequence of data items .
- ✓ The attributes are held as a single record containing information such as the length of the file, timestamps, file type, owner‟s identity and access control lists.
- ✓ A directory is a special file that provides a mapping from text names to internal fileidentifiers.
- ✓ The term metadata is used to refer to all of the extra information stored bya file system that is needed for the management of files.
- ✓ It includes file attributes , directories and all the other persistent information usedby the file system.
- ✓ The file system also maintains the access control lists.

Requirements of distributed file systems

Transparency is operating in such a way as to not be perceived by users. The distributedfile system demands the following transparency requirements:

- **Login Transparency:** User can log in at any host with uniform login procedureand perceive a uniform view of the file system.
- **Access Transparency:** Client process on a host has uniform mechanism to accessall files in system regardless of files are on local/remote host.
- **Location Transparency:** The names of the files do not reveal their physicallocation.
- **Concurrency Transparency:** An update to a file should not have effect on thecorrect execution of other process that is concurrently sharing a file.
- **Replication Transparency:** Files may be replicated to provide redundancy foravailability and also to permit concurrent access for efficiency.

Apart from the above transparency properties, the file systems also need the following:

- **Fault Tolerance:** It is a design that enables a system to continue operation, possibly at a reduced level, rather than failing completely, when some part of the system fails.
- **Network Transparency:** Same access operation as if they are local files.

- **Location Independence:** The file name should not be changed when the physical location of the file changes.
- **User Mobility:** User should be able to access the file from any where.
- **File Mobility:** Moves files from one place to the other in a running system.

Examples:

- ✓ **The GOOGLE File System:** A scalable distributed file system for large distributed data-intensive applications. It provides fault tolerance while running on inexpensive commodity hardware, and it delivers high aggregate performance to a large number of clients.
- ✓ **The CODA distributed file system:** Developed at CMU, incorporates many distinguished features which are not present in any other distributed file system.

Debugging

Distributed problems occur at all logical levels of a distributed system, not just low-level physical machines. Distributed problems get worse at higher levels of the system, due to recursion. Distributed bugs often show up long after they are deployed to a system. Distributed bugs can spread across an entire system.

Distributed systems differ from single-machine programs in ways that are simultaneously positive in providing systems with special capabilities, and negative in presenting software-development and operational challenges.

Heterogeneity

A distributed system's nodes may include mobile phones, laptops, server-class machines, and more. This hardware and software diversity in node resources and network connectivity can make a distributed system more robust, but this heterogeneity forces developers to manage compatibility during both development and debugging.

Concurrency

Simultaneous operation by multiple nodes leads to concurrency, which can make a distributed system outperform a centralized system. However, concurrency may introduce race conditions and deadlocks, which are notoriously difficult to diagnose and debug. Additionally, networks introduce packet delay and loss, exacerbating the issues of understanding and debugging concurrency.

Distributed state Distributing system state across multiple nodes can remove a central point of failure and improve scalability, but distributed state requires intricate node coordination to synchronize state across nodes—for example, nodes must ensure their local states are consistent. Potential inconsistencies are prevented by distributed algorithms, such as those that guarantee a particular flavor of data consistency and cache coherence. Developers may find it difficult, or even impossible, to reconstruct the global state of the system when it is distributed on many nodes. This complicates bug diagnosis and validation.

UNIT IV FAULT TOLERANCE AND SECURITY

Fault Tolerance Services – Atomic Commit Protocols – Concurrency Control In Distributed Transaction – Distributed Deadlocks – Transaction Recovery – Overview Of Security Techniques – Access Control – Cryptography Algorithms – Kerberos.

4.1 FAULT TOLERANCE SERVICES

- Fault Tolerance simply means a system's ability to continue operating uninterrupted despite the failure of one or more of its components. This is true whether it is a computer system, a cloud cluster, a network, or something else.
- Fault Tolerant Services are obtained by using replication.

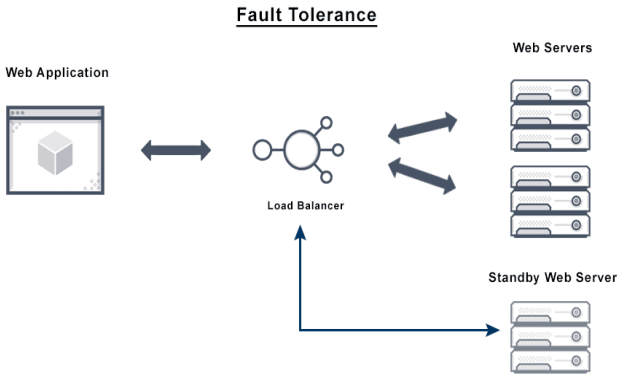


Fig 4.1 Fault Tolerance

- By using multiple independent server's replicas each managing replicated data help in designing a service which exhibit graceful degradation using partial failures and improve overall server performance.

Fault Tolerance Services Models

There are two main fault tolerance service Model

1. **Passive (Primary – backup) Replication**
2. **Active Replication**

Passive (Primary – backup) Replication

– In a passive model of replication for fault tolerance, there is at any time a single primary replica manager and one or more secondary replica managers

i.e., "backups" or "slaves".

- In the pure form of the model, frontend communicate only with the primary replica manager to obtain the service.
- The primary replica manager executes the operation and sends copies of the updated data to the backups.
- If the primary fails, one of the backup is promoted to act as the primary.
- The sequence of events when a client requests an operations to be performed as follows:

Request: The frontend issues the request, containing unique identifier to the primary replica manager.

Coordination: The primary takes each request automatically in the order in which if receives it. It checks the unique identifier, in case it has already executed the request and if so it simply resend the response.

Execution: The primary executes the request and stores the response.

Agreement: If the request is an update then the primary sends the update state, the response and the unique identifier to all the backups. The backups send the acknowledgement.

Response: The Responds (primary) to the frontend, which sends the response back to the client.

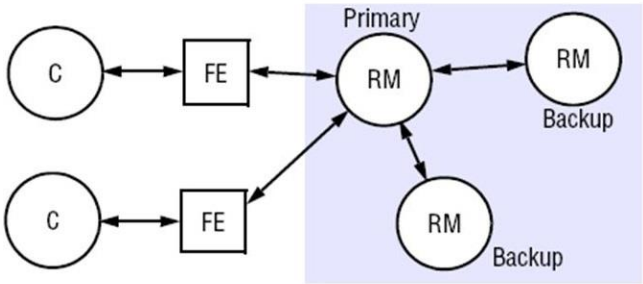
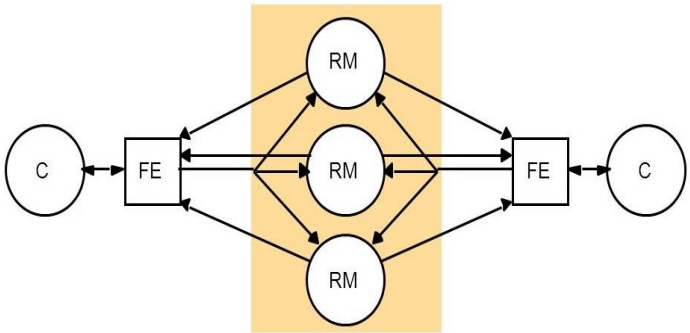


Fig: 4.2. The passive (primary backup) model for fault tolerance

Active Replication

- In the active model of replication for fault tolerance, the replica managers are state machines that play equivalent roles and organized as a group.
- Frontend multicast their request to the group of replica managers and all the replica manager process the request independently but identically and reply.



4.1 ATOMIC COMMIT PROTOCOL

An **atomic commit protocol** (ACP) is a **distributed** algorithm used to ensure the atomicity property of transactions in **distributed** database **systems**. This added cost is due to the overhead of the required coordination messages and log writes at each involved database site to achieve atomicity.

- Atomic Commit Protocol is a protocol used by database manager to ensure that all the sub transactions are consistently committed or aborted.
- In this each server applies local concurrency control to its own objects which ensures the transaction are serialized locally as well as serialize globally.
- When a distributed transaction comes to the end, either all the servers commit the transaction or abort the transaction.
- There are two atomic commit protocol used in distributed database system
  - ***Two Phase Commit Protocol***
  - ***Three Phase Commit Protocol***

Two Phase Commit Protocol

- The **two-phase commit protocol** is designed to allow any participant to abort its part of a transaction.
- Due to the requirement for atomicity, if one part of a transaction is aborted, then the whole transaction must be aborted.
- In the first phase of the protocol, each participant votes for the transaction to be committed or aborted.
- Once a participant has voted to commit a transaction, it is not allowed to abort it.
- In the second phase of the protocol, every participant in the transaction carries out the joint decision.
- If any one participant votes to abort, then the decision must be to abort the transaction.
- If all the participants vote to commit, then the decision is to commit the transaction.
- The following are the operations in two phase commit protocol:
  - **canCommit?(trans)□Yes / No**: Call from coordinator to participant to ask whether it can commit a transaction. Participant replies with its vote.
  - **doCommit(trans):**Call from coordinator to participant to tell participant to commit its part of a transaction.
  - **doAbort(trans):** Call from coordinator to participant to tell participant to abort its part of a transaction.
  - **haveCommitted(trans, participant):** Call from participant to coordinator to confirm that it has committed the transaction.
  - **getDecision(trans) □Yes / No:** Call from participant to coordinator to ask for the decision on a transaction when it has voted Yes but has still had no reply after some delay. Used to recover from server crash or delayed messages

Phase 1 (voting phase):

1. The coordinator sends a canCommit? request to each of the participants in the transaction.

2. When a participant receives a canCommit? request it replies with its vote (Yes or No) to the coordinator. Before voting Yes, it prepares to commit by saving objects in permanent storage. If the vote is No, the participant aborts immediately.

Phase 2 (completion according to outcome of vote):

The coordinator collects the votes (including its own).

(a) If there are no failures and all the votes are Yes, the coordinator decides to commit the transaction and sends a doCommitrequest to each of the participants.

(b) Otherwise, the coordinator decides to abort the transaction and sends doAbortrequests to all participants that voted Yes.

Participants that voted Yes are waiting for a doCommitor doAbortrequest from the coordinator. When a participant receives one of these messages it acts accordingly and, in the case of commit, makes a have Committedcall as confirmation to the coordinator.

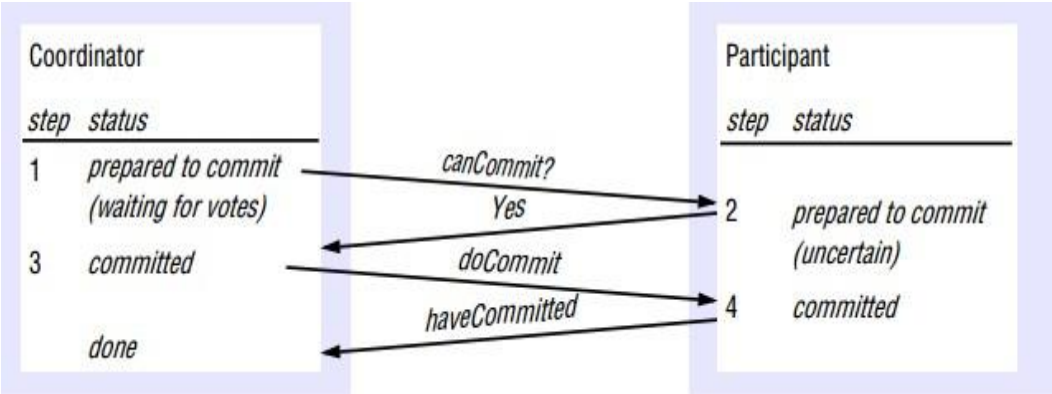


Fig 4.4. Two phase commit protocol

Two phase commit for nested transactions

- The outermost transaction in a set of nested transactions is called the top-level transaction.
- Transactions other than the top-level transaction are called sub-transactions.
- The following are the operations are allowed:
  - openSubTransaction(trans) →subTrans: Opens a new subtransaction whose parent is trans and returns a unique subtransaction identifier.
  - getStatus(trans)→committed, aborted, provisional: Asks the coordinator to report on the status of the transaction trans. Returns values representing one of the following: committed, aborted or provisional.
- When a sub-transaction completes, it makes an independent decision either to commit provisionally or to abort.
- A coordinator for a sub-transaction will provide an operation to open a sub-transaction, together with an operation enabling that coordinator to enquire whether its parent has yet committed or aborted.

4.3 CONCURRENCY CONTROL IN DISTRIBUTED TRANSACTIONS

Fundamentals of Concurrency control

- ❖ Concurrency is needed when multiple users want to access the same data at the same time.
- ❖ Concurrency control (CC) ensures that correct results for parallel operations are generated.
- ✓ CC provides rules, methods, design methodologies and theories to maintain the consistency of components operating simultaneously while interacting with the same object.

All concurrency control protocols are based on serial equivalence and are derived from rules of conflicting operations:

- ✓ Locks used to order transactions that access the same object according to request order.
- ✓ Optimistic concurrency control allows transactions to proceed until they are ready to commit, whereupon a check is made to see any conflicting operation on objects.
- ✓ Timestamp ordering uses timestamps to order transactions that access the same object according to their starting time.

Serial Equivalence

- ✓ If these transactions are done one at a time in some order, then the final result will be correct.
- ✓ If we do not want to sacrifice the concurrency, an interleaving of the operations of transactions may lead to the same effect as if the transactions had been performed one at a time in some order.
- ✓ We say it is a serially equivalent interleaving.
- ✓ The use of serial equivalence is a criterion for correct concurrent execution to prevent lost updates and inconsistent retrievals.

Conflicting Operations

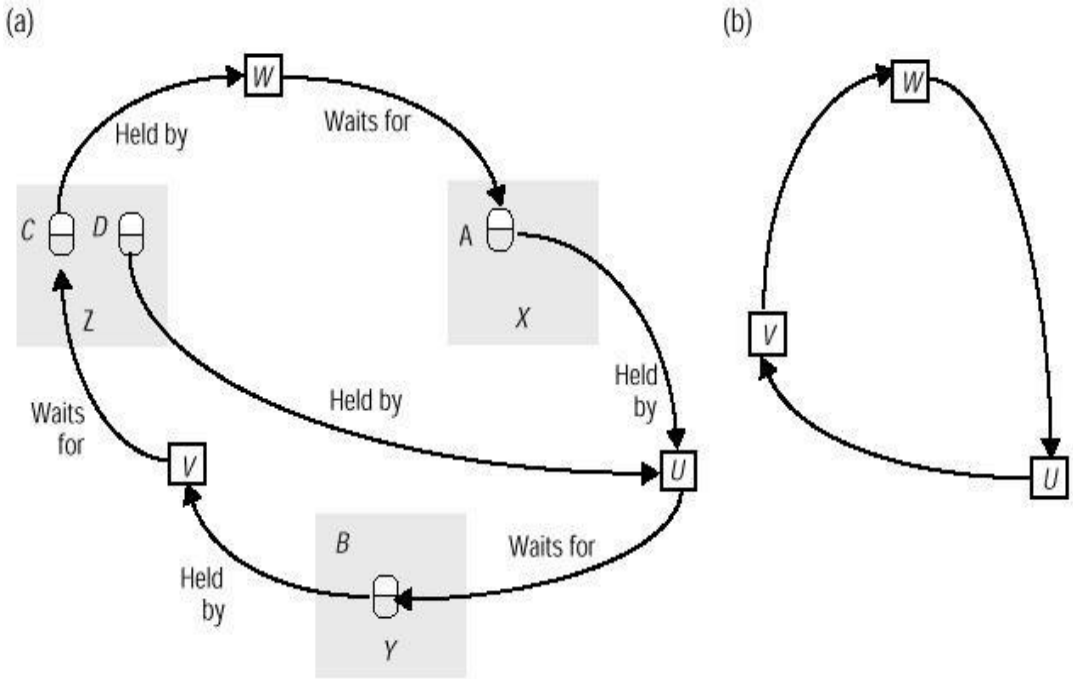
- ✓ When we say a pair of operations conflicts we mean that their combined effect depends on the order in which they are executed. E.g. read and write
- ✓ There are three ways to ensure serializability:
  - ➢ Locking
  - ➢ Timestamp ordering
  - ➢ Optimistic concurrency control

Process 1	**Process 2**	**Conflict**	**Reason**
Read	Read	No	-
Read	Write	Yes	The result of these operations dependon their order of execution.
Write	Write	Yes	The result of these operations depend on their order of execution.

4.3 DISTRIBUTED DEADLOCKS

- ➢ A cycle in the global wait-for graph (but not in any single local one) represents a distributed deadlock.
- ➢ A deadlock that is detected but is not really a deadlock is called a phantom deadlock.
- ➢ Two-phase locking prevents phantom deadlocks; autonomous aborts may cause phantom deadlocks.
- ➢ Permanent blocking of a set of processes that either compete for system resources or communicate with each other is deadlock.
- ➢ No node has complete and up-to-date knowledge of the entire distributed system. This is the cause of deadlocks.

Fig 4.5. Distributed deadlocks and wait for graphs



Types of distributed deadlock

- **Resource deadlock:** Set of deadlocked processes, where each process waits for a resource held by another process (e.g., data object in a database, I/O resource on a server)
- **Communication deadlocks:** Set of deadlocked processes, where each process waits to receive messages (communication) from other processes in the set.

Local and Global Wait for Graphs

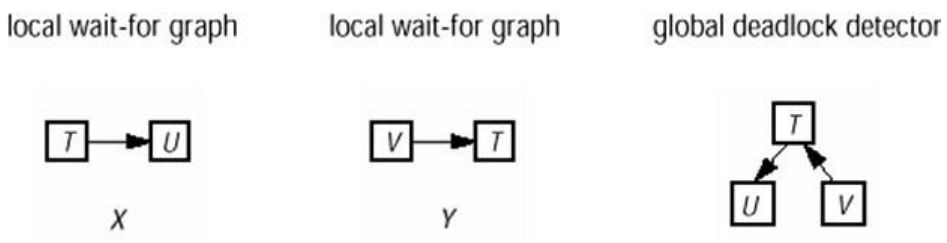


Fig 4.6. Local and Global Wait for Graphs

Edge Chasing

- When a server notes that a transaction T starts waiting for another transaction U, which is waiting to access a data item at another server, it sends a probe containing

⟨T→U⟩ to the server of the data item at which transaction U is blocked.

- **Detection:** receive probes and decide whether deadlock has occurred and whether to forward the probes.

- When a server receives a probe ⟨T→U⟩ and finds the transaction that U is waiting for, say V, is waiting for another data item elsewhere, a probe ⟨T→U→V⟩ is forwarded.
- **Detection:** receive probes and decide whether deadlock has occurred and whether to forward the probes.
  When a server receives a probe ⟨T→U⟩ and finds the transaction that U is waiting for, say V, is waiting for another data item elsewhere, a probe ⟨T→U→V⟩ is forwarded.
- **Resolution:** select a transaction in the cycle to abort

Transaction priorities

- Every transaction involved in a deadlock cycle can cause deadlock detection to be initiated.
- The effect of several transactions in a cycle initiating deadlock detection is that detection may happen at several different servers in the cycle, with the result that more than one transaction in the cycle is aborted.

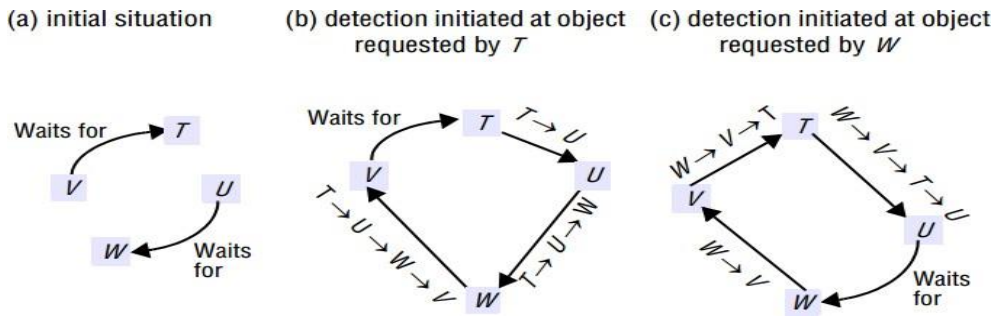


Fig 4.7: Initiated probes

- Consider transactions T, U, V and W, where U is waiting for Wand V is waiting for T.
- At about the same time, T requests the object held by U and W requests the object held by V.
- Two separate probes, <T →U > and <W →V >, are initiated by the servers of these objects and are circulated until deadlocks are detected by each of the servers.
- The cycle is <T →U→W →V →T>and, the cycle is <W →V →T →U →W >.
- In order to ensure that only one transaction in a cycle is aborted, transactions are given priorities in such a way that all transactions are totally ordered.
- In order to ensure that only one transaction in a cycle is aborted, transactions are given priorities in such a way that all transactions are totally ordered.
- The problem is that the order in which transactions start waiting can determine whether or not a deadlock will be detected.
- The above pitfall can be avoided by using as scheme in which coordinators save copies of all the probes received on behalf of each transaction in a **probe queue.**
- When a transaction starts waiting for an object, it forwards the probes in its queue to the server of the object, which propagates the probes on downhill routes.

4.4 TRANSACTION RECOVERY

- Transactions may be performed effectively using distributed transaction processing.
- However, there are instances in which a transaction may fail for a variety of causes. **System failure, hardware failure, network error, inaccurate or invalid data, application problems**, are all probable causes.
- Transaction failures are impossible to avoid. These failures must be handled by the distributed transaction system.

- When mistakes arise, one must be able to identify and correct them. Transaction Recovery is the name for this procedure.
- In distributed databases, the most difficult procedure is recovery. It is extremely difficult to recover a communication network system that has failed.
- Let us consider the following scenario to analyze how transaction fail may occur. Let suppose, we have two-person X and Y. X sends a message to Y and expects a response, but Y is unable to receive it.

The following are some of the issues with this circumstance:

  - The message was not sent due to a network problem.
  - The communication sent by location B was not delivered to place A.
  - Location B was destroyed.
  - As a result, locating the source of a problem in a big communication network is extremely challenging.

- Distributed commit in the network is another major issue that can wreak havoc on a distributed database's recovery.
- One of the most famous methods of Transaction Recovery is the "Two-Phase Commit Protocol".
- The coordinator and the subordinate are the two types of nodes that the Two-Phase Commit Protocol uses to accomplish its procedures.
- The coordinator's process is linked to the user app, and communication channels between the subordinates and the coordinator are formed.
- The two-phase commit protocol contains two stages, as the name implies.
- The first step is the PREPARE phase, in which the transaction's coordinator delivers a PREPARE message.
- The second step is the decision-making phase, in which the coordinator sends a COMMIT message if all of the nodes can complete the transaction, or an abort message if at least one subordinate node cannot.

- **Centralized 2PC, Linear 2PC, and Distributed 2PC** are all ways that may be used to perform the 2PC.

Centralized 2 PC:

- ✓ Contact in the Centralized 2PC is limited to the coordinator's process, and no communication between subordinates is permitted.
- ✓ The coordinator is in charge of sending the PREPARE message to the subordinates, and once all of the subordinates' votes have been received and analyzed, the coordinator chooses whether to abort or commit.
- ✓ There are two stages to this method:
  - **The First Phase:** When a user desires to COMMIT a transaction during this phase, the coordinator sends a PREPARE message to all subordinates.
  - When a subordinate gets the PREPARE message, it either records a PREPARE log and sends a YES VOTE and enters the PREPARED state if the subordinate is willing to COMMIT; or it creates an abort record and sends a NO VOTE if the subordinate is not willing to COMMIT.
  - Because it knows the coordinator will issue an abort, a subordinate transmitting a NO VOTE does not need to enter a PREPARED state. In this situation, the NO VOTE functions as a veto since only one NO VOTE is required to cancel the transaction.
  - **Second Phase:** After the coordinator has reached a decision, it must communicate that decision to the subordinates.
  - If COMMIT is chosen, the coordinator enters the committing state and sends a COMMIT message to all subordinates notifying them of the choice.

- When the subordinates get the COMMIT message, they go into the committing state and send the coordinator an acknowledge (ACK) message.
- The transaction is completed when the coordinator gets the ACK messages. If the coordinator, on the other hand, makes an ABORT decision, it sends an ABORT message to all subordinates. In this case, the coordinator does not need to send an ABORT message to the NO VOTE subordinate(s).

Linear 2 PC:

- ✓ Subordinates in the linear 2PC, can communicate with one another. The sites are numbered 1 to N, with site 1 being the coordinator.
- ✓ As a result, the PREPARE message is propagated in a sequential manner. As a result, the transaction takes longer to complete than centralized or dispersed approaches. Finally, it is node N that sends out the Global COMMIT.

Distributed 2 PC:

- ✓ All of the nodes of a distributed 2PC interact with one another. Unlike other 2PC techniques, this procedure does not require the second phase.
- ✓ Furthermore, in order to know that each node has put in its vote, each node must hold a list of all participating nodes.
- ✓ When the coordinator delivers a PREPARE message to all participating nodes, the distributed 2PC gets started.
- ✓ When a participant receives the PREPARE message, it transmits his or her vote to all other participants.
- ✓ As a result, each node keeps track of every transaction's participants.

4.5 OVERVIEW OF SECURITY TECHNIQUES

Worst-case assumptions and design guidelines

- ❖ ***Interfaces are exposed:*** Distributed systems are composed of processes that offer services or share information. Their communication interfaces are necessarily open (to allow new clients to access them) – an attacker can send a message to any interface.
- ❖ ***Networks are insecure:*** For example, message sources can be falsified – messages can be made to look as though they came from Alice when they were actually sent by Mallory. Host addresses can be 'spoofed' – Mallory can connect to the network with the same address as Alice and receive copies of messages intended for her.
- ❖ ***Limit the lifetime and scope of each secret:*** When a secret key is first generated we can be confident that it has not been compromised. The longer we use it and the more widely it is known, the greater the risk. The use of secrets such as passwords and shared secret keys should be time-limited, and sharing should be restricted.
- ❖ ***Algorithms and program code are available to attackers:*** The bigger and the more widely distributed a secret is, the greater the risk of its disclosure. Secret encryption algorithms are totally inadequate for today's large-scale network environments. Best practice is to publish the algorithms used for encryption and authentication, relying only on the secrecy of cryptographic keys. This helps to ensure that the algorithms are strong by throwing them open to scrutiny by third parties.
- ❖ ***Attackers may have access to large resources:*** The cost of computing power is rapidly decreasing. We should assume that attackers will have access to the largest and most powerful computers projected in the lifetime of a system, then add a few orders of magnitude to allow for unexpected developments.
- ❖ ***Minimize the trusted base:*** The portions of a system that are responsible for the implementation of its security, and all the hardware and software components upon

which they rely, have to be trusted – this is often referred to as the trusted computing base. Any defect or programming error in this trusted base can produce security weaknesses, so we should aim to minimize its size. For example, application programs should not be trusted to protect data from their users.

Cryptography notations

KA	Alice's secret key
KB	Bob's secret key
KAB	Secret key shared between Alice and Bob
KApriv	Alice's private key (known only to Alice)
KApub	Alice's public key (published by Alice for all to read)
{M}K	Message M encrypted with key K
[M]K	Message M signed with key K

Cryptography

- Encryption is the process of encoding a message in such a way as to hide its contents.
- Modern cryptography includes several secure algorithms for encrypting and decrypting messages. They are all based on the use of secrets called keys. A cryptographic key is a parameter used in an encryption algorithm in such a way that the encryption cannot be reversed without knowledge of the key.
- There are two main classes of encryption algorithm in general use.
- The first uses shared secret keys – the sender and the recipient must share a knowledge of the key and it must not be revealed to anyone else. The second class of encryption algorithms uses public/private key pairs. Here the sender of a message uses a public key – one that has already been published by the recipient – to encrypt the message.

- The recipient uses a corresponding private key to decrypt the message. Although many principals may examine the public key, only the recipient can decrypt the message, because they have the private key.
- Both classes of encryption algorithm are extremely useful and are used widely in the construction of secure distributed systems.
- Public-key encryption algorithms typically require 100 to 1000 times as much processing power as secret-key algorithms, but there are situations where their convenience outweighs this disadvantage.

Uses of cryptography

- **Secrecy and integrity:**
  - ✓ Cryptography is used to maintain the secrecy and integrity of information whenever it is exposed to potential attacks – for example, during transmission across networks that are vulnerable to eavesdropping and message tampering.
  - ✓ This use of cryptography corresponds to its traditional role in military and intelligence activities.
  - ✓ It exploits the fact that a message that is encrypted with a particular encryption key can only be decrypted by a recipient who knows the corresponding decryption key.
  - ✓ Thus it maintains the secrecy of the encrypted message as long as the decryption key is not compromised (disclosed to non-participants in the communication) and provided that the encryption algorithm is strong enough to defeat any possible attempts to crack it.
  - ✓ Encryption also maintains the integrity of the encrypted information,

provided that some redundant information such as a checksum is included and checked.

Scenario 1. Secret communication with a shared secret key:

Alice wishes to send some information secretly to Bob. Alice and Bob share a secret
key KAB.

1. Alice uses KAB and an agreed encryption function E(KAB, M) to encrypt and send any number of messages {Mi}KAB to Bob. (Alice can go on using KAB as long as it is safe to assume that KAB has not been compromised.)
2. Bob decrypts the encrypted messages using the corresponding decryption function D(KAB, M).

Bob can now read the original message M. If the decrypted message makes sense, or better, if it includes some value agreed between Alice and Bob (such as a checksum of the message) then Bob knows that the message is from Alice and that it hasn't been tampered with. But there are still some problems:

Problem 1: How can Alice send a shared key KAB to Bob securely?

Problem 2: How does Bob know that any {Mi} isn't a copy of an earlier encrypted message from Alice that was captured by Mallory and replayed later? Mallory needn't have the key KAB to carry out this attack – he can simply copy the bit pattern that represents the message and send it to Bob later. For example, if the message is a request to pay some money to someone, Mallory might trick Bob into paying twice.

4.6 ACCESS CONTROL

- Access control is a security technique that regulates who or what can view or use resources in a computing environment. It is a fundamental concept in security that minimizes risk to the business or organization.
- There are two types of access control: physical and logical. Physical access control limits access to campuses, buildings, rooms and physical IT assets. Logical access control limits connections to computer networks, system files and data.
- To secure a facility, organizations use electronic access control systems that rely on user credentials, access card readers, auditing and reports to track employee access to restricted business locations and proprietary areas, such as data centers. Some of these systems incorporate access control panels to restrict entry to rooms and buildings.

How access control works

- These security controls work by identifying an individual or entity, verifying that the person or application is who or what it claims to be, and authorizing the access level and set of actions associated with the username or Internet Protocol (IP) address.

- Directory services and protocols, including **Lightweight Directory Access Protocol (LDAP)** and **Security Assertion Markup Language (SAML)**, provide access controls for authenticating and authorizing users and entities and enabling them to connect to computer resources, such as distributed applications and web servers.

- Organizations use different access control models depending on their compliance requirements and the security levels of information technology (IT) they are trying to protect.

Types of access control

The main models of access control are the following:

- **Mandatory access control (MAC).** This is a security model in which access rights are regulated by a central authority based on multiple levels of security. Often used in government and military environments, classifications are assigned to system resources and the operating system (OS) or security kernel. It grants or denies access to those resource objects based on the information security clearance of the user or device. For example, Security Enhanced Linux (SELinux) is an implementation of MAC on the Linux OS.
- **Discretionary access control (DAC).** This is an access control method in which owners or administrators of the protected system, data or resource set the policies defining who or what is authorized to access the resource. Many of these systems enable administrators to limit the propagation of access rights. A common criticism of DAC systems is a lack of centralized control.
- **Role-based access control (RBAC).** This is a widely used access control mechanism that restricts access to computer resources based on individuals or groups with defined business functions -- e.g., executive level, engineer level 1, etc. -- rather than the identities of individual users. The role-based security model relies on a complex structure of role assignments, role authorizations and role permissions developed using role engineering to regulate employee access to systems. RBAC systems can be used to enforce MAC and DAC frameworks.
- **Rule-based access control.** This is a security model in which the system administrator defines the rules that govern access to resource objects. Often, these rules are based on conditions, such as time of day or location. It is not uncommon to use some form of both rule-based access control and RBAC to enforce access policies and procedures.
- **Attribute-based access control (ABAC).** This is a methodology that manages access rights by evaluating a set of rules, policies and relationships using the attributes of users, systems and environmental conditions.

Implementing access control

- Access control is a process that is integrated into an organization's IT environment. It can involve identity management and access management systems. These systems provide access control software, a user database, and management tools for access control policies, auditing and enforcement.
- When a user is added to an access management system, system administrators use an automated provisioning system to set up permissions based on access control frameworks, job responsibilities and workflows.
- The best practice of least privilege restricts access to only resources that employees require to perform their immediate job functions.

Challenges of access control

- Many of the challenges of access control stem from the highly distributed nature of modern IT. It is difficult to keep track of constantly evolving assets as they are spread out both physically and logically. Some specific examples include the following:
  - dynamically managing distributed IT environments;
  - password fatigue;
  - compliance visibility through consistent reporting;

- centralizing user directories and avoiding application-specific silos; and
- data governance and visibility through consistent reporting.

❖ Modern access control strategies need to be dynamic. Traditional access control strategies are more static because most of a company's computing assets were held on premises. Modern IT environments consist of many cloud-based and hybrid implementations, which spreads assets out over physical locations and over a variety of unique devices. A singular security fence that protects on-premises assets is becoming less useful because assets are becoming more distributed.

Access control software

❖ There are many types of access control software and technology, and often, multiple components are used together to maintain access control. The software tools may be on premises, in the cloud or a hybrid of both. They may focus primarily on a company's internal access management or may focus outwardly on access management for customers. Some of the types of access management software tools include the following:

- reporting and monitoring applications
- password management tools
- provisioning tools
- identity repositories
- security policy enforcement tools

4.7CRYTOGRAPHIC ALGORITHM

- A message is encrypted by the sender applying some rule to transform the plaintext message (any sequence of bits) to a cipher text (a different sequence of bits).
- The recipient must know the inverse rule in order to transform the cipher text back into the original plaintext.
- Other principals are unable to decipher the message unless they also know the inverse rule.
- The encryption transformation is defined with two parts, a function E and a key K. The resulting encrypted message is written $\{M\}_K$.

$$M(K, M) = \{M\}_K$$

- The encryption function E defines an algorithm that transforms data items in plaintext into encrypted data items by combining them with the key and transposing them in a manner that is heavily dependent on the value of the key.
- We can think of an encryption algorithm as the specification of a large family of functions from which a particular member is selected by any given key.
- Decryption is carried out using an inverse function D, which also takes a key as a parameter. For secret-key encryption, the key used for decryption is the same as that used for encryption:

$$D(K, E(K, M)) = M$$

- Because of its symmetrical use of keys, secret-key cryptography is often referred to as symmetric cryptography, whereas public-key cryptography is referred to as asymmetric because the keys used for encryption and decryption are different.

Symmetric algorithms:

❖ If we remove the key parameter from consideration by defining $F_K([M]) =$

E(K,M) , then it is a property of strong encryption functions that $F_K([M])$ is relatively easy to compute, whereas the inverse, $F_K^{-1}([M])$, is so hard to compute that it is not feasible. Such functions are known as one-way functions. The effectiveness of any method for encrypting information depends upon the use of an encryption function FK that has this one-way property. It is this that protects against attacks designed to discover M given $\{M\}_K$.

- For well-designed symmetric algorithms, their strength against attempts to discover K given a plaintext M and the corresponding ciphertext $\{M\}_K$ depends on the size of K. This is because the most effective general form of attack is the crudest, known as a brute-force attack.
- The brute force approach is to run through all possible values of K, computing E (K, M) until the result matches the value of $\{M\}_K$ that is already known. If K has N bits then such an attack requires 2^{N-1} iterations on average, and a maximum of 2^N iterations, to find K.Hence the time to crack K is exponential in the number of bits in K.

Asymmetric algorithms

- When a public/private key pair is used, one-way functions are exploited in another way. The feasibility of a public-key scheme was first proposed by Diffie and Hellman [1976] as a cryptographic method that eliminates the need for trust between the communicating parties.
- The basis for all public-key schemes is the existence of trap-door functions. A trap-door function is a one-way function with a secret exit – it is easy to compute in one direction but infeasible to compute the inverse unless a secret is known. It was the possibility of finding such functions and using them in practical cryptography that Diffie and Hellman first suggested. Since then, several practical public-key schemes have been proposed and developed. They all depend upon the use of trap-door functions involving large numbers.
- The pair of keys needed for asymmetric algorithms is derived from a common root. For the RSA algorithm, the root is an arbitrarily chosen pair of very large prime numbers. The derivation of the pair of keys from the root is a one-way function.

Block ciphers:

- Most encryption algorithms operate on fixed-size blocks of data; 64 bits is a popular size for the blocks. A message is subdivided into blocks, the last block is padded to the standard length if necessary and each block is encrypted independently.
- ***Cipher block chaining:*** In cipher block chaining mode, each plaintext block is combined with the preceding ciphertext block using the exclusive-or operation (XOR) before it is encrypted (Figure 4.8.1). On decryption, the block is decrypted and then the preceding encrypted block (which should have been stored for this purpose) is XOR-ed with it to obtain the new plaintext block. This works because the XOR operation is its own inverse – two applications of it produce the original value.
- CBC is intended to prevent identical portions of plaintext encrypting to identical pieces of ciphertext. But there is a weakness at the start of each sequence of blocks – if we open encrypted connections to two destinations and send the same message, the encrypted sequences of blocks will be the same, and an eavesdropper might gain some useful information from this.
- To prevent this, we need to insert a different piece of plaintext in front of each message. Such text is called an initialization vector.
- A timestamp makes a good initialization vector, forcing each message to start with a different plaintext block. This, combined with CBC operation, will

result in different ciphertexts even for two identical plaintexts.

- The use of CBC mode is restricted to the encryption of data that is transferred across a reliable connection. Decryption will fail if any blocks of ciphertext are lost, since the decryption process will be unable to decrypt any further blocks.
- It is therefore unsuitable for use in applications such as those in which some data loss can be tolerated. A stream cipher should be used in such circumstances.

Stream ciphers:

- For some applications, such as the encryption of telephone conversations, encryption in blocks is inappropriate because the data streams are produced in real time in small chunks. Data samples can be as small as 8 bits or even a single bit, and it would be wasteful to pad each of these to 64 bits before encrypting and transmitting them. Stream ciphers are encryption algorithms that can perform encryption incrementally, converting plaintext to ciphertext one bit at a time.

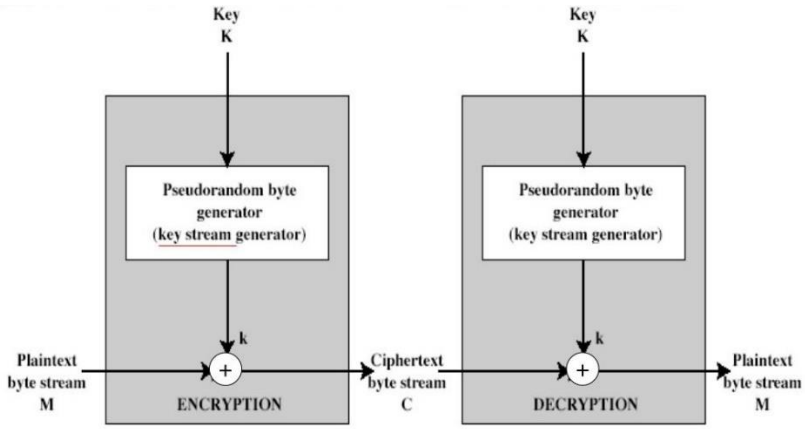


Figure 4.8. Stream cipher

- The idea is analogous to a technique used in the intelligence community to foil eavesdroppers, where 'white noise' is played to hide the conversation in a room while still recording the conversation. If the noisy room sound and the white noise are recorded separately, the conversation can be played back without noise by subtracting the white noise recording from the noisy room recording.

Secret-key (symmetric) algorithms

- Many cryptographic algorithms have been developed and published in recent years. Schneier [1996] describes more than 25 symmetric algorithms, many of which he identifies as secure against known attacks. Here we have room to describe only three of them. We have chosen the first, TEA, for the simplicity of its design and implementation, and we use it to give a concrete illustration of the nature of such algorithms.
- DES was a US national standard for many years, but it is now largely of historical interest because it's 56-bit keys are too small to resist brute-force attack with modern hardware. IDEA uses a 128-bit key. It is one of the most effective symmetric block encryption algorithms and a good all-round choice for bulk encryption.
- In 1997, the US National Institute for Standards and Technology (NIST) issued an invitation for proposals for an algorithm to replace DES as a new US ***Advanced Encryption Standard (AES).*** In October 2000 the winner was selected from 21 algorithms submitted by cryptographers from 11 countries.
- The winning Rijndael algorithm was chosen for its combination of strength and efficiency.

4.9 KERBEROS

- Kerberos is a computer network security protocol that authenticates service requests between two or more trusted hosts across an untrusted network, like the internet. It uses secret-key cryptography and a trusted third party for authenticating client-server applications and verifying users' identities.
- Initially developed by the Massachusetts Institute of Technology (MIT) for Project Athena in the late '80s, Kerberos is now the default authorization technology used by Microsoft Windows. Kerberos implementations also exist for other operating systems such as Apple OS, FreeBSD, UNIX, and Linux.
- Microsoft rolled out its version of Kerberos in Windows 2000, and it's become the go-to protocol for websites and single sign-on implementations over different platforms. The Kerberos Consortium maintains the Kerberos as an open-source project.
- The protocol derives its name from the legendary three-headed dog Kerberos (also known as Cerberus) from Greek myths, the canine guardian to the entrance to the underworld. Kerberos had a snake tail and a particularly bad temper and, despite one notable exception, was a very useful guardian.
- But in the protocol's case, the three heads of Kerberos represent the client, the server, and the Key Distribution Center (KDC). The latter functions as the trusted third-party authentication service.
- Users, machines, and services that use Kerberos depend on the KDC alone, which works as a single process that provides two functions: authentication and ticket-granting. KDC "tickets" offer authentication to all parties, allowing nodes to verify their identity securely.
- The Kerberos authentication process employs a conventional shared secret cryptography that prevents packets traveling across the network from being read or altered, as well as protecting messages from eavesdropping and replay (or playback) attacks.

What is Kerberos Used For?

- Although Kerberos is found everywhere in the digital world, it is employed heavily on secure systems that depend on reliable auditing and authentication features. Kerberos is used in Posix authentication, and Active Directory, NFS, and Samba. It's also an alternative authentication system to SSH, POP, and SMTP.

The Benefits of Kerberos Authentication

Kerberos brings a host of advantages to any cybersecurity setup. Its advantages include:

- ✓ ***Effective Access Control:*** Kerberos gives users a single point to keep track of logins and security policy enforcement.
- ✓ ***Limited Lifetime for Key Tickets:*** Each Kerberos ticket has a timestamp, lifetime data, and authentication duration controlled by the administrator.
- ✓ ***Mutual Authentication:*** Service systems and users can authenticate each other.
- ✓ ***Reusable Authentication:*** Kerberos user authentication is reusable and durable, requiring each user to get verified by the system just once. As long as the ticket is in effect, the user won’t have to keep entering their personal information for authentication purposes.
- ✓ ***Strong and Diverse Security Measures:*** Kerberos security authentication protocols employ cryptography, multiple secret keys, and third-party authorization, creating a strong, secure defense. Passwords do not get sent over networks, and all secret keys are encrypted.

Protocol Flow Overview

Principal entities involved in the typical Kerberos workflow:

- ✓ ***Client:*** The client acts on behalf of the user and initiates communication for a

service request

- ✓ ***Server:*** The server hosts the service the user wants to access
- ✓ ***Authentication Server (AS):*** The AS performs the desired client authentication. If the authentication happens successfully, the AS issues the client a ticket called TGT (Ticket Granting Ticket). This ticket assures the other servers that the client is authenticated
- ✓ ***Key Distribution Center (KDC):*** In a Kerberos environment, the authentication server logically separated into three parts: A database (db), the Authentication Server (AS), and the Ticket Granting Server (TGS). These three parts, in turn, exist in a single server called the Key Distribution Center
- ✓ ***Ticket Granting Server (TGS):*** The TGS is an application server that issues service tickets as a service

  Now let's break down the protocol flow.

  First, there are three crucial secret keys involved in the Kerberos flow. There are unique secret keys for the client/user, the TGS, and the server shared with the AS.

  - ➢ ***Client/user:*** Hash derived from the user's password
  - ➢ ***TGS secret key:*** Hash of the password employed in determining the TGS
  - ➢ ***Server secret key:*** Hash of the password used to determine the server providing the service.

UNIT V CASE STUDIES

Distributed Object Based System – CORBA – Distributed File System – Sun NFS – Andrew File System – Enterprise Java Beans – Fractals – JINI.

5.1 DISTRIBUTED OBJECT BASED SYSTEM

- In distributed object- based systems, the notion of an object plays a key role in establishing distribution transparency. In principle, everything is treated as an object and clients are offered services and resources in the form of objects that they can invoke**.**

- Distributed objects form an important paradigm because it is relatively easy to hide distribution aspects behind an object's interface. Furthermore, because an object can be virtually anything, it is also a powerful paradigm for building systems.

Distributed Objects

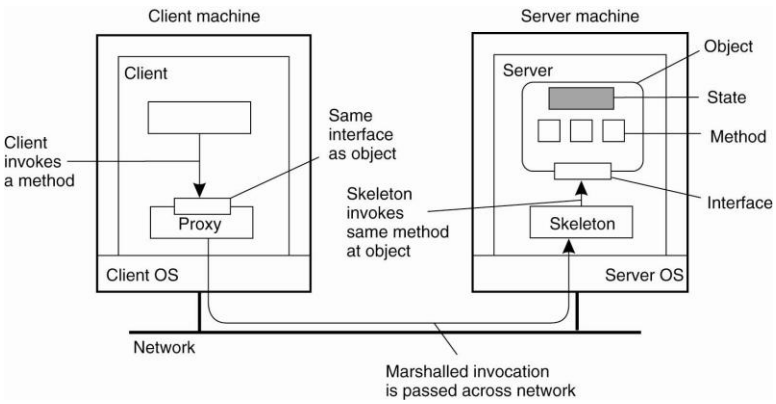
- The key feature of an object is that it encapsulates data, called the state, and the operations on those data, called the methods.

- Methods are made available through an interface. It is important to understand that there is no "legal" way a process can access or manipulate the state of an object other than by invoking methods made available to it via an object's interface.

- An object may implement multiple interfaces. Likewise, given an interface definition, there may be several objects that offer an implementation for it.

- This separation between interfaces and the objects implementing these interfaces is crucial for distributed systems. A strict separation allows us to place an interface at one machine, while the object itself resides on another machine. This organization, which is shown in Fig. 5.1., is commonly referred to as a distributed object.

Fig 5.1. Common organization of a remote object with client-side proxy.



- **The Client:** When a client binds to a distributed object. An implementation of the object's interface, called a proxy, is then loaded into the client's address space. A proxy is analogous to a client stub in RPC systems.

- The only thing it does is marshal method invocations into messages and unmarshal reply messages to return the result of the method invocation to the client.

- The actual object resides at a server machine, where it offers the same interface as it does on the client machine.

- Incoming invocation requests are first passed to a server stub, which unmarshals them to make method invocations at the object's interface at the server.

- **The Server:** The server stub is also responsible for marshaling replies and forwarding reply messages to the client side proxy.

- The server-side stub is often referred to as a skeleton as it provides the bare means for letting the server middleware access the user-defined objects.

- In practice, it often contains incomplete code in the form of a language-specific class that needs to be further specialized by the developer.

- A characteristic of most distributed objects is that their state is not distributed: it resides at a single machine. Only the interfaces implemented by the object are made available on other machines.

- Such objects are also referred to as remote objects. In a general distributed object, the state itself may be physically distributed across multiple machines, but this distribution is also hidden from clients behind the object's interfaces.

Compile- Time versus Runtime Objects

- Objects in distributed systems appear in many forms. The most obvious form is the one that is directly related to language-level objects such as those supported by Java, C++, or other object-oriented languages, which are referred to as compile-time objects.

- In this case, an object is defined as the instance of a class. A class is a description of an abstract type in terms of a module with data elements and operations on that data (Meyer, 1997). Using compile-time objects in distributed systems often makes it much easier to build distributed applications. For example, in Java:

    - An object can be fully defined by means of its class and the interfaces that the class implements.

    - Compiling the class definition results in code that allows it to instantiate Java objects.

- The interfaces can be compiled into client-side and server-side stubs, allowing the

- Java objects to be invoked from a remote machine.

  - A Java developer can be largely unaware of the distribution of objects: he sees only Java programming code.

- The obvious drawback of compile-time objects is the dependency on a particular programming language.

- An alternative way of constructing distributed objects is to do this explicitly during runtime. This approach is independent of the programming language in which distributed applications are written. In particular, an application may be constructed from objects written in multiple languages.

- When dealing with runtime objects, how objects are actually implemented is basically left open. The essence is how to let such an implementation appear to be an object whose methods can be invoked from a remote machine.

- A common approach is to use an object adapter, which acts as a wrapper around the implementation with the sole purpose to give it the appearance of an object. The adapter allows an interface to be converted into something that a client expects.

- An implementation of an interface can then be registered at an adapter, which can subsequently make that interface available for (remote) invocations. The adapter will take care that invocation requests are carried out and thus provide an image of remote objects to its clients.

Persistent and Transient Objects

- A **persistent object** continues to exist even if it is currently not contained in the address space of any server process. In other words, a persistent object is not dependent on its current server.

- In practice, this means that the server that is currently managing the persistent object, can store the object's state on secondary storage and then exit. Later, a newly started server can read the object's state from storage into its own address space, and handle invocation requests.

- In contrast, **a transient object** is an object that exists only as long as the server that is hosting the object is. As the server exits, the object ceases to exist as well. There used to be much controversy about having persistent objects; some people believe that transient objects are enough. To take the discussion away from middleware issues, most object-based distributed systems simply support both types.

5.2 CORBA

Overview of CORBA

- CORBA : Common Object Request Broker Architecture

  Background:
- Developed by the Object Management Group (OMG) in response to industrial demands for object based middleware.
- Currently in versions #2.4 and #3.
- CORBA is a specification: different implementations of CORBA exists.
- Very much of work of a committee: There are over 800 members of the OMG and many of them have to say in what CORBA should look like.
- CORBA provides a simple distributed object model, with specifications for many supporting services it may be here to say(for a couple of years).

Architecture of COBRA

- The object Request Broker (ORB) forms the core of any CORBA distributed system.
- Horizontal facilities consists of general-purpose high-level services that are independent of applications domain.
  - ✓ User interface
  - ✓ Information management
  - ✓ System management
  - ✓ Task management
- Vertical facilities consists of high-level services that are targeted to a specific application domain such as electronic commerce, banking, and manufacturing.

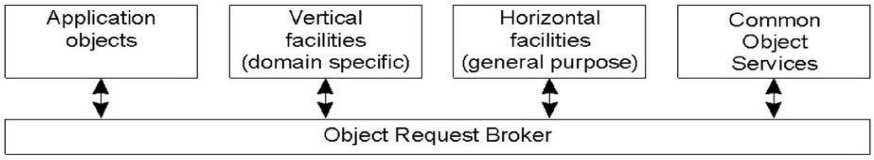


Fig 5.2. Global Architecture of CORBA

Object Model

- ✓ CORBA has a traditional remote object model in which an object residing at an object server is remote accessible through proxies.
- ✓ All CORBA specifications are given by means of interface descriptions, expressed in an **Interface Definition Language (IDL).**
  - An interface is a collection of methods, and objects specify which interfaces they implement.
  - It provides a precise syntax for expressing methods and their parameters.

CORBA follows an interface based approach to objects:

- Not the objects, but interfaces are the really important entities.
- An object may implement one or more interfaces.
- Interface descriptions can be stored in an interface repository, and looked up at runtime.
- Mappings from IDL to specific programming are part of the CORBA specification (languages include C, C++, Smalltalk, COBOL, ADA and Java.

- ✓ In DCOM and Globe interfaces can be specified at a lower level in the form of tables called binary interfaces.
- ✓ **Object Request Broker (ORB):** CORBA's object broker that connects clients, objects and services.
- ✓ **Proxy/Skeleton:** Precompiled code that takes care of (un) marshaling invocations and results.
- ✓ **Dynamic Invocation/Skeleton Interface (DII/DSI):** To allow clients to construct invocation requests at runtime instead of calling methods at a proxy, and having the server side reconstruct those request into regular method invocation.
- ✓ **Object Adapter:** Server side code that handles incoming invocation requests.
- ✓ **Interface Repository:**
  - – Database containing interface definitions and which can be queried at runtime.
  - – Whenever an interface definition is compiled, the IDL compiler assigns a repository identifier to that interface.
- ✓ **Implementation Repository:**
  - – Database containing the implementation (code and possibly also state) of objects.
  - – Given an object reference, an object adaptor could contact the implementation repository to find out exactly what needs to be done.

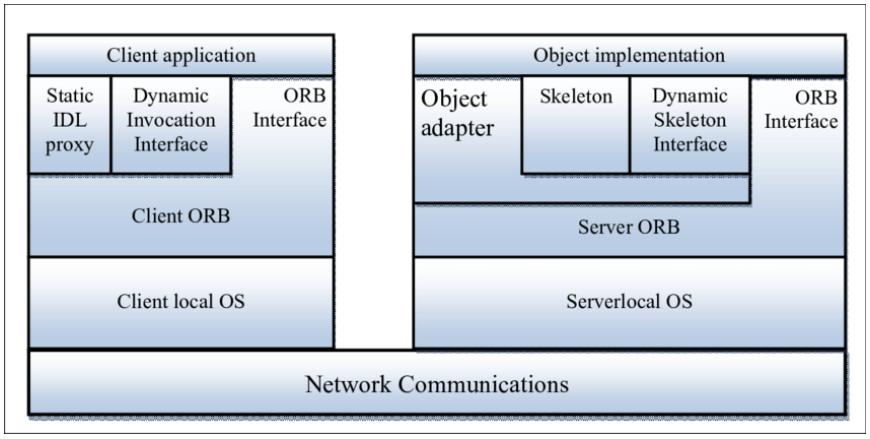


Fig. 5.3. The general organization of a CORBA system

5.3 DISTRIBUTED FILE SYSTEM

- ❖ A Distributed File System (DFS) as the name suggests, is a file system that is distributed on multiple file servers or multiple locations. It allows programs to access or store isolated files as they do with the local ones, allowing programmers to access files from any network or computer.
- ❖ The main purpose of the Distributed File System (DFS) is to allow users of physically distributed systems to share their data and resources by using a Common File System. A collection of workstations and mainframes connected by a Local Area Network (LAN) is a configuration on Distributed File System.
- ❖ A DFS is executed as a part of the operating system. In DFS, a namespace is created and this process is transparent for the clients.

- DFS has two components:
  - **Location Transparency –**
    Location Transparency achieves through the namespace component.
  - **Redundancy –**
    Redundancy is done through a file replication component.
- In the case of failure and heavy load, these components together improve data availability by allowing the sharing of data in different locations to be logically grouped under one folder, which is known as the "DFS root".

- It is not necessary to use both the two components of DFS together, it is possible to use the namespace component without using the file replication component and it is perfectly possible to use the file replication component without using the namespace component between servers.

Features of DFS:

Transparency:

- **Structure transparency –**
  There is no need for the client to know about the number or locations of file servers and the storage devices. Multiple file servers should be provided for performance, adaptability, and dependability.
- **Access transparency –**
  Both local and remote files should be accessible in the same manner. The file system should be automatically located on the accessed file and send it to the client's side.
- **Naming transparency –**
  There should not be any hint in the name of the file to the location of the file. Once a name is given to the file, it should not be changed during transferring from one node to another.
- **Replication transparency –**
  If a file is copied on multiple nodes, both the copies of the file and their locations should be hidden from one node to another.

User mobility:

It will automatically bring the user's home directory to the node where the user logs in.

Performance:

Performance is based on the average amount of time needed to convince the client requests. This time covers the CPU time + time taken to access secondary storage + network access time. It is advisable that the performance of the Distributed File System be similar to that of a centralized file system.

Simplicity and ease of use:

The user interface of a file system should be simple and the number of commands in the file should be small.

High availability:

A Distributed File System should be able to continue in case of any partial failures like a link failure, a node failure, or a storage drive crash.

A high authentic and adaptable distributed file system should have different and independent file servers for controlling different and independent storage devices.

Scalability:

Since growing the network by adding new machines or joining two networks together is routine, the distributed system will inevitably grow over time. As a result, a good distributed file system should be built to scale quickly as the number of nodes and users in the system grows. Service should not be substantially disrupted as the number of nodes and users grows.

High reliability:

The likelihood of data loss should be minimized as much as feasible in a suitable distributed file system. That is, because of the system's unreliability, users should not feel forced to make backup copies of their files. Rather, a file system should create backup copies of key files that can be used if the originals are lost. Many file systems employ stable storage as a high-reliability strategy.

Data integrity:

Multiple users frequently share a file system. The integrity of data saved in a shared file must be guaranteed by the file system. That is, concurrent access requests from many users who are competing for access to the same file must be correctly synchronized using a concurrency control method. Atomic transactions are a high-level concurrency management mechanism for data integrity that is frequently offered to users by a file system.

Security:

A distributed file system should be secure so that its users may trust that their data will be kept private. To safeguard the information contained in the file system from unwanted & unauthorized access, security mechanisms must be implemented.

Heterogeneity:

Heterogeneity in distributed systems is unavoidable as a result of huge scale. Users of heterogeneous distributed systems have the option of using multiple computer platforms for different purposes.

Applications:

- ✓ **NFS:** NFS stands for Network File System. It is a client-server architecture that allows a computer user to view, store, and update files remotely. The protocol of NFS is one of the several distributed file system standards for Network-Attached Storage (NAS).
- ✓ **CIFS:** CIFS stands for Common Internet File System. CIFS is an accent of SMB. That is, CIFS is an application of SIMB protocol, designed by Microsoft.
- ✓ **SMB:** SMB stands for Server Message Block. It is a protocol for sharing a file and was invented by IMB. The SMB protocol was created to allow computers to perform read and write operations on files to a remote host over a Local Area Network (LAN). The directories present in the remote host can be accessed via SMB and are called as "shares".
- ✓ **Hadoop:** Hadoop is a group of open-source software services. It gives a software framework for distributed storage and operating of big data using the MapReduce programming model. The core of Hadoop contains a storage part, known as Hadoop Distributed File System (HDFS), and an operating part which is a MapReduce programming model.
- ✓ **NetWare:** NetWare is an abandon computer network operating system developed by Novell, Inc. It primarily used combined multitasking to run different services on a personal computer, using the IPX network protocol.

Working of DFS:

There are two ways in which DFS can be implemented:

Standalone DFS namespace –

It allows only for those DFS roots that exist on the local computer and are not using Active Directory. A Standalone DFS can only be acquired on those computers on which it is created. It does not provide any fault liberation and cannot be linked to any other DFS. Standalone DFS roots are rarely come across because of their limited advantage.

Domain-based DFS namespace –

It stores the configuration of DFS in Active Directory, creating the DFS namespace root accessible at \\<domainname>\<dfsroot> or \\<FQDN>\<dfsroot>

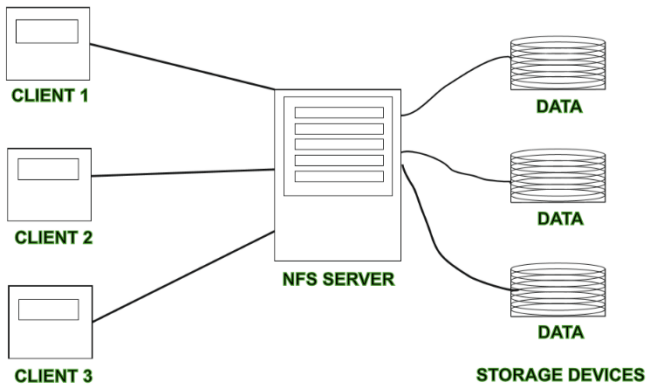


Fig 5.4. Distributed File System

Advantages:

- ✓ DFS allows multiple user to access or store the data.
- ✓ It allows the data to be share remotely.
- ✓ It improved the availability of file, access time, and network efficiency.
- ✓ Improved the capacity to change the size of the data and also improves the ability to exchange the data.
- ✓ Distributed File System provides transparency of data even if server or disk fails.

Disadvantages:

- ✓ In Distributed File System nodes and connections needs to be secured therefore we can say that security is at stake.
- ✓ There is a possibility of loss of messages and data in the network while movement from one node to another.
- ✓ Database connection in case of Distributed File System is complicated.
- ✓ Also handling of the database is not easy in Distributed File System as compared to a single user system.
- ✓ There are chances that overloading will take place if all nodes tries to send data at once.

5.4 Sun NFS

- ❖ The earliest successful distributed system could be attributed to Sun Microsystems, which developed the Network File System (NFS). NFSv2 was the standard protocol followed for many years, designed with the goal of simple and fast server crash recovery. This goal is of utmost importance in multi-client and single-server based network architectures because a single instant of server crash means that all clients are unserviced. The entire system goes down.
- ❖ **Stateful protocols** make things complicated when it comes to crashes. Consider a client A trying to access some data from the server. However, just after the first read, the server crashed. Now, when the server is up and running, client A issues the second read request. However, the server does not know which file the client is referring to, since all that information was temporary and lost during the crash.
- ❖ **Stateless protocols** come to our rescue. Such protocols are designed so as to not store any state information in the server. The server is unaware of what the clients are doing — what blocks they are caching, which files are opened by them and where their current file pointers are. The server simply delivers all the information that is required to service a client request. If a server crash happens, the client would simply have to retry the request. Because of their simplicity, NFS implements a stateless protocol
- ❖ **File Handles:** NFS uses file handles to uniquely identify a file or a directory that the current operation is being performed upon. This consists of the following components:
    - ✓ **Volume Identifier** – An NFS server may have multiple file systems or

partitions. The volume identifier tells the server which file system is being referred to.

- ✓ **Inode Number** – This number identifies the file within the partition.
- ✓ **Generation Number** – This number is used while reusing an inode number.

❖ **File Attributes:** "File attributes" is a term commonly used in NFS terminology. This is a collective term for the tracked metadata of a file, including file creation time, last modified, size, ownership permissions etc. This can be accessed by calling stat () on the file.

❖ **NFSv2 Protocol:**

- ✓ The LOOKUP protocol message is used to obtain the file handle for further accessing data. The NFS mount protocol helps obtain the directory handle for the root (/) directory in the file system.
- ✓ If a client application opens a file /abc.txt, the client-side file system will send a LOOKUP request to the server, through the root (/) file handle looking for a file named abc.txt. If the lookup is successful, the file attributes are returned.
- ✓ Some of the common protocol messages are listed below.

Message	Description
NFSPROC_GETATTR	Given a file handle, returns file attributes.
NFSPROC_SETATTR	Sets/updates file attributes.
NFSPROC_LOOKUP	Given file handle and name of the file to look up, returns file handle.
NFSPROC_READ	Given file handle, offset, count data and attributes, reads the data.
NFSPROC_WRITE	Given file handle, offset, count data and attributes, writes data into the file.
NFSPROC_CREATE	Given the directory handle, name of file and attributes, creates a file.
NFSPROC_REMOVE	Given the directory handle and name of file, deletes the file.
NFSPROC_MKDIR	Given directory handle, name of directory and attributes, creates a new directory.

Client-Side Caching:

- ✓ To improve performance of NFS, distributed file systems cache the data as well as the metadata read from the server onto the clients. This is known as client-side caching. This reduces the time taken for subsequent client accesses. The cache is also used as a temporary buffer for writing. This helps improve efficiency even more since all writes are written onto the server at once.

5.5 ANDREW FILE SYSTEM

- ❖ Andrew File System (AFS) is a distributed network file system developed by Carnegie Mellon University. Enterprises use an AFS to facilitate stored server file access between AFS client machines located in different areas. AFS supports reliable servers for all network clients accessing transparent and homogeneous namespace file locations.
- ❖ An AFS may be accessed from a distributed environment or location independent platform. A user accesses an AFS from a computer running any type of OS with Kerberos authentication and single namespace features. Users share files and applications after logging into machines that interact within the Distributed Computing Infrastructure (DCI).
- ❖ In distributed networks, an AFS relies on local cache to enhance performance and reduce workload. For example, a server responds to a workstation request and stores the data in the workstation's local cache. When the workstation requests the same data, the local cache fulfills the request.
- ❖ AFS networks employ server and client components, as follows:
  - ➢ A client may be any type of machine that generates requests for AFS server files stored on a network.
  - ➢ After a server responds and sends a requested file, the file is stored in the client machine's local cache and presented to the client machine user.
  - ➢ When a user accesses the AFS, the client uses a callback mechanism to send all changes to the server. Frequently used files are stored for quick access in the client machine's local cache.

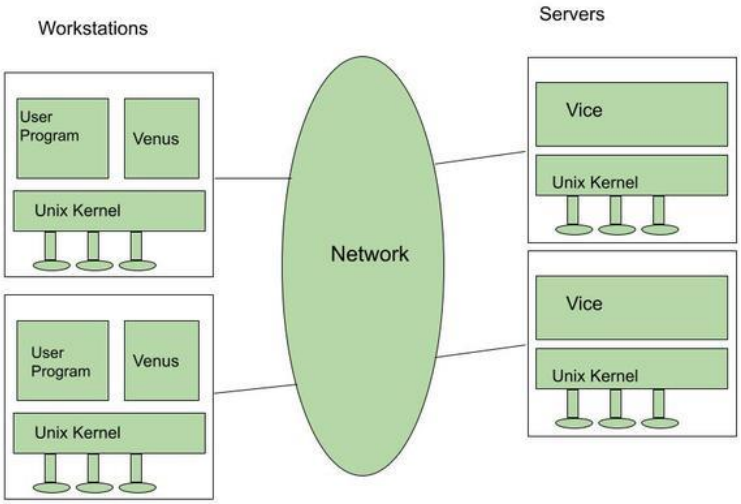


Fig 5.5. AFS Architecture

- ❖ AFS equips users with multiple access control permissions, as follows:
  - ▪ **Look Up (1):** Users may access and list AFS directory and subdirectory content and review a directory's access control list (ACL).
  - ▪ **Insert (i):** Users may add new subdirectories or files.
  - ▪ **Delete (d):** Users may remove directory files.

- **Administer (a):** Users may modify the home directory's ACL.
- **Read (r):** Users may view file directory or subcategory contents, as AFS supports the standard Unix Owner Read permission control set.
- **Write (w):** Users may modify or write files, as AFS supports the Unix Owner Write permission control set.
- **Look (k):** Processors may use the directory to execute programs requiring Flock files.

5.6 ENTERPRISE JAVA BEANS (EJB)

- An EJB is essentially a Java object that is hosted by a special server offering different ways for remote clients to invoke that object. Crucial is that this server provides the support to separate application functionality from systems-oriented functionality. The latter includes functions for looking up objects, storing objects, letting objects be part of a transaction, and so on.
- The important issue is that an EJB is embedded inside a container which effectively provides interfaces to underlying services that are implemented by the application server. The container can more or less automatically bind the EJB to these services, meaning that the correct references are readily available to a programmer.
- Typical services include those for remote method invocation (RMI), database access (JDBC), naming (JNDI), and messaging (JMS). Making use of these services is more or less automated.

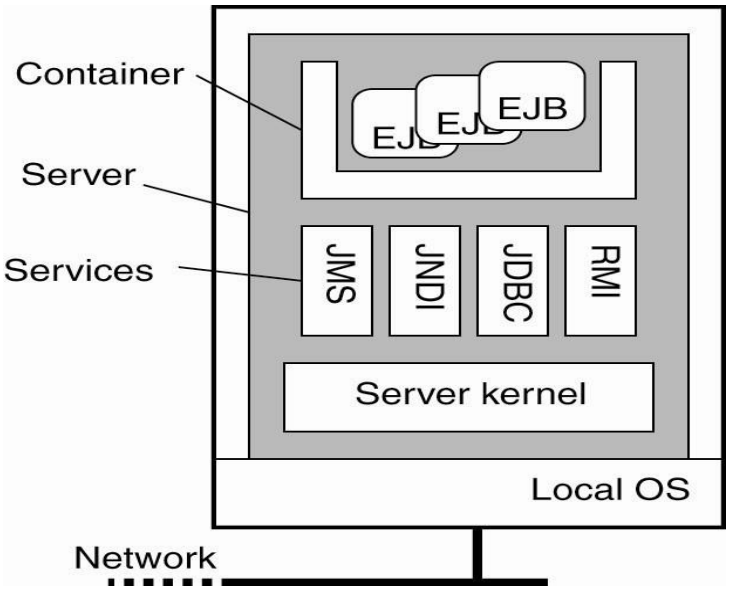


Fig 5.6. General Architecture of an EJB Server

- To run EJB application, you need an application server (EJB Container) such as Jboss, Glassfish, Weblogic, Websphere etc.
- It performs:
  - ✓ life cycle management,
  - ✓ security,
  - ✓ transaction management,
  - ✓ Object pooling.
- EJB application is deployed on the server, so it is called server side component also.

When use Enterprise Java Bean?

- **Application needs Remote Access.** In other words, it is distributed.
- **Application needs to be scalable.** EJB applications supports load balancing, clustering and fail-over.
- **Application needs encapsulated business logic.** EJB application is separated from presentation and persistent layer.

Types of EJBs

- ✓ **Stateless session beans** are transient objects that are invoked once, does its work, after which it discards any information it maintains to perform the service it offered to a client.
- ✓ **Stateful session beans** maintain client-related state.
- ✓ **Entity beans** can be considered to be a long-lived persistent object. Such an entity bean will generally be stored in a database, and likewise, will also be part of distributed transactions.
- ✓ **Message-driven beans** are used to program objects that should react to incoming messages (and likewise, be able to send messages). They cannot be invoked directly by a client, but rather fit into a publish-subscribe way of communication.

Difference between RMI and EJB

RMI	EJB
In RMI, middleware services such as security, transaction management, object pooling etc. need to be done by the java programmer.	In EJB, middleware services are provided by EJB Container automatically.
RMI is not a server-side component. It is not required to be deployed on the server.	EJB is a server-side component, it is required to be deployed on the server.
RMI is built on the top of socket programming.	EJB technology is built on the top of RMI.

Disadvantages of EJB

- ✓ Requires application server
- ✓ Requires only java client. For other language client, you need to go for web service.
- ✓ Complex to understand and develop EJB applications.

5.7 FRACTALS

Overview

- ❖ FRACTAL (framework for distributed computing to trace large accurate lineages) is a deep distributed computing framework that reconstruct extremely large lineages of nucleotide sequences using a software tool of choice (Figure 5.7).
- ❖ From an input sequence pool, a given number of sequences are first randomly subsampled (Step 1). Their sample lineage tree is reconstructed with a rooting or provisional rooting sequence by lineage estimation software of choice (Step 2). (Note that while an outgroup or a true ancestral sequence, for cell lineage tracing, is preferred as the rooting sequence if available, any sequence can be used for unrooted tree estimation).

- Each of the remaining input sequences is then mapped to its most proximal branch of the sample tree by phylogenetic placement (Step 3).
- If all of the input sequences are mapped on the downstream branches of the sample tree to separate them into multiple distinct clades, their upstream lineage is considered to be resolved (Step 4), and the sequence group in each downstream clade is recursively subjected to the first process in a distributed computing node (Step 5).
- If any sequences is mapped on the root branch, which does not allow the grouping of input sequences into clades, the phylogenetic placement is repeated against a new sample tree generated for sequences randomly chosen from a union of the previous subsampled sequences and the "problematic" sequences (Step 6).
- This process generates a new sample tree in a biased manner such that it harbors the previous problematic sequences in its leaves and decreases the probability of acquiring problematic sequences in the subsequent phylogenetic placement step.
- The retrial process is repeated until the problem is solved, but only up to a given threshold number of times than and as long as the number of problematic sequences continues to be reduced in every retrial step.
- When the retrial cycle stops without completely solving the problem, the remaining problematic sequences are discarded, and the other sequence sets are separated into distinct clades and subjected to the first process in separate computing nodes. Accordingly, FRACTAL generates a hierarchy of expanding parallel computing trajectories, where each distributed computing job recursively generates a large set of successive jobs.
- When the number of input sequences is reduced to a certain threshold (hereafter called the naïve computing threshold), the remaining lineage is directly reconstructed using the designated software, and the operation terminates for this computing trajectory (Step 7).

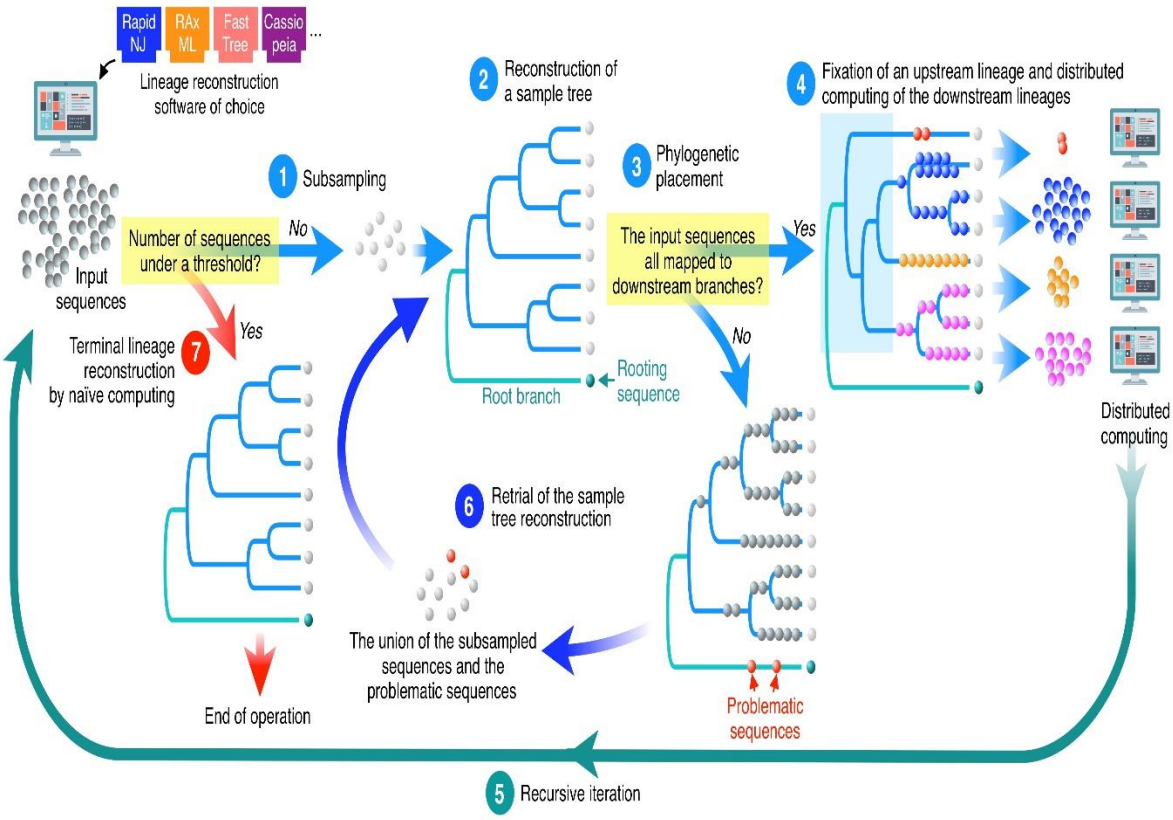


Fig 5.7. Schematic diagram of FRACTAL.

- For unrooted lineage estimation, the provisional rooting sequence can be removed after completing the entire computation. Thus, FRACTAL enables the efficient

reconstruction of large lineages by distributed computing while utilizing limited computing power and memory per node.

- Multiple sequence alignment (MSA) of the input sequences is not a prerequisite in FRACTAL (Figure 5.8). When unaligned sequences are queried to FRACTAL for lineage reconstruction, only the sequences subsampled from the input sequences are aligned by MSA for the sample tree reconstruction in each cycle.
- For phylogenetic placement, each of the remaining input sequences is mapped to the sample tree through "plus-one" alignment to the MSA result that reconstructed the sample tree. This strategy enables the scalable lineage reconstruction of unaligned sequences.

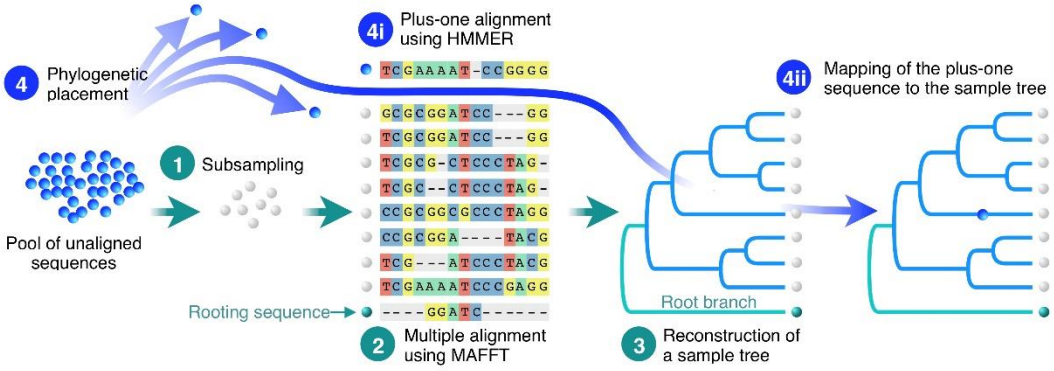


Figure 5.8. Evolutionary lineage reconstruction without preliminary MSA

- A given number of sequences are first randomly subsampled from the input sequences (Step 1). The subsampled sequences are aligned with a common root sequence by MSA using MAFFT (Step 2) and a sample tree is reconstructed by a software tool of choice (Step 3).
- Each of the remaining input sequences are then independently added to the MSA result by "plus-one" alignment using HMMER (Step 4i) and placed on the sample tree (Step 4ii).

Supported Environment

- FRACTAL can be executed on Linux OS
- The distributed computing mode of FRACTAL requires UGE (Univa Grid Engine.

Software Dependency

1. Python3 (version: 3.7.0 or later) with Biopython (version: 1.76) module required
2. RAxML (raxmlHPC-PTHREADS-SSE3 and raxmlHPC-SSE3) (version: 8.2.12) required
3. EPA-ng (version: 0.3.5) required
4. Seqkit (version: 0.11.0) required
5. Trimal (version: 1.4.1) required
6. RapidNJ (version: 2.3.2) optional; for NJ method
7. FastTreeMP (version: 2.1.10) optional; for ML method
8. MAFFT (version: 7.464) optional; for lineage reconstruction with the incremental MSA method.
9. HMMER (version: 3.3) optional; for lineage reconstruction with the incremental MSA method.

Software Installation

Each installation step will take less than 1 min

Installation of FRACTAL

1. Download FRACTAL by

```
git clone https://github.com/yachielab/FRACTAL.git
```

or you can also obtain FRACTAL as follows

```
wget https://github.com/yachielab/FRACTAL/archive/master.zip
unzip master.zip
```

2. Add the absolute path of FRACTAL directory to $PATH
3. Make FRACTAL executable

```
chmod u+x FRACTAL
```

Installation of Anaconda (required)

1. Execute the following commands

```
wget https://repo.anaconda.com/archive/Anaconda3-2018.12-Linux-x86_64.sh
bash Anaconda3-2018.12-Linux-x86_64.sh
bash
```

2. Set $PATH to anaconda3/bin.

Sample Commands

The FRACTAL package contains an example input file in the examples directory so users can check the software functions as follows:

Example 1

Lineage estimation of the sequences in the test input file test.fa with the default parameter set without distributed computing. This will take several minutes.

```
FRACTAL -i test.fa
```

Input:

test.fa (FASTA format)

Output:

FRACTALout.nwk (Newick format) will be created in your current working directory

FRACTAL Usage

Usage:

```
FRACTAL.sh
[-v] [-h] [-i input_file] [-o output_file_path] [-f output_file_name]
[-m method] [-a "options"] [-s script_file_name] [-k sequence_number]
[-u] [-w alignment_frequency] [-b model_name]
[-p placement_method] [-x iteration_number] [-t sequence_number]
[-d job_number] [-c thread_number] [-e] [-r integer] [-n file_path] [-E]
[-O qsub_option] [-I first_qsub_option] [-A last_qsub_option] [-j job_name]
[-l iteration_upper_limit] [-g]
```

5.8 JINI

- Jini is a service-oriented architecture that defines a programming model that both exploits and extends Java technology. This programming model enables the construction of secure, distributed systems consisting of federations of well-behaved network services.
- Jini helps to build networks that are scalable and flexible, which are required attributes in distributed computing scenarios.
- Jini's main objective is to shift the focus of distributed computing from a disk-drive-

oriented approach to a network-adaptive approach by developing scalable, evolvable and flexible dynamic computing environments.

- Jini makes resources over a network look like local resources.
- Jini may also be referred to as Apache River.

History of JINI

1. Jini was introduced in July 1998 by Sun Microsystems. It consists of a set of specifications and a starter kit, which includes the implementation of Jini technology. Both are released under the open-source Apache 2.0 License.
2. Jini is implemented in the Java programming language and is similar to Java remote method invocation, except that it is more advanced.
3. Jini provides desirable features and facilities like nonprotocol dependence and code mobility.
4. In fact, code mobility is the key concept. Jini not only allows for the addition of printers, storage and other devices to a network, it also allows the devices to be detected automatically over the network without having to reboot the system.
5. Hardware devices declare to their own operating systems as well as to other computers, devices and users on the network that they have been added and are available for use. This is possible because the devices define themselves to a network device registry soon after they have been added.

Dynamic Network Architecture

- Traditional network architecture is static in nature. Network designers know in advance which computer hardware and software will participate in a specific customer solution, and the network infrastructure is built in accordance with this concept.
- For example, in the Client/Server network architecture, specific server resources are designated to be served to clients on request, while other software units are indigenous on the clients.
- Dynamic network architecture recognizes that to achieve the best customer solution, it is better not to decide in advance the specific hardware and software that will participate in the solution.
- Because of the multiplicity of software and hardware available on the network that could participate in the solution, it is advantageous to defer the decision until the software or hardware is actually required.
- The idea is to have the solution itself seek on the network the components and resources it requires, choosing them on the basis of availability, performance and quality. Should the selected components and resources degrade or fail during execution, the solution can replace them while continuing to operate, increasing the reliability of the solution.
- It is difficult to draw a network connection diagram for such a solution, because it changes constantly during the solution's execution as the system connects, disconnects and reconnects to various network hardware and software – hence dynamic network architecture.

Service-Oriented Architecture

- **Service-oriented architecture (SOA)** is the dynamic network architecture discussed above as applied to software components. An executing solution seeks on the network the software services it requires, choosing them on the basis of availability, performance and quality.

- SOA takes the existing software components residing on the network and allows them to be published, invoked and discovered by each other. SOA allows a software programmer to model programming problems in terms of services offered by components to anyone, anywhere over the network.

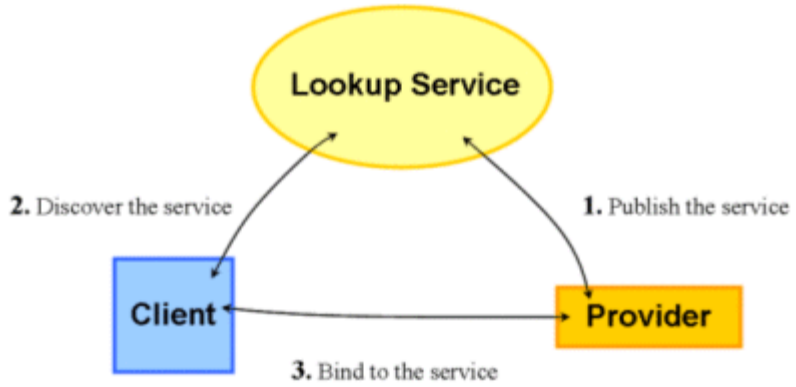


Fig 5.9. Jini Architecture

The Jini architecture is divided into three main parts:

1. **Client:** The user who accesses the resources shared over a network
2. **Server:** The system to which the resources are attached
3. **Lookup Service:** Services for resources such as printers, storage devices and speakers, which are attached to the server and made available to clients over the network.

The Jini technology includes the following services:

- Lookup Service
- Discovery and Join
- Entry Interface
- Leasing
- Remote Events
- Transaction Processing Service

Lookup Service

- In a dynamic federation in which services can join and leave at any time, up-to-date lookup registry is necessary.
- The Lookup Service is the primary means for client programs to find services in the federation, and is the foundation for providing user interfaces through which users and administrators can discover and interact with services in the federation. Descriptive service information stored in the Lookup Service helps clients decide

whether to use the service or not.

Discovery and Join

- Because of the multiplicity of federations on the network and their dynamic nature, a client requires assistance to find the Lookup Service of the appropriate federation.
- Entities that wish to start participating in a federation, must first obtain references to one or more lookup services. Jini technology discovery allows acquisition of these references. Joining the network enable a service to publicly advertise itself with it's type and attributes allows to usefully work with services in a federation.
- The discovery process involves three closely related protocols:

1. **Unicast discovery protocol** – used to establish communications over a local area or wide area network (WAN) with a Lookup Service whose address is known in advance.
2. **Multicast request protocol** – used to discover one or more Lookup Services on a local area network (LAN).
3. **Multicast announcement protocol** – used to announce the presence of a Lookup Service on a local network.

Entry Interface

- A dynamic federation requires simple but powerful search schemes and methods. The Entry interface makes it possible to use template-matching search schemes that are suited to network-wide searches.
- An Entry is not a service used per service but rather an important interface used in several services. The Entry interface and its associated UnusableEntryException are used by the Lookup Service to represent service-specific attributes (see ServiceRegistration), and by the JavaSpaces interface to mark Entries. Entries are used in distributed algorithms for which exact template-match lookup semantics are useful.
- There are three types of service operations that use Entries:

1. **Store operations** – operations that store one or more Entries, usually for future matches.
2. **Match operations** – operations that search for Entries that match one or more templates.
3. **Fetch operations** – operations that return one or more Entries.

Leasing

- In distributed applications on a network, where there may be partial failures of the network or of components, there needs to be a way for components to be timed out if they have failed, or have become unreachable.
- The leasing mechanism addresses the problem. It is used between applications to give access to resources over a period of time in an agreed manner. Leases are requested for a period of time.
- The client call can request the length or duration of a lease in the following ways:
  - Specifying the Lease interface constant FOREVER, requests a lease that never

expires. When granted such a lease, the lease holder is responsible for ensuring that the leased resource is freed when no longer needed.

- Specifying the Lease interface constant ANY, indicates that no particular lease time is desired and that the grantor of the lease should supply a time that is most convenient for the grantor.
- Specifying a particular duration. The lease grantor is required to grant a lease of no more than the requested period of time.

Remote Events

- For dynamic network environments, the notifications of events from remote objects may arrive in different orders on different clients, or may not arrive at all. Further, the object requiring the event notification may not want to have that notification as soon as possible, but only on a schedule determined by the recipient and may want the notification of the event to be sent to a third network party.
- The Jini technology remote event model extends the local event model to dynamic network architectures. It allows an object in one network location to register interest in an event occurring in an object in another network location, and to designate a third network object to receive a notification when an event of that kind occurs.
- For the three network entities:
  - Registrant – the object registering the event
  - Event generator – the object experiencing the event
  - Remote event listener – a third object designated to receive the event notification

Remote events operate as follows:

1. The registrant registers the remote event listener with the event generator.
2. The event generator returns an event registration for the remote event listener to the registrant.
3. The registrant returns the event registration to the remote event listener.
4. The event generator fires a remote event to the listener to indicate the kind of event that occurred.

Transaction Processing Service

- When remote network members participate in a transaction it is important to enforce a consistent model of operations. Then, in case one member participant fails, the transaction can be reliably aborted, ensuring that no partial results are written. The Transaction Processing Service supports a two-phase commit model on a dynamic network.
- A transaction is created and overseen by a manager. A client asks the manager to create a transaction object. The transaction object created is then passed as a parameter when performing operations on a service.
- A transaction completes when any entity either commits or aborts the transaction. If a transaction commits successfully, then all operations performed under that transaction will complete. Aborting a transaction means that all operations performed under that transaction will appear never to have happened.

A Java-Based Technology

As the following block diagram shows, the Jini technology is made possible by the characteristics of the Java language, in particular:

Fig 5.10. Jini Technology

- **Polymorphic type system** – interfaces are not concrete types and can be used by a wide variety of systems.
- **Portable bytecode** – Java moves code from machine to machine and from operating system to operating system, using strong typing, class loading, and security.

❖ The figure 5.9 shows that the Jini services are layered on the Java language, which can itself be used on all major operating systems throughout a network. GigaSpaces provides also C++ and .NET applications to access deployed services.

❖ The polymorphic typing and portable code allows Java-based Jini to span the network and serve various disparate network elements. The JavaSpaces service could be considered a "super" Jini service.

Jini has the following key advantages:

1. Provides stable networking solutions
2. Helps in upgrading systems
3. Helps to keep old clients running while adding new ones
4. Helps build scalable, dynamic and flexible networks

Faculty In-charge **HOD**